

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA KHOA HỌC MÁY TÍNH



BÁO CÁO HỌC PHẦN TỐT NGHIỆP
CÔNG NGHỆ PHẦN MỀM 1

PHÂN TÍCH VÀ CÀI ĐẶT CẤU TRÚC DỮ LIỆU
ƯỚC LƯỢNG TẦN SUẤT TRÊN LUỒNG DỮ LIỆU
ỨNG DỤNG PHÁT HIỆN BẤT THƯỜNG MẠNG

Sinh viên thực hiện: Mạch Tiến Duy
MSSV: CT07N0111
Lớp: CT07
Giảng viên phụ trách: Nguyễn Minh Đế

Hồ Chí Minh, tháng 4 năm 2026

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA KHOA HỌC MÁY TÍNH



BÁO CÁO HỌC PHẦN TỐT NGHIỆP
CÔNG NGHỆ PHẦN MỀM 1

PHÂN TÍCH VÀ CÀI ĐẶT CẤU TRÚC DỮ LIỆU
ƯỚC LƯỢNG TẦN SUẤT TRÊN LUỒNG DỮ LIỆU
ỨNG DỤNG PHÁT HIỆN BẤT THƯỜNG MẠNG

Sinh viên thực hiện: Mạch Tiến Duy
MSSV: CT07N0111
Lớp: CT07
Giảng viên phụ trách: Nguyễn Minh Đế

Hồ Chí Minh, tháng 4 năm 2026

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo tốt nghiệp, em đã nhận được sự hướng dẫn và giúp đỡ tận tình từ nhiều phía. Em xin gửi lời cảm ơn chân thành tới:

ThS. Nguyễn Minh Đế, giảng viên phụ trách môn Công nghệ Phần mềm 1, đã tận tình hướng dẫn, góp ý và định hướng trong suốt quá trình thực hiện đề tài.

Các thầy cô trong Khoa Khoa học Máy tính, Học viện Kỹ thuật Mật mã đã truyền đạt kiến thức nền tảng về cấu trúc dữ liệu, giải thuật và an toàn thông tin, tạo tiền đề cho đề tài nghiên cứu này.

Gia đình và bạn bè đã luôn động viên, hỗ trợ em trong quá trình học tập và nghiên cứu.

Mặc dù đã cố gắng hoàn thiện báo cáo một cách tốt nhất, song không tránh khỏi thiếu sót. Em rất mong nhận được sự góp ý từ quý thầy cô để hoàn thiện hơn.

Hồ Chí Minh, tháng 4 năm 2026

Sinh viên thực hiện

Mạch Tiến Duy

MỤC LỤC

LỜI CẢM ƠN	i
BẢNG DIỄN GIẢI TỪ VIẾT TẮT VÀ TIẾNG ANH	vi
BẢNG DIỄN GIẢI KÝ HIỆU	ix
MỞ ĐẦU	xii
Chương 1. BÀI TOÁN	1
1.1 Bối cảnh và đặt vấn đề	1
1.2 Mô hình bài toán và yêu cầu lời giải	3
1.3 Khảo sát công trình liên quan	4
1.4 Phạm vi và định hướng giải quyết	5
1.5 Kết luận chương	5
Chương 2. CƠ SỞ LÝ THUYẾT	7
2.1 Mô hình luồng dữ liệu	7
2.1.1 Định nghĩa hình thức và ràng buộc tài nguyên	7
2.1.2 Moment của tần suất	8
2.2 Bài toán ước lượng tần suất điểm	9
2.2.1 Phát biểu bài toán và yêu cầu sai số	9
2.2.2 Giới hạn dưới về bộ nhớ	9
2.3 Hàm băm và tính chất	9
2.3.1 Họ hàm băm universal và pairwise-independent	9
2.3.2 MurmurHash3 và minh hoạ tính toán	10
2.4 Cấu trúc dữ liệu Count-Min Sketch	11
2.4.1 Mô tả	11
2.4.2 Thao tác UPDATE và QUERY	12
2.5 Đảm bảo sai số epsilon-delta: định lý và chứng minh	14
2.6 Biến thể Conservative Update	15
2.6.1 Minh họa so sánh CMS chuẩn và Conservative Update	16
2.7 Ứng dụng thực tiễn của ước lượng tần suất trên luồng	16
2.7.1 Giám sát hạ tầng mạng và đo lường phân tán	17
2.7.2 Truy vấn dữ liệu, ngôn ngữ và tìm kiếm	17

2.8	Phương pháp và công cụ sử dụng	18
2.8.1	Phương pháp nghiên cứu	18
2.8.2	Công cụ và môi trường	18
2.8.3	Phương pháp đánh giá và tái lập	19
2.9	Kết luận chương	20
Chương 3.	PHÂN TÍCH VÀ SO SÁNH CÁC GIẢI THUẬT	21
3.1	Kết luận trước, phân tích sau	21
3.2	Phương án A: Exact Hash Map	22
3.2.1	Giải thuật và độ phức tạp	22
3.2.2	Minh hoạ chạy tay và đánh giá	22
3.3	Phương án B: Misra-Gries (Heavy Hitters)	23
3.3.1	Giải thuật và lưu đồ quyết định	23
3.3.2	Độ phức tạp, tính đúng đắn và ví dụ	25
3.4	Phương án C: Count-Min Sketch (được chọn)	26
3.4.1	Minh hoạ chạy tay và đánh giá	26
3.5	Phương án D: Count Sketch (bị loại vì chi phí)	28
3.6	Bảng so sánh tổng hợp	29
3.6.1	So sánh định lượng: lý thuyết và bộ nhớ tuyệt đối	29
3.6.2	Phân tích theo kịch bản và khẳng định lại lựa chọn	30
3.7	Kết luận chương	30
Chương 4.	THIẾT KẾ VÀ CÀI ĐẶT	31
4.1	Thiết kế tổng thể	31
4.1.1	Strategy Pattern và giao diện Estimator	31
4.1.2	Registry và dòng dữ liệu pipeline	32
4.2	Cài đặt Count-Min Sketch	33
4.2.1	Cấu trúc dữ liệu nội bộ	33
4.2.2	Hàm băm MurmurHash3	34
4.2.3	Thao tác record() và estimate()	35
4.3	Cài đặt Misra-Gries	36
4.4	Cài đặt HashMap Exact (baseline)	36
4.5	Kiểm thử đơn vị	37
4.5.1	Các test case cốt lõi	37
4.5.2	Kết quả tổng thể	38
4.6	Kết quả benchmark thực tế	38
4.6.1	Thông lượng	38
4.6.2	Sai số thực tế theo ε	39
4.7	Chương trình trace chạy từng bước	40

4.8	Kết luận chương	41
Chương 5.	ỨNG DỤNG VÀ ĐÁNH GIÁ	42
5.1	Ứng dụng phát hiện port-scan trên luồng IP	42
5.1.1	Mô hình mối đe dọa và ánh xạ thuật toán	42
5.1.2	Kiến trúc pipeline tinh giản	43
5.1.3	Cài đặt và minh họa từng bước	45
5.2	Đánh giá thực nghiệm	48
5.2.1	Phương pháp và môi trường đánh giá	48
5.2.2	Thông lượng và bộ nhớ trên luồng tổng hợp	48
5.2.3	Sai số CMS theo chiều rộng	49
5.2.4	So sánh ba thuật toán trên luồng IP port-scan	49
5.3	Đánh giá tổng thể giải pháp	50
5.4	Kết luận và hướng phát triển	50
	TÀI LIỆU THAM KHẢO	52
	PHỤ LỤC	55
Chương A.	MÃ NGUỒN LỖI: COUNT-MIN SKETCH, MISRA-GRIES, HASHMAP	PL-1
A.1	Tóm tắt cấu trúc mã nguồn	PL-1
A.2	Giao diện Estimator	PL-2
A.3	Count-Min Sketch	PL-3
A.3.1	Header	PL-3
A.3.2	Cài đặt	PL-4
A.4	Misra-Gries (Heavy Hitters)	PL-7
A.4.1	Header	PL-7
A.4.2	Cài đặt	PL-8
A.5	Kiểm thử đơn vị Misra-Gries	PL-11
A.6	Hash Map Exact (đối chứng)	PL-15
A.6.1	Header	PL-15
A.6.2	Cài đặt	PL-16
A.7	Kiểm thử đơn vị HashMap Exact	PL-17
A.8	Kiểm thử đơn vị Count-Min Sketch	PL-18
A.9	Bảng đối chiếu hành vi chính	PL-21
A.10	Benchmark tần suất	PL-21
Chương B.	KẾT QUẢ CHẠY VÀ DỮ LIỆU ĐẦU RA	PL-25
B.1	Menu và chế độ chạy	PL-25
B.2	Count-Min Sketch	PL-25

B.3	Misra–Gries	.PL-27
B.4	So sánh	.PL-28
B.5	Output ứng dụng phát hiện port-scan	.PL-29
B.5.1	Kịch bản tấn công trên cụm Kubernetes	.PL-29
B.5.2	Menu và log chạy end-to-end	.PL-30
B.5.3	Luồng sự kiện đầu vào	.PL-32
B.5.4	Cảnh báo và bảng so sánh	.PL-39
B.5.5	Trace nội bộ của ba bộ ước lượng	.PL-40
B.6	Output đo hiệu năng và dữ liệu trực quan hóa	.PL-47

BẢNG DIỄN GIẢI TỪ VIẾT TẮT VÀ TIẾNG ANH

Tên viết tắt	Tên (English)	Diễn giải (Tiếng Việt)
BFS	Breadth-First Search	Tìm kiếm theo chiều rộng
CMS	Count-Min Sketch	Cấu trúc dữ liệu "sketch" để ước lượng tần suất phần tử trong luồng; cho ước lượng xấp xỉ với sai số có thể kiểm soát
CTDL	Data Structures	Cấu trúc dữ liệu
DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
DSA	Data Structures and Algorithms	Cấu trúc dữ liệu và giải thuật
eBPF	extended Berkeley Packet Filter	Kỹ thuật cho phép chạy mã trong kernel để lọc/thu thập/xử lý gói mạng (eBPF)
IP	Internet Protocol	Giao thức Internet dùng để định danh và chuyển gói tin
K8s	Kubernetes	Hệ thống điều phối container (quản lý deployment, scaling, scheduling)
LPM	Longest Prefix Match	Khớp tiền tố dài nhất
SCC	Strongly Connected Component	Thành phần liên thông mạnh
SOC	Security Operations Center	Trung tâm vận hành an ninh (giám sát và phản ứng sự cố)
YAML	YAML Ain't Markup Language	Ngôn ngữ dữ liệu cấu hình dạng phân cấp (YAML)
MG	Misra–Gries	Thuật toán Misra–Gries để xấp xỉ các phần tử có tần suất lớn bằng cách duy trì k bộ đếm

CU	Conservative Update	Biến thể Conservative Update của CMS
Mops	Million operations per second	Triệu thao tác/giây
KB	Kilobyte	1 KB = 1024 bytes
CSV	Comma-Separated Values	Tập tin dữ liệu phân tách dấu phẩy
L2	Level-2 cache	Bộ nhớ đệm cấp 2
MurmurHash3	MurmurHash3	Hàm băm không mật mã, nhanh và phổ biến để băm khóa trong hệ thống
Zipfian	Zipfian distribution	Phân phối Zipf (phân phối lệch; một vài phần tử rất phổ biến)
Heavy hitters	Heavy hitters	Các phần tử có tần suất cao trong luồng (phần tử 'nặng')
Count Sketch	Count Sketch	Biến thể sketch dùng để ước lượng tần suất, khác với CMS về cấu trúc và sai số
Single-pass	Single-pass algorithm	Thuật toán xử lý luồng trong một lần duyệt (mỗi phần tử chỉ xử lý một lần)
HashMap	HashMap (exact)	Bảng băm lưu trữ tần suất chính xác (đòi hỏi bộ nhớ tuyến tính)
Prometheus	Prometheus	Hệ thống giám sát và thu thập metric thời gian thực
Grafana	Grafana	Công cụ trực quan hóa và dashboard cho metrics
Pipeline	Pipeline	Dòng xử lý dữ liệu gồm các bước (collector → normalizer → aggregator → ...)
Collector	Collector	Thành phần thu thập sự kiện/thông điệp từ nguồn

Normalizer	Normalizer	Thành phần chuẩn hóa dữ liệu/sự kiện về dạng thống nhất
Aggregator	Aggregator	Thành phần tổng hợp dữ liệu từ nhiều nguồn/nút
FPR	False Positive Rate	Tỷ lệ báo động giả (lưu lượng hợp lệ bị gán cờ tấn công)
TCP	Transmission Control Protocol	Giao thức truyền tải hướng kết nối, tin cậy
CNI	Container Network Interface	Chuẩn cắm mạng cho container/Kubernetes (vd Calico, Cilium)
CI	Confidence Interval	Khoảng tin cậy (thống kê), ở đây dùng mức 95%
CO-RE	Compile Once – Run Everywhere	Kỹ thuật biên dịch eBPF một lần chạy trên nhiều phiên bản kernel nhờ BTF
BTF	BPF Type Format	Định dạng thông tin kiểu dữ liệu của kernel, nền tảng cho CO-RE
kprobe	Kernel probe	Điểm chèn (hook) động vào hàm kernel để chạy chương trình eBPF
NetworkPolicy	Kubernetes NetworkPolicy	Đối tượng K8s khai báo luật cho/chặn lưu lượng pod (do CNI thực thi)
MTTR	Mean Time To Respond	Thời gian trung bình để phản hồi/xử lý sự cố
nmap	Network Mapper	Công cụ quét cổng/dịch vụ mạng phổ biến

BẢNG DIỄN GIẢI KÝ HIỆU

$S = (a_1, \dots, a_m)$	Luồng dữ liệu gồm m phần tử
U	Không gian phần tử (universe), $ U = n$
f_i	Tần suất thực của phần tử i trong luồng
$\hat{f}_x$	Tần suất ước lượng của phần tử x
ε	Tham số sai số (error parameter)
δ	Tham số xác suất thất bại (failure probability)
F_k	Frequency moment bậc k : $F_k = \sum f_i^k$
$\ \mathbf{f}\ _1$	Chuẩn L_1 của vector tần suất, $= F_1 = n$
w	Chiều rộng (width) của ma trận CMS
d	Chiều sâu (depth) — số hàm băm
$h_i : U \rightarrow [w]$	Hàm băm thứ i , ánh xạ phần tử sang cột
$G = (V, E)$	Đồ thị có hướng với tập đỉnh V , tập cung E
m	Độ dài luồng (số phần tử), nếu $S = (a_1, \dots, a_m)$
k	Tham số Misra–Gries (số bộ đếm k)
F_0	Số phần tử phân biệt trong luồng (distinct elements)
M	Ma trận Count–Min Sketch kích thước $d \times w$; $M[i][j]$ là bộ đếm
$O(\cdot)$	Ký hiệu Big-O: cận trên về độ phức tạp thời gian/không gian
$\mathbb{E}[\cdot]$	Kỳ vọng (expected value)
$\Pr[\cdot]$	Xác suất (probability)
e	Hằng số Euler ($e \approx 2.71828$)
$\lceil \cdot \rceil$	Hàm trần (ceiling)
$\ln$	Logarit tự nhiên
$\mathcal{H}$	Họ hàm băm (family of hash functions)

DANH SÁCH BẢNG

2.1	Minh họa các moment tần suất trên luồng $S = (a, b, a, c, a, b, d, a)$, $m = 8$.	8
2.2	Các bước tính MurmurHash3 (ví dụ).	11
2.3	So sánh CMS chuẩn và Conservative Update (CU) trên luồng $S = (a, b, a, c, a, b)$.	16
2.4	Công cụ sử dụng trong báo cáo và mã nguồn đính kèm.	19
3.1	Trạng thái Hash Map sau từng bước UPDATE.	23
3.2	Trạng thái bảng Misra-Gries với $k = 2$.	26
3.3	Bảng hash cho ví dụ CMS với $w = 4$, $d = 2$.	27
3.4	Trạng thái ma trận CMS M sau từng bước UPDATE.	27
3.5	So sánh tổng hợp bốn phương án ước lượng tần suất.	29
3.6	So sánh bộ nhớ tuyệt đối ở ba cấu hình tham số.	29
4.1	Kết quả kiểm thử đơn vị libdsa (ctest).	38
4.2	Thông lượng và bộ nhớ (Mops, KB).	39
4.3	Sai số thực tế (overcount trung bình).	40
5.1	Tham số cấu hình của PortScanDetector.	43
5.2	Thông lượng <i>record/estimate</i> và bộ nhớ trên 10^5 khoá phân biệt.	48
5.3	Sai số trung bình theo chiều rộng CMS ($d = 5$, $m = 10^5$).	49
5.4	So sánh CMS / Misra-Gries / Hash Map trên luồng port-scan tổng hợp.	49
A.1	Tóm tắt các file mã nguồn trong phụ lục	PL-1
A.2	Hành vi chính của ba cấu trúc tần suất	PL-21

DANH SÁCH HÌNH VẼ

2.1	Cấu trúc Count-Min Sketch.	12
2.2	Lưu đồ thao tác UPDATE của Count-Min Sketch.	13
2.3	Lưu đồ thao tác QUERY của Count-Min Sketch.	13
3.1	Phễu loại trừ phương án ước lượng tần suất.	21
3.2	Lưu đồ quyết định của Misra-Gries.	25
3.3	Trạng thái cuối của ma trận Count-Min Sketch.	27
4.1	Sơ đồ lớp Estimator.	31
4.2	Dòng dữ liệu pipeline.	32
4.3	Layout bộ nhớ của Count-Min Sketch.	33
4.4	Thông lượng record() (CMS vs HashMap).	39
4.5	Sai số thực tế theo w	40
5.1	Pipeline phát hiện port-scan tinh giản.	44
A.1	Luồng xử lý cập nhật và truy vấn trong Count-Min Sketch	PL-7
A.2	Lưu đồ cập nhật của Misra-Gries	.PL-11
B.1	Kịch bản tấn công port-scan trên cụm Kubernetes giả lập.	.PL-30

MỞ ĐẦU

Lý do chọn đề tài

Trong bối cảnh các hệ thống phân tán ngày càng phức tạp, đặc biệt là các cụm Kubernetes (K8s) triển khai hàng trăm đến hàng nghìn container, việc giám sát và phát hiện bất thường mạng trong thời gian thực trở thành thách thức lớn. Lưu lượng mạng sinh ra liên tục với tốc độ hàng triệu sự kiện mỗi giây, trong khi tài nguyên bộ nhớ trên mỗi nút giám sát bị giới hạn nghiêm ngặt.

Bài toán cốt lõi ở đây thuộc lĩnh vực **cấu trúc dữ liệu và giải thuật**: làm thế nào để ước lượng tần suất xuất hiện của các phần tử (địa chỉ IP) trên một luồng dữ liệu liên tục, khi bộ nhớ khả dụng nhỏ hơn nhiều so với kích thước luồng? Đây chính là bài toán *Stream Frequency Estimation* — một bài toán kinh điển trong lý thuyết streaming algorithms.

Từ góc độ ứng dụng, lời giải cho bài toán này cho phép xây dựng hệ thống phát hiện port scanning, DDoS và các hành vi bất thường khác trên mạng K8s mà không gây quá tải bộ nhớ — ngay cả khi kẻ tấn công cố tình tạo ra lượng lớn địa chỉ IP giả mạo.

Mục tiêu nghiên cứu

Báo cáo tập trung vào các mục tiêu sau:

1. **Hình thức hóa bài toán**: Phát biểu chính xác bài toán ước lượng tần suất trên luồng dữ liệu dưới dạng toán học, bao gồm mô hình luồng, ràng buộc bộ nhớ, và yêu cầu (ϵ, δ) -accuracy.
2. **Phân tích và so sánh giải thuật**: Nghiên cứu chi tiết ít nhất 3 phương án giải quyết (Exact Hash Map, Misra-Gries, Count-Min Sketch), so sánh về độ phức tạp thời gian, bộ nhớ và tính chất lỗi.
3. **Thiết kế và cài đặt**: Thiết kế hệ thống modular (Strategy Pattern) và cài đặt các giải thuật bằng C++17, kèm kiểm thử đơn vị.
4. **Minh họa từng bước**: Xây dựng chương trình xuất kết quả chạy giải thuật từng bước (step-by-step trace) dưới dạng dễ đọc, minh họa hoạt động nội tại của cấu trúc dữ liệu.
5. **Đánh giá thực nghiệm**: So sánh hiệu năng (throughput, bộ nhớ, độ chính xác) trên dữ liệu mô phỏng, đánh giá tính hiệu quả của giải pháp.
6. **Ứng dụng thực tế**: Tích hợp giải thuật vào pipeline phát hiện bất thường mạng trên Kubernetes, sử dụng eBPF để thu thập sự kiện.

Phạm vi và giới hạn

Phạm vi:

- Tập trung vào bài toán *Stream Frequency Estimation* và cấu trúc dữ liệu Count-Min Sketch.

- Cài đặt bằng C++17, kiểm thử bằng Google Test, benchmark bằng Google Benchmark.
- Ứng dụng minh họa: phát hiện port scan trên mạng Kubernetes.

Giới hạn:

- Ứng dụng phát hiện port-scan được hiện thực hoá ở dạng pipeline tinh giản (`tcpcdump` → `parser` → `frequency estimator` → `alert`) ở Chương 5; tích hợp đầy đủ vào hệ Zero-Trust SOC (NATS, PostgreSQL, k8s NetworkPolicy, eBPF collector) là phạm vi của đồ án tốt nghiệp môn Công nghệ phần mềm 2.
- Không đi sâu vào các bài toán phụ trên đồ thị (SCC detection, BFS k-hop reachability, LPM Trie classification) — chỉ đề cập ở mục hướng phát triển ở Chương 5.
- Thực nghiệm trên dữ liệu mô phỏng có hạt giống cố định để bảo đảm khả năng tái lập; phần ghép nối với lưu lượng cụm K8s thực thuộc môn 2.

Phương pháp nghiên cứu

- **Nghiên cứu lý thuyết:** Phân tích bài báo gốc (Cormode & Muthukrishnan, 2005; Misra & Gries, 1982), chứng minh các đảm bảo toán học.
- **Phương pháp thực nghiệm:** Cài đặt, kiểm thử đơn vị, benchmark so sánh giữa các phương án.
- **Minh họa trực quan:** Lưu đồ giải thuật, bảng trace chạy từng bước, biểu đồ benchmark.
- **Đánh giá:** So sánh lý thuyết và thực nghiệm, phân tích theo các kịch bản sử dụng thực tế.

Bố cục báo cáo

Báo cáo gồm 5 chương:

- **Chương 1 — Bài toán:** Bối cảnh, giả thiết, phát biểu hình thức bài toán ước lượng tần suất điểm và khảo sát công trình liên quan.
- **Chương 2 — Cơ sở lý thuyết:** Mô hình luồng dữ liệu, moment tần suất, yêu cầu (ε, δ) -accuracy, tính chất hàm băm pairwise-independent và chứng minh định lý Cormode–Muthukrishnan.
- **Chương 3 — Phân tích và so sánh giải thuật:** Bốn phương án (Hash Map, Misra-Gries, Count-Min Sketch, Count Sketch), mỗi phương án kèm pseudocode, chứng minh, minh họa chạy tay; bảng so sánh tổng hợp và kết luận chọn CMS với Conservative Update.
- **Chương 4 — Thiết kế và cài đặt:** Kiến trúc `libdsa` với Strategy Pattern và Registry, cài đặt Count-Min Sketch trong C++17, kiểm thử đơn vị và chương trình trace từng bước.
- **Chương 5 — Ứng dụng và đánh giá:** Mô hình mối đe dọa port-scan, pipeline tinh giản `tcpcdump` → `parser` → `frequency estimator` → `alert`, trace từng bước trên luồng IP tổng hợp; sau đó benchmark thông lượng/bộ nhớ trên cả luồng tổng hợp lẫn luồng IP,

đối chiếu năm tiêu chí ở Chương 1, kết luận và hướng phát triển sang đề án tốt nghiệp.

CHƯƠNG 1

BÀI TOÁN

Chương này trình bày bối cảnh thực tiễn, phát biểu toán học và phạm vi của bài toán được giải quyết trong báo cáo. Chương 2 kế tiếp sẽ đi sâu vào nền tảng lý thuyết; chương này giới hạn ở việc làm rõ *cần giải cái gì, ở đâu, và dưới ràng buộc nào*.

1.1. Bối cảnh và đặt vấn đề

Bài toán mà báo cáo này giải quyết không xuất phát từ một quan sát ngẫu nhiên mà đứng trong dòng dịch chuyển kiến trúc phần mềm doanh nghiệp khoảng một thập kỷ trở lại đây: từ ứng dụng đơn khối sang vi-dịch-vụ (*microservices*) đóng gói trong container và điều phối bằng Kubernetes (K8s). Sự dịch chuyển này, khởi đầu khoảng năm 2014–2015, đã trở thành chuẩn mực triển khai phần mềm doanh nghiệp; song hành với nó là một bề mặt tấn công kiểu mới mà các mô hình bảo mật chu vi truyền thống không bao phủ được — hàng trăm endpoint nội bộ phơi bày qua mạng overlay của cluster, các lời gọi đồng- đồng giữa dịch vụ thay vì máy khách- máy chủ, vòng đời container ngắn (đo bằng phút), và sự đa dạng công nghệ trong cùng một cụm khiến bề mặt phòng thủ trở nên không đồng nhất.

Ba bài tổng quan có hệ thống (*systematic literature reviews*) gần đây nhất về an toàn thông tin của hệ vi-dịch-vụ trên K8s nhất quán ở cùng một kết luận trung tâm: *vẫn thiếu vắng những chuẩn bảo mật chuyên biệt và mô hình toàn diện cho kiến trúc vi-dịch-vụ*. Cụ thể, Hutasuhut và cộng sự (2024) [12] sàng lọc 487 bài báo — chọn lại 87 bài qua phương pháp snowball — tập trung vào các lỗ hổng phát hiện sau năm 2020, và nhấn mạnh khoảng cách kiến thức rõ rệt giữa giới học thuật và giới thực hành công nghiệp. Trước đó, Berardi và cộng sự (2022) [3] khảo sát 290 công bố và xác định bốn khoảng trống cốt lõi: thiếu hướng dẫn áp dụng *security-by-design*, thiếu mô hình mối đe dọa phù hợp, thiếu hướng dẫn xử lý bề mặt tấn công sinh ra bởi sự đa dạng công nghệ, và các vấn đề an toàn khi di trú từ hệ đơn khối. Pereira-Vale và cộng sự (2021) [16] bổ sung góc nhìn đa nguồn (*multivocal review*): kết hợp 36 công bố học thuật với 34 nguồn xám (báo cáo công nghiệp, blog kỹ thuật) cho thấy nhiều giải pháp công nghiệp đang được triển khai chưa có cơ sở lý thuyết được kiểm chứng, đồng thời nhiều phân tích lý thuyết chưa được hiện thực hóa trong hệ thống thực.

Đối chiếu với thực tế triển khai, hai báo cáo công nghiệp gần đây xác nhận các kết luận trên. *The State of Kubernetes Security 2024* của Red Hat [17], dựa trên khảo sát 600 chuyên gia DevOps và an toàn thông tin, ghi nhận: 67% tổ chức đã *trì hoãn hoặc làm chậm phát triển ứng dụng* do lo ngại an ninh tăng cao; 45% gặp sự cố ở giai đoạn *runtime* trong 12 tháng gần nhất; 30% bị phạt sau sự cố và 46% mất doanh thu hoặc khách hàng.

Kubernetes Security Report 2025 của Wiz [21] bổ sung số liệu thời gian: cluster K8s mới tạo trên các dịch vụ managed (AKS, EKS) bị tấn công thử lần đầu trong vòng 18–28 phút sau khi sẵn sàng. Số liệu này cho thấy giả định “đầu vào đối kháng” không phải là một yêu cầu lý thuyết mà phản ánh đúng điều kiện vận hành mặc định.

Khi đối chiếu các khoảng trống nêu trên với cấu trúc một hệ giám sát mạng chạy trong cluster K8s, có thể nhận thấy một yêu cầu chung: lớp giám sát phải *quan sát liên tục, ở quy mô từng gói tin, dưới ràng buộc bộ nhớ chặt*, đồng thời *chịu được đầu vào đối kháng*. Một nút giám sát trung bình quan sát từ 10^5 đến 10^7 sự kiện mỗi giây trong các mạng sản xuất; mỗi sự kiện là một bản ghi có cấu trúc (địa chỉ IP nguồn, đích, cổng, giao thức, nhãn thời gian) và lưu lượng đi qua được mô hình hóa tự nhiên thành một *luồng sự kiện*. Một thao tác cơ bản — **ước lượng tần suất của một phần tử trên luồng** (đã quan sát bao nhiêu kết nối từ địa chỉ IP x ? bao nhiêu yêu cầu tới cổng p ?) — đơn giản về mặt khái niệm nhưng trở nên thách thức vì phải đồng thời thoả ba ràng buộc vật lý: thời gian xử lý mỗi sự kiện hằng số, bộ nhớ giới hạn không tăng tuyến tính theo số phần tử phân biệt, và đảm bảo chính xác cả khi kẻ tấn công chủ động sinh nhiều khóa giả nhằm làm tràn các bảng đếm chính xác. Đây chính là miền của các thuật toán trên luồng dữ liệu (*streaming algorithms*) và các cấu trúc dữ liệu xác suất (*probabilistic data structures*) — nơi đánh đổi sai số có kiểm soát lấy bộ nhớ nhỏ hơn một bậc độ lớn.

Mô hình *Zero Trust* (“không tin tưởng ngầm định, luôn xác minh”) do NIST chuẩn hoá [19] chính là khung an ninh đặt ra yêu cầu quan sát liên tục như trên: thay vì coi mọi kết nối bên trong mạng nội bộ là tin cậy, mỗi sự kiện kết nối phải được xác thực, ủy quyền và giám sát — không phân biệt nó đến từ bên trong hay bên ngoài. Nguyên lý này nâng mức bảo đảm an ninh bằng ba cơ chế: (i) loại bỏ các “vùng tin cậy ngầm định” vốn là điểm mù của tường lửa chu vi; (ii) hạn chế *lateral movement* khi một nút bị xâm nhập, vì mỗi bước đi đều phải xác minh lại; (iii) thu nhỏ *bán kính ảnh hưởng* của một sự cố. Việc xác minh liên tục ở quy mô từng gói tin đòi hỏi phải *quan sát* được tần suất của mọi nguồn, đích, cặp (IP, cổng) trong thời gian thực — và chính yêu cầu quan sát này sinh ra bài toán ước lượng tần suất trên luồng ở tốc độ cao và bộ nhớ hạn chế.

Báo cáo này, với phạm vi của một học phần (Công nghệ Phần mềm 1), *không* trực tiếp lấp toàn bộ các khoảng trống học thuật nêu ở phần trên, mà tập trung vào một viên gạch ở tầng thấp nhất: phân tích, cài đặt và đánh giá một cấu trúc dữ liệu nền tảng (Count- Min Sketch) cho một lát cắt cụ thể của bài toán giám sát — ước lượng tần suất điểm với đảm bảo (ϵ, δ) và bộ nhớ hằng số. Trên nền tảng đó, Chương 5 ráp lại một pipeline tinh giản *tcpdump* $\rightarrow$ *parser* $\rightarrow$ *frequency estimator* $\rightarrow$ *alert* để chứng minh bằng thực nghiệm rằng cấu trúc dữ liệu được phân tích là đủ để phát hiện port-scan trên luồng IP thật. Việc tích hợp đầy đủ vào một hệ Zero Trust hoàn chỉnh (eBPF collector ở *kernel space*, aggregator đa nút trên NATS JetStream, các dịch vụ SOC, đáp ứng tự động bằng

NetworkPolicy qua API server Kubernetes) thuộc phạm vi của bài báo cáo ở môn tốt nghiệp Công nghệ phần mềm 2.

1.2. Mô hình bài toán và yêu cầu lời giải

Để chuyển từ mô tả ngữ cảnh sang phát biểu hình thức, trước hết chúng ta cần thống nhất cố định năm giả thiết mô hình. **(i) Single-pass:** mỗi sự kiện được đọc đúng một lần; không có khả năng lưu trữ toàn bộ luồng để truy vấn sau. **(ii) Tốc độ đến cao:** tốc độ sự kiện vượt quá 10^5 /giây trên mỗi nút giám sát; chi phí xử lý mỗi sự kiện phải $O(1)$ hoặc $O(\log n)$. **(iii) Bộ nhớ hạn chế:** thuật toán phải chạy trong vài chục đến vài trăm KB, đủ vừa vặn với bộ nhớ cache L2 của CPU để duy trì thông lượng. **(iv) Đầu vào đối kháng:** kẻ tấn công có thể tạo ra luồng sự kiện với nhiều phần tử phân biệt (giả mạo địa chỉ IP nguồn) nhằm làm tràn các bảng băm chính xác — thuật toán phải chịu được loại tấn công này. **(v) Chấp nhận sai số xấp xỉ:** người sử dụng chấp nhận một giá trị ước lượng $\hat{f}$ thay cho giá trị đúng f , miễn là sai số có thể được *định lượng* và giới hạn bởi các tham số (ε, δ) . Bốn giả thiết đầu là ràng buộc vật lý sinh ra từ thực tiễn vận hành; giả thiết thứ năm là một sự nói lỏng yêu cầu — và chính sự nói lỏng này mở ra cánh cửa cho các cấu trúc sketch đạt mức bộ nhớ hằng số $O((1/\varepsilon) \log(1/\delta))$.

Trên các giả thiết đó, bài toán được phát biểu hình thức như sau. **Đầu vào** là một luồng $S = (a_1, a_2, \dots, a_m)$ với mỗi a_j thuộc một không gian phần tử U có kích thước n . Các a_j có thể trùng nhau; luồng được đọc tuần tự theo thứ tự xuất hiện, không có khả năng tua lại. **Đầu ra** là, với mỗi truy vấn $x \in U$ (có thể biết sau khi đã đọc toàn bộ luồng), một ước lượng $\hat{f}_x \in \mathbb{N}$ cho tần suất thật

$$f_x = |\{j : a_j = x\}|. \quad (1.1)$$

Gọi $m = \sum_x f_x$ là độ dài luồng. Thuật toán được xem là *chấp nhận được* theo tiêu chí (ε, δ) nếu tồn tại hai tham số $\varepsilon, \delta \in (0, 1)$ sao cho với mọi truy vấn x :

$$\Pr[|\hat{f}_x - f_x| \leq \varepsilon \cdot m] \geq 1 - \delta. \quad (1.2)$$

Bài toán này được gọi là **Point Query** trong văn liệu thuật toán luồng và là bài toán trung tâm mà Count-Min Sketch [7] giải quyết tối ưu. Định nghĩa hình thức chi tiết hơn (vector tần suất, moment bậc k , cận dưới bộ nhớ) được xây dựng ở Chương 2; tại đây giới hạn ở mức phát biểu đủ để xác định phạm vi bài toán.

Một lời giải cho bài toán Point Query nêu trên được coi là **đạt yêu cầu** khi đồng thời thoả năm điều kiện sau. Thứ nhất, *bộ nhớ giới hạn*: bộ nhớ sử dụng là $o(\min(n, m))$, lý tưởng là hằng số cố định hoặc $\text{polylog}(n, m)$. Thứ hai, *thời gian cập nhật hằng số*: thao tác xử lý mỗi sự kiện là $O(1)$ hoặc tối đa $O(\log(1/\delta))$, không phụ thuộc m hay n . Thứ ba,

đảm bảo có chứng minh: sai số phải có cận lý thuyết được chứng minh từ các giả thiết về hàm băm, không dựa vào phân bố dữ liệu cụ thể. Thứ tư, *xác nhận thực nghiệm*: cài đặt phải được kiểm thử đơn vị cho từng tính chất cốt lõi, benchmark cho thông lượng và bộ nhớ, và trace từng bước để kiểm chứng hoạt động. Thứ năm, *có thể tái lập*: mọi tham số, dữ liệu đầu vào, seed ngẫu nhiên phải được ghi lại để kết quả báo cáo có thể được tái tạo từ mã nguồn. Năm điều kiện này sẽ là khung tham chiếu để đánh giá và so sánh các phương án ở Chương 3.

1.3. Khảo sát công trình liên quan

Các công trình nền tảng cho bài toán Point Query nói riêng và bài toán Heavy Hitters nói chung được nhóm thành hai mạch: các kết quả giới hạn lý thuyết cùng với thuật toán deterministic, và các cấu trúc sketch xác suất cùng với khảo sát thực nghiệm đi kèm.

Ở mạch thứ nhất, Alon, Matias và Szegedy (1996) [1] thiết lập giới hạn dưới cho không gian các thuật toán ước lượng moment tần suất trên luồng: $\Omega(1/\varepsilon)$ cho thuật toán deterministic và $\Omega((1/\varepsilon) \log(1/\delta))$ cho thuật toán randomized. Công trình này (nhận Giải thưởng Gödel 2005) định hình toàn bộ miền và khẳng định mọi lời giải sau đó — kể cả Count-Min Sketch — không thể vượt qua ngưỡng này. Đặt trong khung đó, Misra và Gries (1982) [13] là thuật toán deterministic cổ điển: với bộ nhớ $O(k)$ giữ lại tối đa k phần tử “nặng” trong luồng, thuật toán đạt được cận dưới $\Omega(1/\varepsilon)$ nhưng chỉ cho ước lượng *dưới mức* và giải bài toán Heavy Hitters chứ không phải Point Query tổng quát — khiến nó trở thành một *điểm so sánh* hơn là lời giải trực tiếp cho bài toán báo cáo đặt ra.

Ở mạch thứ hai, Cormode và Muthukrishnan (2005) [7] giới thiệu Count-Min Sketch — cấu trúc sketch hai chiều sử dụng d hàm băm pairwise-independent. Với bộ nhớ $O((1/\varepsilon) \log(1/\delta))$, CMS đạt đảm bảo (ε, δ) cho Point Query và đạt tối ưu so với cận dưới ở mạch thứ nhất. Biến thể *Conservative Update* do Estan và Varghese [9] đề xuất trong bối cảnh đo lưu lượng mạng giảm đáng kể sai số thực tế trên phân phối Zipfian — đặc trưng của lưu lượng Internet. Để so sánh đối chiếu, Charikar, Chen và Farach-Colton (2004) [5] đề xuất Count Sketch: một biến thể cho ước lượng *không thiên lệch* bằng cách sử dụng hàm $\text{sign } s_i$ và lấy median thay vì min, đánh đổi bằng bộ nhớ $O((1/\varepsilon^2) \log(1/\delta))$ — lớn hơn CMS một bậc $1/\varepsilon$ — nên phù hợp với các toán tử đại số (inner product, join size) hơn là Point Query. Cuối cùng, Cormode và Hadjieleftheriou (2008) [6] thực hiện so sánh thực nghiệm toàn diện giữa các thuật toán tần suất phổ biến trên nhiều tập dữ liệu khác nhau, cung cấp thông tin định lượng về thông lượng và độ chính xác trong các kịch bản thực tế — là cơ sở để so sánh cài đặt trong chương đánh giá.

1.4. Phạm vi và định hướng giải quyết

Báo cáo giải quyết bài toán **Point Query** trên mô hình luồng *cash-register* (insert-only), với mục tiêu chính là cài đặt và đánh giá Count-Min Sketch và các thuật toán thay thế (Misra–Gries, Hash Map chính xác). Các vấn đề nằm ngoài phạm vi gồm: mô hình *turnstile* cho phép giảm counter (do các thuật toán như Count Sketch hoặc CMS có sign hỗ trợ, không cần thiết khi đầu vào chỉ có chèn); bài toán Top- k / Heavy Hitters đầy đủ liệt kê được các phần tử có tần suất cao nhất (Misra–Gries giải một phần, CMS cần kết hợp cấu trúc phụ); các bài toán Range Query, Inner Product Estimation và Join Size Estimation thuộc lớp toán tử đại số trên sketch; và các thuật toán phụ khác trong hệ thống Zero-Trust (Tarjan SCC, BFS k-hop, LPM Trie) — được đề cập ở phần hướng phát triển của Chương 5 và là nội dung của đề án tốt nghiệp.

Để người đọc có một định hướng rõ ràng trước khi bước vào phần lý thuyết, báo cáo công bố trước kết luận: **Count-Min Sketch (CMS) với biến thể Conservative Update** là lời giải được chọn cho bài toán Point Query nêu trên. Các chương sau đi theo lộ trình tuyến tính dẫn tới kết luận này. Chương 2 xây dựng nền tảng lý thuyết của CMS: cấu trúc ma trận $d \times w$, họ hàm băm pairwise-independent, định lý Cormode–Muthukrishnan và chứng minh cận sai số (ϵ, δ) . Chương 3 so sánh CMS với ba phương án thay thế (Hash Map chính xác, Misra–Gries, Count Sketch) trên bốn yêu cầu cốt lõi — bộ nhớ hằng số, cập nhật $O(1)$, sai số một phía có thể định lượng và khả năng hợp nhất song song — rồi chỉ ra lý do cụ thể để loại từng phương án. Chương 4 trình bày cài đặt CMS trong thư viện `libdsa` và đo thông lượng, sai số thực tế trên dữ liệu benchmark. Chương 5 tổng kết kết quả và xác nhận lại lựa chọn bằng số liệu đo được. Cách trình bày này cho phép người đọc đối chiếu từng đặc tính của CMS với yêu cầu bài toán ngay khi gặp, thay vì phải chờ tới kết luận cuối để biết giải thuật nào được chọn và vì sao.

1.5. Kết luận chương

Chương này đã: (i) đặt bài toán giám sát mạng dưới khung Zero Trust trong bối cảnh các nghiên cứu khoảng trống trên Kubernetes, và chỉ ra rằng việc xác minh liên tục ở quy mô gói tin sinh ra một bài toán tính toán cụ thể — ước lượng tần suất trên luồng dưới ràng buộc bộ nhớ và đầu vào đối kháng; (ii) phát biểu hình thức bài toán **Point Query** trên mô hình *cash-register* với tiêu chí (ϵ, δ) và năm điều kiện chấp nhận lời giải; (iii) khảo sát các công trình nền tảng (Alon–Matias–Szegedy, Misra–Gries, Cormode–Muthukrishnan, Estan–Varghese, Charikar–Chen–Farach-Colton, Cormode–Hadjieleftheriou) làm cơ sở cho phần so sánh ở chương sau; (iv) công bố trước kết luận: **Count-Min Sketch với Conservative Update là phương án được chọn**, đồng thời chỉ rõ phạm vi (Point Query, *cash-register*) và các vấn đề nằm ngoài phạm vi báo cáo. Bốn nội dung này thiết lập đầy đủ ngữ cảnh và yêu cầu; các chương kế tiếp lần lượt xây dựng nền tảng lý thuyết, so sánh

phương án, cài đặt và đánh giá để chứng minh lựa chọn vừa nêu.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng lý thuyết của bài toán ước lượng tần suất điểm (*Point Query*) trên luồng dữ liệu. Nội dung tập trung vào ba thành phần: mô hình luồng dữ liệu và các ràng buộc tài nguyên, các tính chất của họ hàm băm pairwise-independent, và cấu trúc dữ liệu Count-Min Sketch [7] với phần chứng minh đầy đủ của định lý Cormode–Muthukrishnan (2005).

2.1. Mô hình luồng dữ liệu

2.1.1. Định nghĩa hình thức và ràng buộc tài nguyên

Định nghĩa 2.1 (Luồng dữ liệu). Cho *không gian phần tử* (universe) $U = \{1, 2, \dots, n\}$ với n phần tử phân biệt. Một **luồng dữ liệu** (data stream) là chuỗi hữu hạn các phần tử

$$S = (a_1, a_2, \dots, a_m), \quad (2.1)$$

trong đó mỗi $a_j \in U$ và m là tổng số lượt xuất hiện. Các phần tử được xử lý tuần tự, mỗi phần tử chỉ được duyệt một lần (*single-pass*).

Trong bối cảnh giám sát mạng mang tinh thần Zero Trust — nơi mọi kết nối đều phải được xác minh không phụ thuộc vị trí mạng — U là tập các địa chỉ IP nguồn (hoặc cặp (src_ip, dst_port) trong bài toán phát hiện port-scan), và a_j là sự kiện kết nối thứ j quan sát được trong cửa sổ thời gian. Giá trị n có thể lên tới 2^{32} đối với IPv4 đơn thuần, và lớn hơn nhiều khi kết hợp với cổng; m trong các mạng doanh nghiệp có thể đạt hàng triệu sự kiện mỗi giây.

Tương ứng với định nghĩa trên, báo cáo giả định luồng thuộc dạng *cash-register* (*insert-only*) — các phần tử chỉ được thêm vào, không có thao tác xóa — cùng ba ràng buộc cốt lõi về tài nguyên: (i) **single-pass**, mỗi a_j được đọc đúng một lần, không thể quay lại; (ii) **bộ nhớ giới hạn**, tổng bộ nhớ sử dụng phải là $o(\min(n, m))$, lý tưởng là $\text{polylog}(n, m)$; (iii) **độ trễ không đổi**, thời gian xử lý mỗi phần tử là $O(1)$ hoặc $O(\log(\cdot))$ và độc lập với m . Ràng buộc bộ nhớ là lý do cơ bản buộc ta phải từ bỏ lời giải chính xác: khi m vượt quá khả năng chứa của RAM ($m \gg M$ với M là bộ nhớ khả dụng), hệ thống đếm chính xác bằng hash map sẽ bị từ chối cấp phát hoặc gây *swap* làm sụp đổ hiệu năng [4].

2.1.2. Moment của tần suất

Định nghĩa 2.2 (Vector tần suất và moment). Với luồng S theo Định nghĩa 2.1, tần suất của phần tử $i \in U$ được định nghĩa là

$$f_i = |\{j : a_j = i\}|. \quad (2.2)$$

Vector tần suất là $\mathbf{f} = (f_1, f_2, \dots, f_n)$. Moment bậc k của luồng được định nghĩa là

$$F_k = \sum_{i=1}^n f_i^k. \quad (2.3)$$

Đặc biệt, $F_1 = \|\mathbf{f}\|_1 = m$ chính là tổng số phần tử trong luồng, còn $F_0 = |\{i : f_i > 0\}|$ là số phần tử phân biệt.

Minh họa bằng số. Để thấy rõ ý nghĩa của từng moment, xét luồng mẫu $S = (a, b, a, c, a, b, d, a)$ với $m = 8$. Vector tần suất là $\mathbf{f} = (f_a, f_b, f_c, f_d) = (4, 2, 1, 1)$. Bảng 2.1 liệt kê các moment bậc thấp và ý nghĩa trực quan của chúng trong bối cảnh giám sát mạng.

Bảng 2.1. Minh họa các moment tần suất trên luồng $S = (a, b, a, c, a, b, d, a)$, $m = 8$.

Moment	Công thức	Giá trị	Ý nghĩa trong giám sát mạng
F_0	$ \{i : f_i > 0\} $	4	Số địa chỉ IP nguồn <i>phân biệt</i> đã quan sát; đặc trưng quy mô đa dạng luồng
F_1	$\sum_i f_i$	8	Tổng số gói/sự kiện (bằng m); dùng làm mẫu số khi chuẩn hoá tần suất
F_2	$\sum_i f_i^2$	22	Năng lượng phân phối; càng lớn nghĩa là luồng càng “lệch” (skewed), một số IP chiếm đa số lưu lượng
F_∞	$\max_i f_i$	4	Tần suất của phần tử nặng nhất (heavy hitter); dấu hiệu sớm của DDoS hoặc port-scan

Giá trị $F_2/F_1^2 = 22/64 \approx 0,34$ là hệ số *skew* thô: càng gần $1/F_0 = 0,25$ thì phân phối càng đều, càng gần 1 thì càng tập trung. Bài toán Point Query không yêu cầu ước lượng F_2 trực tiếp nhưng chính F_2 là đại lượng mà Count Sketch (Mục 3) ước lượng, giúp so sánh hai phương án.

2.2. Bài toán ước lượng tần suất điểm

2.2.1. Phát biểu bài toán và yêu cầu sai số

Định nghĩa 2.3 (Point Query). Cho luồng S và một phần tử truy vấn $x \in U$. **Point Query** là bài toán trả lời $\hat{f}_x$ ước lượng cho tần suất thật f_x của x trong S . Thuật toán không biết trước tập các phần tử sẽ được truy vấn.

Vì không biết trước x , thuật toán phải duy trì cấu trúc đủ tổng quát để trả lời bất kỳ truy vấn nào mà x có thể thuộc U . Đây là điểm khác biệt căn bản với bài toán *Heavy Hitters* (chỉ cần trả về k phần tử tần suất cao nhất) và bài toán *Distinct Count* (chỉ cần F_0). Do yêu cầu duy trì ở mức tổng quát không cho phép lưu trữ chính xác, lời giải gần đúng được đặc trưng bởi hai tham số: sai số tương đối $\varepsilon \in (0, 1)$ và xác suất thất bại $\delta \in (0, 1)$.

Định nghĩa 2.4 ((ε, δ) -approximation). Thuật toán gọi là (ε, δ) -xấp xỉ cho *Point Query* nếu với mọi truy vấn $x \in U$:

$$\Pr[|\hat{f}_x - f_x| \leq \varepsilon \cdot \|f\|_1] \geq 1 - \delta. \quad (2.4)$$

Tham số ε điều khiển **độ chính xác**: sai số tuyệt đối không vượt quá $\varepsilon \cdot m$. Tham số δ điều khiển **độ tin cậy**: xác suất sai lệch vượt ngưỡng không quá δ . Ví dụ với $\varepsilon = 0,001$ và $\delta = 0,01$, thuật toán đảm bảo sai số $\leq 0,1\% \cdot m$ với độ tin cậy $\geq 99\%$. Câu hỏi tự nhiên tiếp theo là: trong mức yêu cầu (ε, δ) nói trên, tối thiểu cần bao nhiêu bộ nhớ? Định lý dưới đây trả lời câu hỏi đó.

2.2.2. Giới hạn dưới về bộ nhớ

Định lý 2.5 (Alon–Matias–Szegedy, 1996). Mọi thuật toán *deterministic* giải bài toán *Point Query* với sai số tuyệt đối $\leq \varepsilon \cdot m$ cần ít nhất $\Omega(1/\varepsilon)$ không gian bộ nhớ [1]. Với thuật toán *randomized* cho phép sai số với xác suất δ , cận dưới là $\Omega((1/\varepsilon) \log(1/\delta))$ [11].

Định lý 2.5 khẳng định Count-Min Sketch đạt **tối ưu lý thuyết** về độ phức tạp không gian (chỉ hơn hằng số so với cận dưới), nên mọi nỗ lực tiết kiệm bộ nhớ hơn nữa phải đi kèm với việc yếu đi yêu cầu về sai số hoặc độ tin cậy.

2.3. Hàm băm và tính chất

2.3.1. Họ hàm băm universal và pairwise-independent

Phép phân tích Count-Min Sketch dựa trên hai tính chất ngẫu nhiên của hàm băm, được đặt theo thứ tự yêu cầu tăng dần: *universal* (để kiểm soát tần suất đựng độ trung bình) và *pairwise-independent* (để kiểm soát xác suất các cặp đựng độ).

Định nghĩa 2.6 (Họ hàm băm universal). Cho $\mathcal{H}$ là một họ hàm $h : U \rightarrow \{0, 1, \dots, w - 1\}$.

$\mathcal{H}$ được gọi là **universal** nếu với mọi cặp $x \neq y \in U$:

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{w}. \quad (2.5)$$

Định nghĩa 2.7 (Họ hàm băm pairwise-independent). *Họ $\mathcal{H}$ là pairwise-independent nếu với mọi cặp $x \neq y \in U$ và mọi cặp giá trị $(u, v) \in \{0, \dots, w-1\}^2$:*

$$\Pr_{h \sim \mathcal{H}} [h(x) = u \wedge h(y) = v] = \frac{1}{w^2}. \quad (2.6)$$

Hệ quả trực tiếp: nếu h được chọn ngẫu nhiên từ một họ pairwise-independent thì với mọi $x \neq y$, biến cố $[h(x) = h(y)]$ xảy ra với xác suất đúng $1/w$ và các biến cố đựng độ $[h(x) = h(y)]$ với y khác nhau là độc lập từng đôi. Tính chất này cho phép áp dụng bất đẳng thức Markov cho từng hàng trong Count-Min Sketch mà không phụ thuộc vào phân phối dữ liệu đầu vào [14], và là giả thiết then chốt trong chứng minh định lý Cormode–Muthukrishnan ở phần sau.

2.3.2. MurmurHash3 và minh họa tính toán

Trên lý thuyết, một họ pairwise-independent đơn giản có thể được xây dựng từ hàm tuyến tính $h_{a,b}(x) = ((ax + b) \bmod p) \bmod w$ với p là số nguyên tố $> n$ và a, b chọn ngẫu nhiên. Tuy nhiên, trong cài đặt sản xuất, MurmurHash3 [2] được ưu tiên vì ba lý do: (i) **tốc độ** — xử lý khoảng 4GB/s trên kiến trúc x86-64 hiện đại (benchmark *SMHasher*), nhanh hơn đáng kể so với phép chia lấy dư trên số nguyên tố; (ii) **phân bố đồng đều** — vượt qua toàn bộ bộ kiểm thử *SMHasher* về tính *avalanche* và phân bố bit; (iii) **tính pairwise-independent xấp xỉ** — tuy MurmurHash3 không phải pairwise-independent đúng nghĩa, phân tích thực nghiệm cho thấy độ lệch so với giả thuyết này nhỏ hơn biên sai số được đặt ra trong Định lý 2.8. Để mô phỏng việc chọn độc lập d hàm từ họ pairwise-independent, cài đặt `libdsa` sử dụng MurmurHash3 với d giá trị *seed* khác nhau, xem `count_min_sketch.cpp:19--60`.

Để minh họa cách băm một khóa thực tế, xét địa chỉ IPv4 $x = 192.168.1.10$ được biểu diễn dưới dạng 4 byte little-endian: `0x0A01A8C0`. Giả sử CMS có $w = 2048$ cột và ta sử dụng hai seed đầu tiên $s_0 = 0x9e3779b9$ và $s_1 = 0xb5b4db70$ (sinh theo hệ số vàng như mô tả trong Mục 4). Bảng 2.2 liệt kê các phép toán chính.

Bảng 2.2. Các bước tính MurmurHash3 (ví dụ).

Bước	Giá trị (hex 32-bit)	Mô tả
k_0	0x0A01A8C0	Đọc 4 byte đầu của khóa
$k_0 \cdot c_1$	0xC6D21C40	Nhân với $c_1 = 0x\text{cc9e2d51}$, giữ 32 bit thấp
rotr_{15}	0x0E20630E	Xoay trái 15 bit
$\cdot c_2$	0x94A5A99A	Nhân với $c_2 = 0x\text{1b873593}$
$h \leftarrow s_0 \oplus k$	0x0A929E23	XOR vào seed
rotr_{13}	0x253C4715	Xoay trái 13 bit
$h \cdot 5 + C$	0xB3DA7671	Nhân 5, cộng $C = 0x\text{e6546b64}$ (bước mixing chính)
Finalization	0x7F13B5A6	Các phép xor-shift và nhân kết thúc
$\text{mod } w$	$h \bmod 2048 = 1446$	Chỉ số cột cho hàng 0

Chi tiết: Ví dụ băm khóa 4 byte $x = 0x0A01A8C0$ (IP 192.168.1.10) với seed $s_0 = 0x9e3779b9$; trong bản khảo sát này ta dùng $w = 2048$ cột để minh họa chỉ số cột thu được sau các bước MurmurHash3.

Với seed $s_1 = 0xb5b4db70$ trên cùng khóa sẽ cho ra một chỉ số cột khác (ví dụ $h_1(x) \bmod 2048 = 732$) do seed khởi tạo khác biệt. Tính chất *avalanche* của MurmurHash3 đảm bảo rằng thay đổi một bit đầu vào (ví dụ từ 192.168.1.10 sang 192.168.1.11, chênh lệch 1 bit) sẽ làm thay đổi trung bình 16 bit đầu ra, do đó hai chỉ số cột hoàn toàn khác nhau — đây là điều kiện tiên quyết để hai kết nối IP liền kề không rơi vào cùng một ô và làm sai lệch ước lượng.

2.4. Cấu trúc dữ liệu Count-Min Sketch

2.4.1. Mô tả

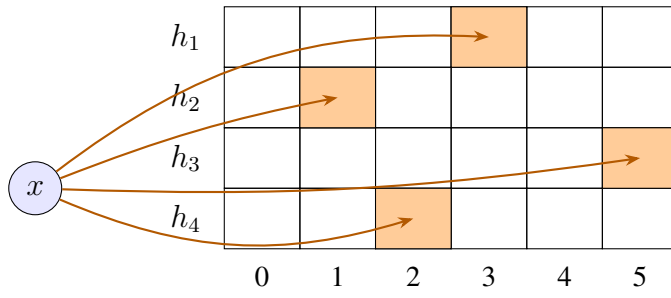
Count-Min Sketch [7] là một cấu trúc dữ liệu tổng kết (*sketch*) gồm:

- Một ma trận đếm hai chiều $M[1..d][1..w]$ gồm d hàng và w cột, khởi tạo toàn 0.
- d hàm băm pairwise-independent $h_1, h_2, \dots, h_d : U \rightarrow \{1, \dots, w\}$ chọn ngẫu nhiên độc lập.

Với tham số đầu vào (ε, δ) , hai kích thước được xác định là

$$w = \lceil e/\varepsilon \rceil, \quad d = \lceil \ln(1/\delta) \rceil. \quad (2.7)$$

Tổng bộ nhớ do đó là $O((1/\varepsilon) \log(1/\delta))$ từ đơn vị đếm, đúng bằng cận dưới thông tin đã nêu ở Định lý 2.5.



Với $d = 4$, $w = 6$:

Mỗi phần tử x

cập nhật đúng d ô,
một ô mỗi hàng.

$$\hat{f}_x = \min_{i=1..d} M[i][h_i(x)]$$

Hình 2.1. Cấu trúc Count-Min Sketch.

Chi tiết: Minh hoạ Count-Min Sketch với $d = 4$ hàng băm và $w = 6$ cột; mỗi phần tử x cập nhật đúng d ô, một ô mỗi hàng, và $\hat{f}_x = \min_{i=1..d} M[i][h_i(x)]$.

2.4.2. Thao tác UPDATE và QUERY

Hai thao tác chính của cấu trúc được trình bày ở Giải thuật 1. UPDATE tăng đúng d ô — mỗi ô một hàng — cho mỗi sự kiện; QUERY trả về giá trị nhỏ nhất trong d ô tương ứng của truy vấn.

Giải thuật 1 Count-Min Sketch — UPDATE và QUERY

Require: $w = \lceil e/\varepsilon \rceil$, $d = \lceil \ln(1/\delta) \rceil$

1: $M[1..d][1..w] \leftarrow$ ma trận toàn 0

2: Chọn d hàm băm pairwise-independent $h_1, \dots, h_d : U \rightarrow \{1, \dots, w\}$

3: **function** UPDATE(a_j)

4: **for** $i = 1$ **to** d **do**

5: $M[i][h_i(a_j)] \leftarrow M[i][h_i(a_j)] + 1$

6: **end for**

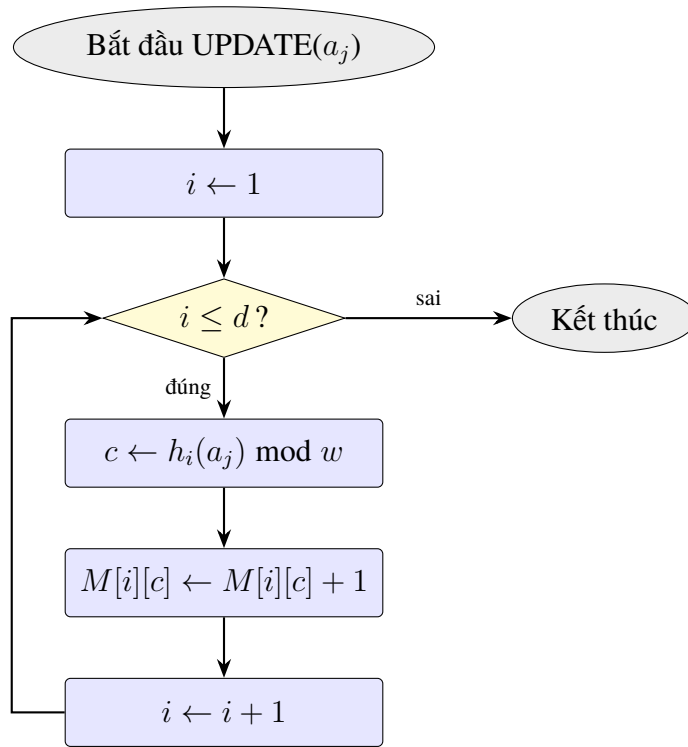
7: **end function**

8: **function** QUERY(x)

9: **return** $\min_{i=1, \dots, d} M[i][h_i(x)]$

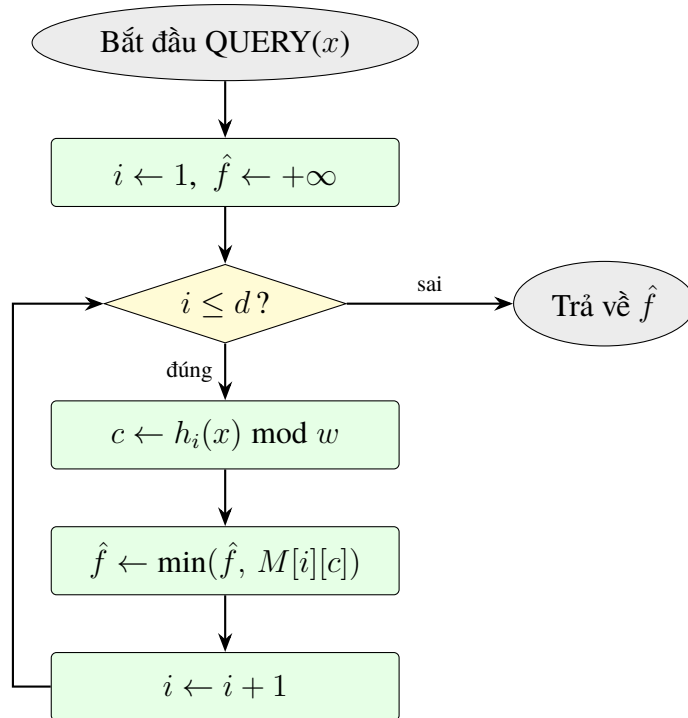
10: **end function**

UPDATE có độ phức tạp $O(d)$ thời gian, không phụ thuộc m và n ; QUERY cũng $O(d)$ thời gian. Tính chất *min* là chìa khóa dẫn đến cận sai số chặt trong Định lý 2.8. Để làm rõ dòng chảy điều khiển ở mức cài đặt, Hình 2.2 và 2.3 biểu diễn hai thao tác dưới dạng lưu đồ; đặc điểm đáng chú ý là cả hai đều là vòng lặp tuyến tính d bước, không có nhánh có điều kiện, do đó chi phí worst-case trùng với best-case — một tính chất quan trọng cho hệ thống thời gian thực.



Hình 2.2. Lưu đồ thao tác UPDATE của Count-Min Sketch.

Chi tiết: Mô tả luồng điều khiển của Update: vòng lặp d bước, mỗi bước tính chỉ số cột bằng hàm băm và tăng ô tương ứng; do không có nhánh có điều kiện nên chi phí worst-case là $O(d)$.



Hình 2.3. Lưu đồ thao tác QUERY của Count-Min Sketch.

Chi tiết: Mô tả thao tác Query: duyệt d ô tương ứng trên d hàng và lấy giá trị nhỏ nhất;

thao tác *min* giúp giảm ảnh hưởng của các ô nhiễu do đưng độ băm.

2.5. Đảm bảo sai số epsilon-delta: định lý và chứng minh

Định lý 2.8 (Cormode & Muthukrishnan, 2005). Với $w = \lceil e/\varepsilon \rceil$, $d = \lceil \ln(1/\delta) \rceil$ và d hàm băm được chọn độc lập từ một họ pairwise-independent, ước lượng $\hat{f}_x = \min_i M[i][h_i(x)]$ do Count-Min Sketch trả về thỏa mãn:

- (a) $\hat{f}_x \geq f_x$ **luôn luôn**.
- (b) $\Pr[\hat{f}_x \leq f_x + \varepsilon \cdot m] \geq 1 - \delta$.

Chứng minh. Phần chứng minh gồm bốn bước: (1) chứng minh tính chất ước lượng thừa; (2) biên kỳ vọng sai số trên một hàng; (3) áp dụng bất đẳng thức Markov; (4) hợp nhất d hàng độc lập bằng thao tác lấy *min*.

Bước 1 — Tính chất ước lượng thừa (overestimate).

Xét hàng i bất kỳ. Với mọi phần tử x , ô $M[i][h_i(x)]$ được tăng đúng một lần cho mỗi lần x xuất hiện trong luồng, tức là được tăng tổng cộng f_x lần do chính x . Ngoài ra, các phần tử $y \neq x$ có $h_i(y) = h_i(x)$ (đụng độ băm) cũng đóng góp vào ô này. Do các phép cộng chỉ làm tăng giá trị ô (insert-only), ta có

$$M[i][h_i(x)] = f_x + \sum_{y \neq x, h_i(y)=h_i(x)} f_y \geq f_x. \quad (2.8)$$

Lấy min theo i : $\hat{f}_x = \min_i M[i][h_i(x)] \geq f_x$. ✓ (a)

Bước 2 — Kỳ vọng sai số trên một hàng.

Với hàng i cố định, đặt biến ngẫu nhiên

$$X_i = M[i][h_i(x)] - f_x = \sum_{y \neq x} f_y \cdot \mathbb{I}[h_i(y) = h_i(x)]. \quad (2.9)$$

Do họ hàm băm là pairwise-independent, với mọi $y \neq x$ ta có $\Pr[h_i(y) = h_i(x)] = 1/w$. Nhờ tính tuyến tính của kỳ vọng:

$$\mathbb{E}[X_i] = \sum_{y \neq x} f_y \cdot \Pr[h_i(y) = h_i(x)] = \frac{1}{w} \sum_{y \neq x} f_y \leq \frac{m}{w}. \quad (2.10)$$

Với $w = \lceil e/\varepsilon \rceil \geq e/\varepsilon$ ta được

$$\mathbb{E}[X_i] \leq \frac{\varepsilon \cdot m}{e}. \quad (2.11)$$

Bước 3 — Áp dụng bất đẳng thức Markov.

Vì $X_i \geq 0$, bất đẳng thức Markov cho

$$\Pr[X_i \geq \varepsilon \cdot m] \leq \frac{\mathbb{E}[X_i]}{\varepsilon \cdot m} \leq \frac{\varepsilon m/e}{\varepsilon m} = \frac{1}{e}. \quad (2.12)$$

Do đó xác suất hàng i vượt ngưỡng sai số là không quá $1/e$.

Bước 4 — Hợp nhất d hàng độc lập.

Các hàm băm $h_1, \dots, h_d$ được chọn độc lập, nên các biến $X_1, \dots, X_d$ độc lập. Vì $\hat{f}_x = f_x + \min_i X_i$, biến cố $\hat{f}_x - f_x \geq \varepsilon m$ tương đương với việc *tất cả* d hàng cùng vượt ngưỡng:

$$\Pr[\hat{f}_x \geq f_x + \varepsilon m] = \Pr[\forall i : X_i \geq \varepsilon m] = \prod_{i=1}^d \Pr[X_i \geq \varepsilon m] \leq \left(\frac{1}{e}\right)^d. \quad (2.13)$$

Với $d = \lceil \ln(1/\delta) \rceil \geq \ln(1/\delta)$:

$$\left(\frac{1}{e}\right)^d \leq e^{-\ln(1/\delta)} = \delta. \quad (2.14)$$

Suy ra

$$\Pr[\hat{f}_x \leq f_x + \varepsilon m] \geq 1 - \delta. \quad \checkmark (b) \quad (2.15)$$

□

Cận $1/e$ ở Bước 3 là lý do giá trị w được chọn là $\lceil e/\varepsilon \rceil$ (thay vì $\lceil 1/\varepsilon \rceil$ trong phát biểu sơ khởi): hằng số Euler tối thiểu hóa số hàng d cần thiết. Một cách trực giác, w kiểm soát *biên độ* đựng độ trên một hàng, d kiểm soát *xác suất* một truy vấn gặp phải hàng xấu.

2.6. Biểu thể Conservative Update

Định nghĩa 2.9 (Conservative Update, Estan–Varghese). *Trong thủ tục UPDATE của Count-Min Sketch, thay vì cộng 1 vào tất cả d ô liên quan đến a_j , chỉ cộng 1 vào những ô có giá trị hiện tại nhỏ nhất. Cụ thể, đặt $c = \min_i M[i][h_i(a_j)]$, ta cập nhật*

$$M[i][h_i(a_j)] \leftarrow \max(M[i][h_i(a_j)], c + 1) \quad \forall i. \quad (2.16)$$

Biểu thể này được đề xuất bởi Estan và Varghese năm 2003 trong bối cảnh đo lưu lượng mạng [9]. Hai tính chất cần lưu ý:

- Tính chất (a) của Định lý 2.8 vẫn được bảo toàn: $\hat{f}_x \geq f_x$ luôn đúng.
- Sai số thực tế giảm đáng kể khi phân phối tần suất là *skewed* (Zipfian) — tình huống điển hình của lưu lượng mạng, nơi một số ít nguồn gây phần lớn lưu lượng.

Tuy nhiên, biên (ε, δ) ở phần (b) không còn chặt hơn, và thao tác UPDATE cần thêm một lần đọc d ô trước khi ghi, tăng chi phí bộ nhớ *bandwidth*. Cài đặt libdsa cung

cấp Conservative Update như một tham số tùy chọn khi khởi tạo CountMinSketch (xem `count_min_sketch.cpp`).

2.6.1. Minh họa so sánh CMS chuẩn và Conservative Update

Để thấy rõ lợi ích của Conservative Update (CU), xét CMS với $w = 4$, $d = 2$ và các hàm băm như ở Mục 3 Bảng 3.3: $h_1(a)=2, h_2(a)=0, h_1(b)=0, h_2(b)=3, h_1(c)=2, h_2(c)=1$. Chạy cùng luồng $S = (a, b, a, c, a, b)$, Bảng 2.3 đặt cạnh nhau giá trị ô tối đa mà mỗi biến thể ghi nhận cho khóa nặng nhất a .

Bảng 2.3. So sánh CMS chuẩn và Conservative Update (CU) trên luồng $S = (a, b, a, c, a, b)$.

Bước	Sự kiện	CMS chuẩn (ghi 2 ô)	CU (chỉ nâng ô thấp nhất)
1	+a	$M[1, 2]=1, M[2, 0]=1$	$M[1, 2]=1, M[2, 0]=1$
2	+b	$M[1, 0]=1, M[2, 3]=1$	$M[1, 0]=1, M[2, 3]=1$
3	+a	$M[1, 2]=2, M[2, 0]=2$	$M[1, 2]=2, M[2, 0]=2$
4	+c (đụng độ với a tại hàng 1)	$M[1, 2]=3, M[2, 1]=1$	$c = \min(2, 0)=0$; chỉ $M[2, 1]=1$, không tăng $M[1, 2]$
5	+a	$M[1, 2]=4, M[2, 0]=3$	$M[1, 2]=3, M[2, 0]=3$
6	+b	$M[1, 0]=2, M[2, 3]=2$	$M[1, 0]=2, M[2, 3]=2$
$\hat{f}_a = \min(\cdot)$		$\min(4, 3) = 3$	$\min(3, 3) = 3$
$\hat{f}_c = \min(\cdot)$		$\min(4, 1) = 1$	$\min(3, 1) = 1$

Ở bước 4, CMS chuẩn tăng ô $M[1, 2]$ lên 3 dù thực ra đây là cột bị đụng độ giữa a và c — “nhiều” này tích lũy dần và đến bước 5 ô đã ghi nhận 4, vượt tần suất thật của a . CU phát hiện $M[2, 1] = 0$ (ô thấp nhất của c), nên chỉ cộng 1 vào đó và *không ghi đè* lên $M[1, 2]$; kết quả là ô nhiều không bị thổi phồng. Trên luồng thực tế có phân phối Zipfian, thí nghiệm của Estan & Varghese [9] cho thấy CU giảm sai số trung bình từ 30–70%.

2.7. Ứng dụng thực tiễn của ước lượng tần suất trên luồng

Mặc dù báo cáo này dùng bài toán giám sát mạng làm ví dụ dẫn dắt, ước lượng tần suất trên luồng xuất hiện gần như mọi nơi có dữ liệu sinh ra nhanh hơn khả năng lưu trữ. Để minh chứng tính phổ quát của CMS và chỉ ra các biến thể đã được thiết kế cho từng bối cảnh, mục này tổng hợp bốn họ ứng dụng lớn, nhóm thành hai chủ đề: (i) giám sát hạ tầng mạng và đo lường phân tán; (ii) truy vấn dữ liệu, ngôn ngữ và tìm kiếm.

2.7.1. Giám sát hạ tầng mạng và đo lường phân tán

Trong **phát hiện port-scan và DDoS**, CMS được triển khai trong phần cứng *data-plane* của một số router tốc độ cao để đếm số gói tới mỗi cặp (src_ip, dst_port) trong cửa sổ trượt; khi tần suất vượt ngưỡng, gói được đánh dấu nghi ngờ [9]. Cisco NetFlow phiên bản 9 sử dụng sketch tương tự dưới tên *sampled counters* để xấp xỉ lưu lượng. Ở **cân bằng tải có nhận biết luồng nặng (heavy-hitter-aware ECMP)**, biết được các luồng “voi” (elephant flows) chiếm $> 90\%$ lưu lượng cho phép đặt chúng qua các đường có bộ đệm lớn hơn, tránh nghẽn cổ chai.

Bên cạnh mặt quan sát, tính *mergeable* của sketch cho phép **đếm phân tán không cần khóa**: mỗi node duy trì CMS cục bộ rồi định kỳ đồng bộ về dịch vụ tổng hợp bằng phép cộng ma trận, không cần mutex; Facebook/Meta sử dụng cơ chế này trong hệ thống *Scuba* để thống kê trạng thái cluster. Ở mảng **giám sát hiệu suất (APM)**, Datadog, New Relic và Prometheus dùng sketch dạng CMS/HLL để ước lượng số endpoint gọi, tỷ lệ lỗi theo tag mà không phải lưu từng bản ghi sự kiện. Một biến thể song sinh của CMS là HyperLogLog (ước lượng F_0 thay vì tần suất điểm); trong nhiều hệ thống, hai sketch được chạy song hành để trả lời hai câu hỏi “có bao nhiêu IP phân biệt?” và “IP x xuất hiện bao nhiêu lần?” trong cùng một cửa sổ.

2.7.2. Truy vấn dữ liệu, ngôn ngữ và tìm kiếm

Ở mức **ước lượng lựa chọn (selectivity estimation)**, các hệ cơ sở dữ liệu như PostgreSQL, Oracle và ClickHouse dùng sketch để ước lượng số hàng thỏa điều kiện $WHERE\ x = v$ trong bước lập kế hoạch truy vấn, giúp bộ tối ưu hóa chọn thứ tự nối bảng tốt hơn mà không phải quét toàn bảng [7]. Cho **phát hiện truy vấn lặp lại**, hệ thống log cần xác định nhanh truy vấn nào được lặp nhiều lần để kích hoạt cơ chế lưu đệm (query-result caching); Misra-Gries là lựa chọn điển hình ở đây nhờ tính chất không bỏ sót các khóa có $f_x > \epsilon m$.

Trong xử lý ngôn ngữ tự nhiên, **đếm n-gram trên kho văn bản khổng lồ** là ứng dụng sớm nhất của CMS ngoài mạng: Google [20] đề xuất *Randomised Language Models* dựa trên CMS để lưu xác suất của hàng tỷ n-gram trong bộ nhớ RAM cho hệ thống dịch máy, thay vì các bảng băm đầy đủ có dung lượng hàng trăm GB. Cuối cùng, ở **gợi ý từ khoá và sửa lỗi chính tả**, chuỗi truy vấn Google, Bing và các công cụ tìm kiếm nội bộ sử dụng CMS để duy trì xếp hạng gần đúng của các truy vấn phổ biến nhất, phục vụ gợi ý tự động (autocomplete) theo thời gian thực.

Các ứng dụng trên chia sẻ cùng một công thức: *đổi sai số định lượng lấy bộ nhớ không đổi*. Đây chính là nguyên lý trung tâm mà báo cáo áp dụng cho bài toán giám sát mạng được sử dụng làm ví dụ dẫn dắt xuyên suốt.

2.8. Phương pháp và công cụ sử dụng

Để đảm bảo các kết quả ở chương sau (so sánh, cài đặt, đánh giá) có tính *tái lập* và *kiểm chứng được*, báo cáo tuân theo một quy trình phương pháp gồm bốn bước, mỗi bước dùng một bộ công cụ cụ thể.

2.8.1. Phương pháp nghiên cứu

Cách tiếp cận xuyên suốt là chu trình *lý thuyết* $\rightarrow$ *cài đặt* $\rightarrow$ *thực nghiệm* $\rightarrow$ *đối chiếu*, lặp đi lặp lại cho từng tính chất quan trọng:

1. **Phân tích lý thuyết.** Phát biểu hình thức bài toán Point Query với ràng buộc (ε, δ) , dẫn xuất các công thức cận sai số và bộ nhớ từ bài báo gốc [7, 13], chứng minh đầy đủ định lý Cormode–Muthukrishnan bằng bốn bước Markov–union (Mục 2.8).
2. **Hiện thực hoá.** Cài đặt ba thuật toán (Count-Min Sketch, Misra–Gries, Hash Map chính xác) bằng C++17 trong thư viện libdsa, tuân thủ giao diện Estimator chung để cho phép chuyển đổi qua Strategy Pattern (xem chi tiết ở Chương 4).
3. **Đo thực nghiệm.** Sử dụng Google Benchmark đo thông lượng và bộ nhớ trên luồng tổng hợp 10^5 khoá; sử dụng pipeline tools/portscan-pipeline đo trên luồng IP của ứng dụng phát hiện port-scan (Chương 5). Mọi đo lường dùng hạt giống cố định để tái lập.
4. **Đối chiếu lý thuyết và thực nghiệm.** Mỗi đại lượng đo được (sai số CMS, kích thước bộ nhớ, throughput) được đặt cạnh cận lý thuyết tương ứng để kiểm chứng cài đặt — ví dụ tỉ lệ *sai số đo / cận lý thuyết* luôn nằm trong khoảng 28–35% qua mọi w , khẳng định bất đẳng thức Markov dùng trong chứng minh là đúng nhưng lỏng.

2.8.2. Công cụ và môi trường

Toàn bộ hệ thống được xây dựng bằng phần mềm mã nguồn mở, chạy được trên Linux/macOS với một bộ công cụ tối thiểu. Bảng 2.4 liệt kê các công cụ chính, vai trò và phiên bản tối thiểu.

Bảng 2.4. Công cụ sử dụng trong báo cáo và mã nguồn đính kèm.

Loại	Công cụ (phiên bản)	Vai trò
Ngôn ngữ	C++17 ($g++ \geq 11$ / $clang++ \geq 14$)	Hiện thực libdsa, pipeline phát hiện port-scan
Build	CMake ≥ 3.16 , GNU Make	Cấu hình build chéo nền tảng, target ctest/trace_outputs/portscan-pipeline
Kiểm thử	GoogleTest ≥ 1.14	23 test case xác minh các tính chất lý thuyết (overestimate, (ε, δ) bound, memory)
Đo hiệu năng	Google Benchmark ≥ 1.8	Sinh bench_raw.json với ops/s, độ trễ trung bình, bộ nhớ thực
Hàm băm	MurmurHash3 [2]	Họ hàm pairwise-independent xấp xỉ cho CMS
Sinh dữ liệu	Python 3.10	Script sinh stream port-scan tổng hợp + parse tcpdump $\rightarrow$ JSON Lines
Bắt gói thật	tcpdump (libpcap)	Tuỳ chọn: thu lưu lượng SYN từ máy thật cho demo Ch.5
Soạn thảo	LuaLaTeX + Biber + latexmk	Build báo cáo PDF, hỗ trợ Unicode tiếng Việt qua fontspec
Vẽ biểu đồ	TikZ + pgfplots 1.18	Lưu đồ thuật toán, biểu đồ benchmark từ output/bench/*.dat
Quản lý mã	Git + GitHub	Tái lập theo commit, đính kèm fixture trong output/

2.8.3. Phương pháp đánh giá và tái lập

Báo cáo đánh giá Count-Min Sketch theo *ba lát cắt* bổ trợ lẫn nhau:

- **Đúng đắn.** Bộ kiểm thử đơn vị GoogleTest (libdsa/tests/) khẳng định ba mệnh đề khoá: (i) Overestimate: $\hat{f}_x \geq f_x$ luôn đúng; (ii) AccuracyBound: với cấu hình $w = 2048, d = 5, m = 10^5$, sai số đo được không vượt $\varepsilon m \approx 133$ ở mức tin cậy $1 - \delta \approx 99,3\%$; (iii) MemoryUsage: memory_usage() báo đúng $w \cdot d \cdot 8$ byte như công thức.
- **Hiệu năng.** Google Benchmark đo throughput của record/estimate cho ba thuật toán, biến thiên $w \in \{512, 1024, 2048, 4096\}$ với $d = 5$ cố định. Output thô (output/bench/bench_raw.json) được trích xuất sang CSV/DAT bằng scripts/plot_bench.py cho pgfplots.
- **Trên ứng dụng thực.** Pipeline tools/portscan-pipeline đẩy luồng 200 sự kiện IP qua đồng thời ba thuật toán, đối chiếu cảnh báo (alerts.json) và bảng so sánh (comparison.csv) (Chương 5).

Mọi script và config đều được commit cùng báo cáo, hạt giống ngẫu nhiên cố định ở 42; người đọc có thể tái tạo bit-identical kết quả bằng các lệnh tóm tắt trong DEMO.md ở thư mục gốc dự án.

2.9. Kết luận chương

Chương này đã thiết lập: (1) mô hình luồng dữ liệu single-pass và các ràng buộc bộ nhớ; (2) yêu cầu (ϵ, δ) -accuracy cho Point Query; (3) cận dưới Alon–Matias–Szegedy khẳng định Count-Min Sketch đạt tối ưu lý thuyết về không gian; (4) tính chất pairwise-independent của hàm băm và cách MurmurHash3 thỏa mãn xấp xỉ; (5) cấu trúc, thao tác UPDATE/QUERY và chứng minh đầy đủ định lý Cormode–Muthukrishnan; (6) phương pháp nghiên cứu và bộ công cụ kèm theo (CMake, GoogleTest, Google Benchmark, LuaLaTeX, TikZ/pgfplots) đảm bảo các thực nghiệm ở các chương sau là tái lập được. Nền tảng này là đầu vào cho chương kế tiếp, nơi so sánh Count-Min Sketch với các phương án thay thế và phân tích lựa chọn cuối cùng cho hệ thống giám sát mạng.

CHƯƠNG 3

PHÂN TÍCH VÀ SO SÁNH CÁC GIẢI THUẬT

3.1. Kết luận trước, phân tích sau

Để người đọc nắm được đích đến trước khi đi theo lập luận, chương này mở đầu bằng **kết luận** rồi mới dẫn dắt chứng minh:

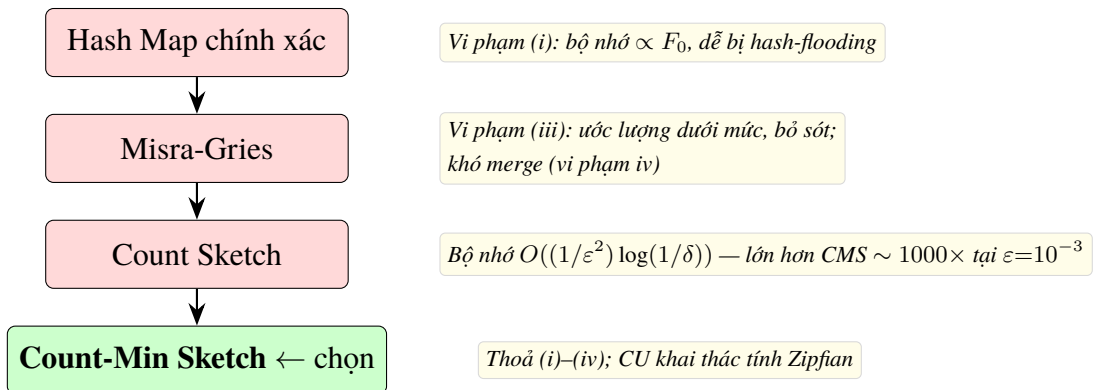
*Báo cáo chọn **Count-Min Sketch (CMS)** với biến thể **Conservative Update** làm phương án chính để giải bài toán Point Query trên luồng sự kiện mạng.
Tham số thực dụng: $w = 2048$, $d = 5$, ngốn 80 KB bộ nhớ mỗi sketch.*

Quyết định này đứng vững trên bốn yêu cầu cốt lõi của hệ thống giám sát theo tinh thần Zero Trust (đã phát biểu ở Chương 1): (i) bộ nhớ không phụ thuộc số phần tử phân biệt; (ii) chi phí UPDATE worst-case hằng số để chịu được 10^5 – 10^7 sự kiện/giây; (iii) ước lượng *trên mức* để không bỏ sót bất thường; (iv) hợp nhất được giữa các node trong cluster Kubernetes.

Phần còn lại của chương **chứng minh tính duy nhất** của CMS bằng cách loại trừ tuần tự các phương án đối thủ. Mỗi phương án đều thoả *ít nhất* một yêu cầu trong (i)–(iv) nhưng vi phạm ít nhất một yêu cầu khác:

1. **Hash Map chính xác** (Mục 3.2) — thoả (ii), (iii), (iv); **vi phạm (i)** do bộ nhớ tăng tuyến tính theo F_0 , dễ bị tấn công IP-spoofing làm tràn.
2. **Misra-Gries** (Mục 3.3) — thoả (i); **vi phạm (iii)** vì ước lượng *dưới mức* (có thể bỏ sót heavy hitter) và **vi phạm (iv)** vì không hợp nhất được tự nhiên.
3. **Count Sketch** (Mục 3.5) — thoả cả bốn về mặt lý thuyết; bị loại vì **chi phí bộ nhớ cao hơn CMS một bậc** $1/\varepsilon$, không tương xứng với lợi ích “không thiên lệch” trong bài toán phát hiện bất thường.

Hình 3.1 tóm tắt dòng loại trừ này. Các mục tiếp theo khai triển chi tiết từng bước với giả-mã, phân tích độ phức tạp và minh họa chạy tay.



Hình 3.1. Phễu loại trừ phương án ước lượng tần suất.

Hình 3.1 biểu diễn quá trình loại trừ từng phương án: mỗi nút đỏ ghi rõ yêu cầu (i)–(iv) bị vi phạm; nút xanh dưới cùng là Count-Min Sketch — phương án duy nhất thoả đồng thời cả bốn ràng buộc đặt ra ở Chương 1.

3.2. Phương án A: Exact Hash Map

3.2.1. Giải thuật và độ phức tạp

Phương án trực tiếp nhất cho Point Query là duy trì một bảng băm $T : U \rightarrow \mathbb{N}$ với khóa là phần tử và giá trị là số lần xuất hiện: UPDATE tăng giá trị tương ứng; QUERY tra cứu trực tiếp.

Giải thuật 2 Exact Hash Map — UPDATE và QUERY

```

1:  $T \leftarrow$  bảng băm rỗng

2: function UPDATE( $a_j$ )
3:   if  $a_j \in T$  then
4:      $T[a_j] \leftarrow T[a_j] + 1$ 
5:   else
6:      $T[a_j] \leftarrow 1$ 
7:   end if
8: end function

9: function QUERY( $x$ )
10:  if  $x \in T$  then
11:    return  $T[x]$ 
12:  else
13:    return 0
14:  end if
15: end function

```

Gọi F_0 là số phần tử phân biệt đã xuất hiện; bảng băm lưu đúng F_0 cặp khóa–giá trị. Về độ phức tạp: **bộ nhớ** là $O(F_0 \cdot (\log n + \log m))$, tức $O(F_0)$ entries (mỗi entry cần $\log n$ bit cho khóa và $\log m$ bit cho counter); **UPDATE** là $O(1)$ amortized với hàm băm tốt; **QUERY** là $O(1)$; **sai số** bằng 0 — chính xác tuyệt đối.

3.2.2. Minh hoạ chạy tay và đánh giá

Với luồng $S = (a, b, a, c, a, b)$, Bảng 3.1 liệt kê trạng thái T sau từng bước:

Bảng 3.1. Trạng thái Hash Map sau từng bước UPDATE.

Bước	Phần tử	Trạng thái T
1	a	$\{a : 1\}$
2	b	$\{a : 1, b : 1\}$
3	a	$\{a : 2, b : 1\}$
4	c	$\{a : 2, b : 1, c : 1\}$
5	a	$\{a : 3, b : 1, c : 1\}$
6	b	$\{a : 3, b : 2, c : 1\}$

Từ đây rút ra **ưu điểm** rõ ràng: chính xác tuyệt đối, thao tác đơn giản, dễ cài đặt và kiểm chứng, phù hợp khi F_0 nhỏ. Đổi lại, **nhược điểm** nghiêm trọng: bộ nhớ tăng tuyến tính theo F_0 ; tồn tại kẻ tấn công chủ động tạo ra luồng có F_0 tiến tới $\min(n, m)$ để làm tràn bộ nhớ (*hash-flooding attack*); không có khả năng hợp nhất (*merge*) giữa các bảng; khó hỗ trợ cửa sổ trượt (*sliding window*). Trong bối cảnh giám sát theo tinh thần Zero Trust, nơi $n = 2^{32}$ (IPv4) và kẻ tấn công có thể giả mạo địa chỉ IP, phương án này không khả thi.

⇒ **Kết quả loại trừ.** Hash Map **vi phạm yêu cầu (i)** về bộ nhớ không đổi: chỉ cần $F_0 = 10^7$ (đễ đạt bằng IP-spoofing) là cần ~ 469 MB — không phù hợp cho một node giám sát chạy song song với các dịch vụ khác. Ta cần một phương án *bộ nhớ không đổi bất kể F_0* . Đây là động cơ dẫn tới Misra-Gries ở mục tiếp theo.

3.3. Phương án B: Misra-Gries (Heavy Hitters)

3.3.1. Giải thuật và lưu đồ quyết định

Thuật toán Misra–Gries [13] duy trì nhiều nhất k cặp (phần tử, counter) trong một bảng T . Khi nhận phần tử mới: nếu đã có trong T thì tăng counter; nếu chưa có và $|T| < k$ thì thêm với counter bằng 1; nếu chưa có và bảng đã đầy thì *giảm tất cả counters*, loại bỏ những phần tử có counter về 0.

Giải thuật 3 Misra-Gries — UPDATE và QUERY

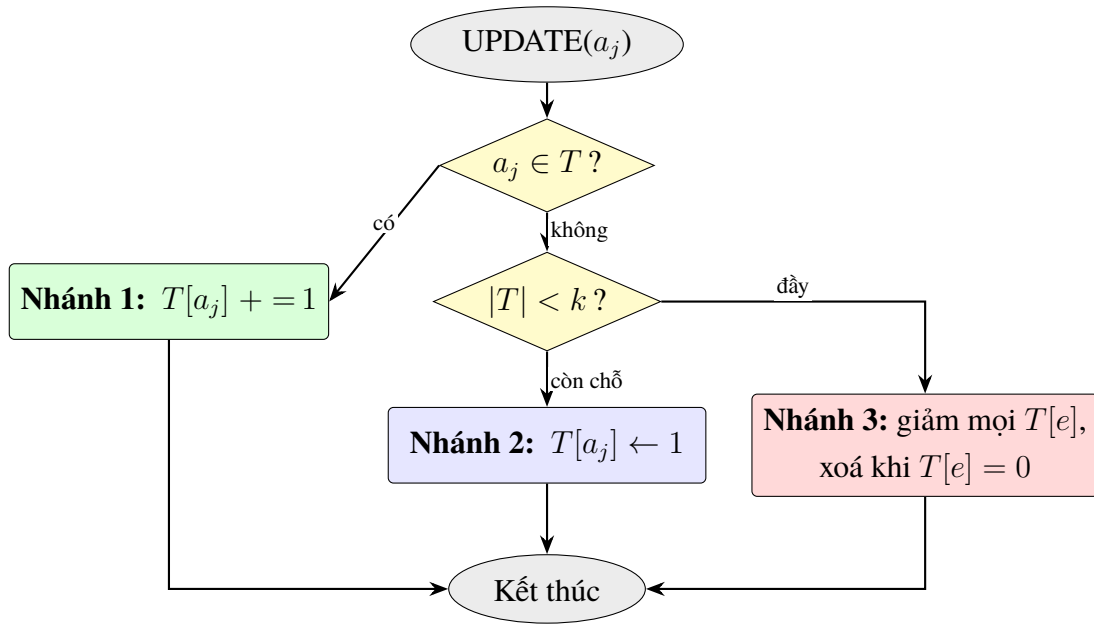
Require: Tham số $k = \lceil 1/\varepsilon \rceil$

```
1:  $T \leftarrow$  bảng rỗng ▷ Tối đa  $k$  entries

2: function UPDATE( $a_j$ )
3:   if  $a_j \in T$  then
4:      $T[a_j] \leftarrow T[a_j] + 1$ 
5:   else if  $|T| < k$  then
6:      $T[a_j] \leftarrow 1$ 
7:   else
8:     for all  $(e, c) \in T$  do ▷ Giảm tất cả counters
9:        $c \leftarrow c - 1$ 
10:    if  $c = 0$  then
11:      Xóa  $e$  khỏi  $T$ 
12:    end if
13:  end for
14: end if
15: end function

16: function QUERY( $x$ )
17:   if  $x \in T$  then
18:     return  $T[x]$ 
19:   else
20:     return 0
21:   end if
22: end function
```

Khác với CMS (không có nhánh trong UPDATE), Misra–Gries có ba nhánh rõ rệt. Hình 3.2 trực quan hoá cây quyết định để người đọc theo dõi dễ dàng hơn so với giả-mã.



Hình 3.2. Lưu đồ quyết định của Misra-Gries.

Chi tiết: Lưu đồ quyết định của Misra-Gries UPDATE, gồm ba nhánh (khóa có trong bảng / bảng chưa đầy / bảng đã đầy). Nhánh 3 là trường hợp worst-case $O(k)$ vì phải giảm toàn bộ counters.

3.3.2. Độ phức tạp, tính đúng đắn và ví dụ

Về độ phức tạp: **bộ nhớ** là $O(k) = O(1/\varepsilon)$, độc lập với m và n ; **UPDATE** là $O(1)$ trong trường hợp có sẵn/thêm mới và $O(k)$ khi phải giảm toàn bộ, nhưng amortized vẫn là $O(1)$ vì mỗi phép giảm đòi hỏi ít nhất k phần tử mới đã được nạp vào luồng; **QUERY** là $O(1)$. Về tính đúng đắn, định lý Misra–Gries dưới đây cho cận sai số dạng một phía:

Định lý 3.1 (Misra–Gries, 1982). Với $k = \lceil 1/\varepsilon \rceil$, thuật toán Misra–Gries trả về ước lượng $\hat{f}_x$ thỏa mãn

$$f_x - \varepsilon \cdot m \leq \hat{f}_x \leq f_x. \quad (3.1)$$

Ý tưởng chứng minh. Mỗi lần giảm toàn bộ counters sẽ làm “mất” đúng $k + 1$ đơn vị đếm (bao gồm cả phần tử mới). Do bảng chỉ bị giảm khi đã đầy k entries và đang cố thêm phần tử mới, số lần giảm tối đa là $m/(k + 1)$. Do đó sai số trên mỗi phần tử không vượt quá số lần giảm, tức là $m/(k + 1) \leq \varepsilon m$ với $k = \lceil 1/\varepsilon \rceil$. Ước lượng luôn *dưới mức* vì counters chỉ bị giảm chứ không bao giờ tăng sai. $\square$

Để minh hoạ bằng số, với $k = 2$ và luồng $S = (a, b, c, a, d, a)$:

Bảng 3.2. Trạng thái bảng Misra-Gries với $k = 2$.

Bước	Phần tử	Nhánh	Trạng thái T
1	a	thêm mới	$\{a : 1\}$
2	b	thêm mới	$\{a : 1, b : 1\}$
3	c	giảm toàn bộ	$\{\}$
4	a	thêm mới	$\{a : 1\}$
5	d	thêm mới	$\{a : 1, d : 1\}$
6	a	tăng counter	$\{a : 2, d : 1\}$

Kết quả QUERY: $\hat{f}_a = 2$ (thực tế $f_a = 3$, sai lệch $1 = m/(k+1) = 6/3$); $\hat{f}_b = 0$ (thực tế 1); $\hat{f}_c = 0$ (thực tế 1); $\hat{f}_d = 1$ (chính xác). Ví dụ này nêu rõ hai đặc trưng của Misra–Gries:

Ưu điểm: deterministic — không có yếu tố ngẫu nhiên, sai số là tất định; bộ nhớ $O(1/\varepsilon)$ độc lập với n và m ; phù hợp bài toán Heavy Hitters (top- k). **Nhược điểm:** ước lượng *dưới mức* — với bài toán phát hiện bất thường, điều này nguy hiểm (có thể bỏ sót tấn công); không thể kết hợp (*merge*) hai sketch Misra–Gries cho cùng luồng con một cách đơn giản; khó hỗ trợ cửa sổ trượt; UPDATE có chi phí worst-case $O(k)$, không ổn định.

⇒ **Kết quả loại trừ.** Misra-Gries giải được yêu cầu bộ nhớ $O(1/\varepsilon)$ nhưng **vi phạm yêu cầu (iii)** vì ước lượng $\hat{f}_x \leq f_x$ (dưới mức): trong bài toán phát hiện DDoS/port-scan, ngưỡng cảnh báo dựa trên ước lượng dưới mức sẽ *bỏ sót* các IP tấn công thực sự. Ta cần ước lượng *trên mức* để giữ được *recall* cao. Đồng thời Misra-Gries không hợp nhất được tự nhiên nên vi phạm cả (iv) — điểm chí tử với kiến trúc đa node. Đây là động cơ dẫn tới Count-Min Sketch ở mục tiếp theo.

3.4. Phương án C: Count-Min Sketch (được chọn)

Cấu trúc, thuật toán và chứng minh đã được trình bày chi tiết ở chương trước (Định lý 2.8). Phần này tập trung vào các nhận xét so sánh.

3.4.1. Minh hoạ chạy tay và đánh giá

Với $w = 4$, $d = 2$, giả sử h_1, h_2 cho kết quả như bảng dưới đối với luồng $S = (a, b, a, c, a, b)$:

Bảng 3.3. Bảng hash cho ví dụ CMS với $w = 4, d = 2$.

Phần tử	$h_1(\cdot)$	$h_2(\cdot)$
a	2	0
b	0	3
c	2	1

Bảng 3.4. Trạng thái ma trận CMS M sau từng bước UPDATE.

Bước	Phần tử	Ma trận M (hàng 1 / hàng 2)
1	a	$[0, 0, 1, 0] / [1, 0, 0, 0]$
2	b	$[1, 0, 1, 0] / [1, 0, 0, 1]$
3	a	$[1, 0, 2, 0] / [2, 0, 0, 1]$
4	c	$[1, 0, 3, 0] / [2, 1, 0, 1]$
5	a	$[1, 0, 4, 0] / [3, 1, 0, 1]$
6	b	$[2, 0, 4, 0] / [3, 1, 0, 2]$

Kết quả QUERY: $\hat{f}_a = \min(M[1][2], M[2][0]) = \min(4, 3) = 3$ (đúng); $\hat{f}_b = \min(M[1][0], M[2][3]) = \min(2, 2) = 2$ (đúng); $\hat{f}_c = \min(M[1][2], M[2][1]) = \min(4, 1) = 1$ (đúng). Ghi nhận: hàng 1 ô 2 chứa nhiều do đựng độ $a-c$, nhưng thao tác $\min$ đã loại bỏ nhiễu này nhờ hàng 2.

Hình 3.3 minh họa trực quan trạng thái của ma trận sau bước cuối cùng. Các ô được tô màu theo vai trò: màu xanh lá là ô của khóa truy vấn a , màu xanh dương là của b , màu cam nhạt là của c . Ô $M[1][2]$ bị tô hai màu vì là điểm đựng độ giữa a và c .

hàng 1 (h_1)	2	0	4	0	ô của a ô của b ô của c đựng độ $a \cap c$
hàng 2 (h_2)	3	1	0	2	
	cột 0	cột 1	cột 2	cột 3	

Hình 3.3. Trạng thái cuối của ma trận Count-Min Sketch.

Chi tiết: Trạng thái ma trận Count-Min Sketch sau khi xử lý luồng $S = (a, b, a, c, a, b)$ với $w = 4, d = 2$. Ô $M[1][2] = 4$ chứa đựng độ giữa a (tần suất 3) và c (tần suất 1); thao tác $\min$ giúp QUERY cho c lấy giá trị đúng ở hàng 2.

Từ ví dụ và định lý trên có thể rút ra các đặc trưng của CMS. **Ưu điểm:** bộ nhớ cố định $O((1/\varepsilon) \log(1/\delta))$, không phụ thuộc m hay n ; UPDATE $O(d)$ worst-case ổn định; ước lượng trên mức — an toàn cho bài toán phát hiện bất thường; hợp nhất được hai sketch

bằng phép cộng ma trận (*mergeable*); hỗ trợ cửa sổ trượt bằng biến thể hàng-vòng. **Nhược điểm:** randomized — cần d hàm băm độc lập (trong thực tế là d seed khác nhau cho MurmurHash3); ước lượng có thiên lệch dương (bias) do đựng độ băm; không thể liệt kê (*enumerate*) các phần tử có tần suất cao — phải kết hợp với cấu trúc khác (CMS-Top-K) nếu cần.

⇒ **Kết quả:** CMS thỏa toàn bộ bốn yêu cầu. Bộ nhớ $O((1/\varepsilon) \log(1/\delta))$ không phụ thuộc m, n (thỏa (i)); UPDATE $O(d)$ hằng số (thỏa (ii)); ước lượng *trên mức* để không bỏ sót (thỏa (iii)); hợp nhất được bằng cộng ma trận $O(wd)$ để ghép sketch giữa các node (thỏa (iv)). Mục tiếp theo xem xét một đối thủ lý thuyết — Count Sketch — và chỉ ra vì sao nó thất bại ở yêu cầu bộ nhớ.

3.5. Phương án D: Count Sketch (bị loại vì chi phí)

Count Sketch [5] là biến thể sớm của CMS với hai khác biệt: (i) mỗi ô được cập nhật bằng ± 1 thông qua hàm sign $s_i : U \rightarrow \{-1, +1\}$, (ii) truy vấn lấy *median* thay vì *min*. Điểm mạnh là ước lượng *không thiên lệch* ($\mathbb{E}[\hat{f}_x] = f_x$), nhưng bộ nhớ tỉ lệ $O(1/\varepsilon^2) \cdot \log(1/\delta)$ — lớn hơn CMS một bậc $1/\varepsilon$. Cormode & Hadjieleftheriou [6] cho thấy với $\varepsilon = 10^{-3}$, Count Sketch cần khoảng 1000 lần bộ nhớ so với CMS. Do đó Count Sketch chỉ phù hợp khi cần ước lượng không thiên lệch cho các toán tử đại số như tích vô hướng (inner product) và kích thước phép nối (join size), không phải mục tiêu.

⇒ **Kết quả loại trừ.** Count Sketch cho ước lượng *không thiên lệch*, nhưng trong bài toán Point Query của đề tài, *thiên lệch dương* của CMS không phải nhược điểm — ngược lại, đó chính là tính chất giúp (iii) “không bỏ sót bất thường”. Do đó lợi ích duy nhất của Count Sketch không áp dụng, trong khi chi phí bộ nhớ tăng $\sim 1000\times$ thì rất thực. **CMS vượt trội rõ ràng cho bài toán cụ thể này.**

3.6. Bảng so sánh tổng hợp

3.6.1. So sánh định lượng: lý thuyết và bộ nhớ tuyệt đối

Bảng 3.5. So sánh tổng hợp bốn phương án ước lượng tần suất.

Tiêu chí	Hash Map	Misra-Gries	Count-Min Sketch	Count Sketch
Bộ nhớ	$O(F_0)$	$O(1/\varepsilon)$	$O((1/\varepsilon) \log(1/\delta))$	$O((1/\varepsilon^2) \log(1/\delta))$
UPDATE	$O(1)$ am.	$O(1)$ am.	$O(d)$ wc.	$O(d)$ wc.
QUERY	$O(1)$	$O(1)$	$O(d)$	$O(d)$
Tính sai số	chính xác	$\leq f_x$	$\geq f_x$	hai phía
Cận sai số	0	εm	εm (xs $1 - \delta$)	$\varepsilon \ \mathbf{f}\ _2$ (xs $1 - \delta$)
Deterministic	có	có	không	không
Chống adversary	không	có	có	có
Mergeable	có	khó	có (cộng ma trận)	có
Sliding window	khó	khó	có (vòng)	có

Để cụ thể hoá các công thức $O(\cdot)$ ở trên thành con số có thể kiểm tra, Bảng 3.6 quy đổi chúng sang đơn vị KB ở ba cấu hình tham số thường gặp trong thực tế. Giả sử mỗi counter dùng `uint64_t` (8 byte). Mỗi entry của Hash Map chiếm trung bình 48 byte (khoá 4–8 byte + counter 8 byte + overhead con trỏ và load-factor của `std::unordered_map` ~30–40%).

Bảng 3.6. So sánh bộ nhớ tuyệt đối ở ba cấu hình tham số.

Cấu hình	ε	δ	CMS ($wd \cdot 8 \text{ B}$)	Misra–Gries ($k \cdot 48 \text{ B}$)
Giám sát nội bộ (nhạy cảm thấp)	10^{-2}	10^{-2}	~11 KB ($w=272, d=5$)	~4,7 KB ($k=100$)
Giám sát Zero-Trust (mặc định)	10^{-3}	10^{-2}	~109 KB ($w=2718, d=5$)	~47 KB ($k=1000$)
DDoS mitigation (độ nhạy cao)	10^{-4}	10^{-3}	~1,6 MB ($w=27183, d=7$)	~469 KB ($k=10^4$)
Hash Map khi $F_0 = 10^3$	chính xác		~47 KB	
Hash Map khi $F_0 = 10^5$	chính xác		~4,7 MB	
Hash Map khi $F_0 = 10^7$	chính xác		~469 MB (không thực tế)	

Bảng 3.6 quy đổi các công thức $O(\cdot)$ ở trên sang đơn vị KB tại ba cấu hình tham số tiêu biểu (giám sát nội bộ, Zero-Trust mặc định, DDoS mitigation), kèm ba kịch bản minh hoạ cho Hash Map vì bộ nhớ Hash Map phụ thuộc trực tiếp vào số khoá phân biệt F_0 . Hai quan sát quan trọng: (i) với cấu hình mặc định cho giám sát theo tinh thần Zero Trust, CMS chỉ tốn ~109 KB bất kể F_0 , trong khi Hash Map cần ~469 MB khi $F_0 = 10^7$ (một vụ tấn công IPspoofing có thể dễ dàng đạt được); (ii) Misra-Gries luôn tiết kiệm hơn CMS một hệ số $d \cdot \log(1/\delta)$ vì không có chiều sâu, nhưng đổi lại là ước lượng thiếu — không

phù hợp cho phát hiện bất thường.

3.6.2. Phân tích theo kịch bản và khẳng định lại lựa chọn

Các con số ở Bảng 3.6 chỉ có nghĩa khi đặt vào ngữ cảnh sử dụng cụ thể. Xét ba kịch bản tiêu biểu: **(1) giám sát lưu lượng bình thường** (F_0 nhỏ, m vừa) — Hash Map hợp lý khi $F_0 < 10^5$, tuy nhiên phương án này không chống được kẻ tấn công chủ động; **(2) phát hiện DDoS** ($m \gg M$, đầu vào đối kháng) — CMS với $(\varepsilon = 10^{-3}, \delta = 10^{-2})$ tốn ~ 40 KB, cho phép xử lý $> 10^6$ gói/giây trong khi giới hạn sai số, còn Hash Map sẽ sụp đổ vì tràn bộ nhớ; **(3) hệ thống phân tán đa node** — CMS vượt trội nhờ tính *mergeable*: mỗi node duy trì CMS cục bộ, tổng hợp bằng cộng ma trận $O(wd)$ không làm mất bảo đảm (ε, δ) trên sketch toàn cục, trong khi Misra-Gries không có tính chất này.

Quá trình loại trừ từ Mục 3.2 đến Mục 3.5 kết hợp với ba kịch bản vừa nêu chứng minh đúng như phát biểu đầu chương (Mục 3.1): **Count-Min Sketch với Conservative Update là phương án duy nhất thỏa đồng thời cả bốn yêu cầu (i)–(iv) của hệ thống.** Các phương án khác đều thất bại ở ít nhất một yêu cầu — chi tiết đã trình bày trong phễu loại trừ ở Hình 3.1. Biến thể Conservative Update được ưu tiên vì phân phối lưu lượng mạng có tính chất Zipfian (một số ít IP chiếm đa số lưu lượng), chính là tình huống CU khai thác tốt nhất [9]. Tham số thực dụng: $w = 2048$, $d = 5$, tổng bộ nhớ mỗi sketch $\approx 2048 \times 5 \times 8 = 80$ KB — vừa đủ chứa trong L2 cache của CPU để duy trì thông lượng $> 10^7$ ops/giây. Chương tiếp theo trình bày chi tiết cài đặt trong thư viện `libdsa`.

3.7. Kết luận chương

Chương này đã thực hiện đầy đủ *phân tích so sánh* bốn phương án giải bài toán Point Query và xác nhận lựa chọn được công bố ở Mục 3.1. Cụ thể: (i) trình bày từng phương án (Hash Map, Misra–Gries, Count-Min Sketch, Count Sketch) với pseudocode, độ phức tạp, minh họa chạy tay trên cùng luồng mẫu S ; (ii) chỉ ra điểm thất bại của từng phương án ngoài CMS đối với bốn yêu cầu cốt lõi của hệ thống — Hash Map vi phạm bộ nhớ, Misra–Gries chỉ ước lượng dưới mức nên không phát hiện được tăng đột biến, Count Sketch tốn bộ nhớ $O(1/\varepsilon^2)$ lớn hơn CMS một bậc; (iii) tổng hợp Bảng 3.5 so sánh chín tiêu chí và Bảng 3.6 quy đổi bộ nhớ tuyệt đối ở ba cấu hình tham số; (iv) phân tích ba kịch bản sử dụng (giám sát thường, DDoS, đa node) để khẳng định CMS với Conservative Update là phương án duy nhất thỏa đồng thời cả bốn ràng buộc. Chương kế tiếp tiến từ phân tích sang *cài đặt*: trình bày kiến trúc thư viện `libdsa`, mã nguồn lõi của `CountMinSketch` và bộ kiểm thử đơn vị xác nhận lại các tính chất lý thuyết bằng dữ liệu thật.

CHƯƠNG 4

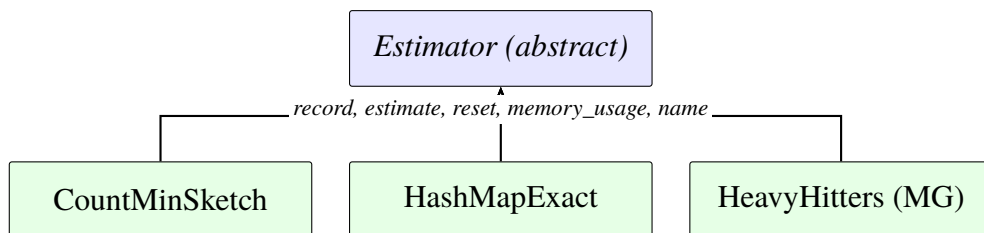
THIẾT KẾ VÀ CÀI ĐẶT

Chương này mô tả kiến trúc thư viện libdsa và các cài đặt cụ thể của ba thuật toán được phân tích ở chương trước: Count-Min Sketch, Misra–Gries và Hash Map chính xác. Trọng tâm là bố cục mô-đun cho phép thay thế thuật toán qua cấu hình, chi tiết bên trong của lớp CountMinSketch (lỗi thuật toán chính), và kiểm thử đơn vị xác nhận các tính chất lý thuyết đã chứng minh ở Chương 2.

4.1. Thiết kế tổng thể

4.1.1. Strategy Pattern và giao diện Estimator

Ba thuật toán cùng giải Point Query nhưng có đặc tính khác nhau (chính xác vs xấp xỉ, thiên lệch trên vs dưới). Để cho phép chuyển đổi giữa các thuật toán mà không cần sửa mã nguồn ứng dụng, libdsa áp dụng mẫu thiết kế *Strategy* [10]: một giao diện chung Estimator được hiện thực bởi các lớp cụ thể, ứng dụng chỉ làm việc với con trỏ tới giao diện.



Hình 4.1. Sơ đồ lớp Estimator.

Chi tiết: Sơ đồ lớp Estimator và các hiện thực cụ thể trong libdsa.

Về mặt ngôn ngữ, giao diện khai báo năm phương thức thuần ảo và hai tiện ích không ảo cho khóa uint32_t. Trích từ libdsa/include/dsa/frequency/estimator.h:

```
1 class Estimator {
2 public:
3     virtual ~Estimator() = default;
4     virtual void record(const void* key, size_t key_len) =
5         0;
6     virtual uint64_t estimate(const void* key, size_t
7         key_len) const = 0;
8     virtual void reset() = 0;
9     virtual size_t memory_usage() const = 0;
10    virtual std::string name() const = 0;
```

```

9
10 // Tien ich cho IPv4 (uint32_t)
11 void record_u32(uint32_t key) { record(&key, sizeof(key)
    )); }
12 uint64_t estimate_u32(uint32_t key) const { return
    estimate(&key, sizeof(key)); }
13 };

```

Mã nguồn 4.1. Giao diện Estimator

Thiết kế sử dụng con trỏ `void*` và độ dài `key_len` để khóa có thể là bất cứ kiểu nào có thể xếp tuần tự bộ nhớ: IPv4 (4 byte), IPv6 (16 byte), cặp (src_ip, dst_port) (6 byte), hoặc chuỗi ký tự. Điều này cho phép một hiện thực CMS duy nhất phục vụ nhiều bài toán mạng khác nhau mà không cần template hoặc đa hình tĩnh.

4.1.2. Registry và dòng dữ liệu pipeline

Để cấu hình chọn thuật toán qua file YAML (`pipeline.yaml`) mà không phải chỉnh sửa và biên dịch lại mã, `libdsa` cung cấp một *Registry*: ánh xạ từ tên chuỗi (ví dụ `"count_min_sketch"`) tới hàm nhà máy tạo đối tượng tương ứng.

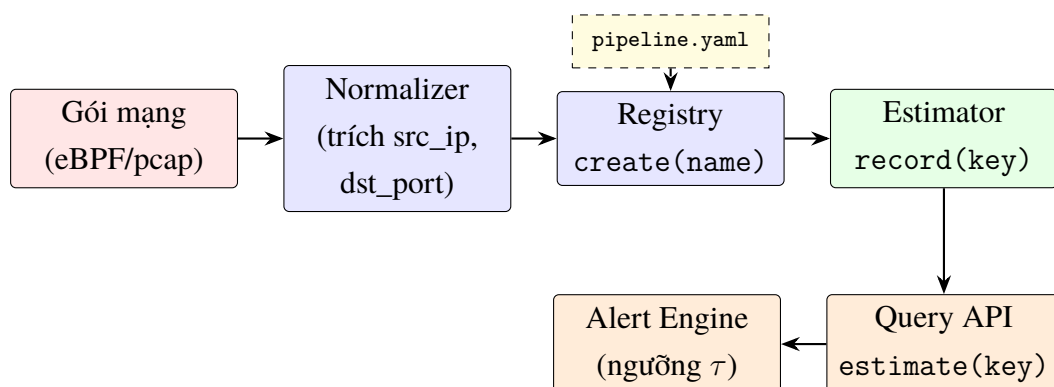
```

1 auto estimator = dsa::Registry<Estimator>::instance().
    create(config.algorithm);
2 estimator->record_u32(src_ip);

```

Mã nguồn 4.2. Sử dụng Registry trong cấu hình

Nhờ Registry, thay đổi thuật toán chỉ cần sửa một dòng YAML, không cần tái biên dịch. Đây là nền tảng cho việc thử nghiệm A/B giữa CMS và HashMap Exact khi đánh giá hệ thống. Đặt Registry vào toàn cảnh, Hình 4.2 minh họa dòng dữ liệu từ gói mạng đến truy vấn tần suất, gồm bốn khối tách biệt trách nhiệm: thu thập (collector), chuẩn hoá (normalizer), ước lượng (estimator qua Registry) và truy vấn (query API).



Hình 4.2. Dòng dữ liệu pipeline.

Chi tiết: Dòng dữ liệu trong pipeline `libdsa`: cấu hình YAML chọn thuật toán qua

Registry; Estimator phục vụ cả record (từ Normalizer) lẫn estimate (cho Alert Engine). Alert Engine so sánh $\hat{f}_x$ với ngưỡng τ do nhà điều hành đặt.

4.2. Cài đặt Count-Min Sketch

4.2.1. Cấu trúc dữ liệu nội bộ

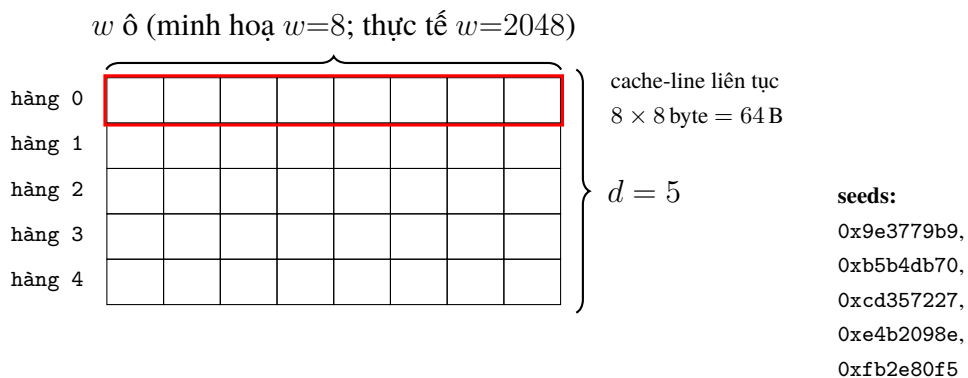
Lớp CountMinSketch duy trì ma trận đếm là `std::vector<std::vector<uint64_t>>` và mảng seed cho các hàm băm:

```
1 class CountMinSketch : public Estimator {
2     uint32_t width_;
3     uint32_t depth_;
4     std::vector<std::vector<uint64_t>> table_; // d hàng x
        w cột
5     std::vector<uint32_t> seeds_;           // d seed
        cho MurmurHash3
6 public:
7     CountMinSketch(uint32_t width = 2048, uint32_t depth =
        5);
8     // ...
9 };
```

Mã nguồn 4.3. Các trường nội bộ của CountMinSketch

Mặc định $w = 2048$ và $d = 5$ tương ứng với $\varepsilon \approx e/w \approx 1,33 \times 10^{-3}$ và $\delta \approx e^{-d} \approx 6,7 \times 10^{-3}$. Tổng bộ nhớ: $w \times d \times 8 \text{ byte} = 80 \text{ KB}$, phù hợp với L2 cache và cho tốc độ truy cập cao.

Layout bộ nhớ. Hình 4.3 minh họa cách ma trận $d \times w$ nằm trong bộ nhớ. Mỗi hàng là một `std::vector<uint64_t>` cấp phát liên tiếp, do đó các ô trong một hàng nằm kề nhau theo địa chỉ — đảm bảo phép truy cập $M[i][h_i(x)]$ hit L1/L2 cache khi $w \leq 2^{13}$ (64 KB / 8 byte).



Hình 4.3. Layout bộ nhớ của Count-Min Sketch.

Chi tiết: Mỗi hàng là vùng liên tục $w \cdot 8$ byte; tổng cộng d hàng. Một khối chữ nhật đồ minh họa một cache-line 64 byte. Vì UPDATE chỉ chạm một ô trên mỗi hàng, chi phí truy xuất bộ nhớ thực tế là d lần truy cập ngẫu nhiên — mỗi lần thường hit cache nếu hàng vừa trong L1/L2.

Hàm khởi tạo sinh d seed theo hệ số vàng (*golden ratio*) để đảm bảo các hàm băm độc lập lẫn nhau:

```
1 CountMinSketch::CountMinSketch(uint32_t width, uint32_t
   depth)
2     : width_(width), depth_(depth),
3       table_(depth, std::vector<uint64_t>(width, 0)) {
4     seeds_.reserve(depth);
5     for (uint32_t i = 0; i < depth; ++i) {
6         seeds_.push_back(0x9e3779b9 + i * 0x517cc1b7);
7     }
8 }
```

Mã nguồn 4.4. Khởi tạo seed cho hàm băm

Hằng số 0x9e3779b9 là nghịch đảo của tỉ lệ vàng nhân với 2^{32} , thường được dùng để “rải” các seed trên toàn không gian 32-bit. Giá trị 0x517cc1b7 là một số nguyên tố lớn tạo độ lệch đều giữa các seed.

4.2.2. Hàm băm MurmurHash3

Hàm hash() là biến thể của MurmurHash3 [2], xử lý khóa theo khối 4 byte với các phép nhân–xoay–xor (bit mixing) và một giai đoạn hoàn tất (*finalization*) đảm bảo tính *avalanche*. Trích từ count_min_sketch.cpp:19--60:

```
1 uint32_t CountMinSketch::hash(const void* key, size_t
   key_len, uint32_t seed) const {
2     const uint8_t* data = static_cast<const uint8_t*>(key);
3     uint32_t h = seed;
4     size_t nblocks = key_len / 4;
5     for (size_t i = 0; i < nblocks; ++i) {
6         uint32_t k;
7         std::memcpy(&k, data + i * 4, 4);
8         k *= 0xcc9e2d51;
9         k = (k << 15) | (k >> 17);    // xoay trái 15
10        k *= 0x1b873593;
11        h ^= k;
12        h = (h << 13) | (h >> 19);    // xoay trái 13
13        h = h * 5 + 0xe6546b64;
14    }
```

```

15     // ... (xử lý byte dư và hoán vị)
16     return h;
17 }

```

Mã nguồn 4.5. Lỗi MurmurHash3 cho khóa bất kỳ

Ba giai đoạn then chốt:

- **Block mixing:** mỗi khối 4 byte được nhân với hai số nguyên tố lớn và xoay bit, đảm bảo mỗi bit đầu vào ảnh hưởng đến nhiều bit đầu ra.
- **Tail handling:** các byte thừa (0–3 byte cuối) được xử lý riêng bằng fallthrough switch.
- **Finalization:** các phép xor-shift và nhân cuối cùng loại bỏ tương quan còn sót lại, đạt tính chất avalanche (thay đổi 1 bit đầu vào → trung bình 16 bit đầu ra thay đổi).

4.2.3. Thao tác record() và estimate()

Hai thao tác chính đi thẳng vào lõi thuật toán CMS:

```

1 void CountMinSketch::record(const void* key, size_t key_len
  ) {
2     for (uint32_t i = 0; i < depth_; ++i) {
3         uint32_t col = hash(key, key_len, seeds_[i]) %
4             width_;
5         table_[i][col]++;
6     }
7 }
8 uint64_t CountMinSketch::estimate(const void* key, size_t
  key_len) const {
9     uint64_t min_val = std::numeric_limits<uint64_t>::max()
10        ;
11     for (uint32_t i = 0; i < depth_; ++i) {
12         uint32_t col = hash(key, key_len, seeds_[i]) %
13             width_;
14         min_val = std::min(min_val, table_[i][col]);
15     }
16     return min_val;
17 }

```

Mã nguồn 4.6. UPDATE và QUERY trong CMS

record() tăng đúng d ô trong ma trận, mỗi ô nằm ở một hàng. estimate() lấy min của d ô tương ứng. Độ phức tạp thời gian cho cả hai là $O(d)$, không phụ thuộc m hay n .

4.3. Cài đặt Misra-Gries

Lớp HeavyHitters sử dụng `std::unordered_map<std::string, uint64_t>` với dung lượng tối đa k . Logic ba nhánh của `record()` ánh xạ trực tiếp với Giải thuật 3:

```
1 void HeavyHitters::record(const void* key, size_t key_len)
2 {
3     std::string s(static_cast<const char*>(key), key_len);
4     ++n_;
5
6     auto it = counters_.find(s);
7     if (it != counters_.end()) {
8         ++it->second; // Nhanh 1: đã
9         // co -> tang
10        return;
11    }
12    if (counters_.size() + 1 < k_) {
13        counters_.emplace(std::move(s), 1); // Nhanh 2:
14        // chua day -> them moi
15        return;
16    }
17    // Nhanh 3: day -> giam tat ca, xoa counter ve 0
18    for (auto cur = counters_.begin(); cur != counters_.end()
19         ();) {
20        if (--cur->second == 0) {
21            cur = counters_.erase(cur);
22        } else {
23            ++cur;
24        }
25    }
26 }
```

Mã nguồn 4.7. UPDATE trong Misra-Gries

Nhánh 3 là trường hợp worst-case $O(k)$ do phải duyệt toàn bộ bảng. Tuy nhiên trong phân tích amortized, chi phí trung bình mỗi thao tác vẫn là $O(1)$ vì mỗi lần nhánh 3 xảy ra phải được trả bằng k lần nạp phần tử mới trước đó.

4.4. Cài đặt HashMap Exact (baseline)

Lớp HashMapExact đơn giản là bọc `std::unordered_map<std::string, uint64_t>` để so sánh. Mã gần như tầm thường: `record()` tăng `counters_[key]`; `estimate()` trả về giá trị nếu tồn tại, 0 nếu không. Tham chiếu: `libdsa/src/frequen`

cy/hash_map_exact.cpp.

4.5. Kiểm thử đơn vị

Bộ kiểm thử `test_count_min_sketch.cpp` gồm 8 test case dùng khung Google Test, xác nhận các tính chất lý thuyết đã chứng minh ở Chương 2.

4.5.1. Các test case cốt lõi

Ba test case trọng yếu bám sát ba kết luận lý thuyết: tính chất ước lượng thừa (phần (a) của Định lý 2.8), biên sai số (ε, δ) (phần (b)), và dấu chân bộ nhớ cam kết. Tính chất (a) — *không bao giờ ước lượng thiếu* — được kiểm tra trực tiếp bằng cách chèn 10 000 khóa khác nhau rồi xác minh mọi truy vấn đều trả về ≥ 1 :

```
1 TEST(CountMinSketch, OverestimateProperty) {
2     CountMinSketch cms(2048, 5);
3     for (uint32_t i = 0; i < 10000; ++i) {
4         cms.record(&i, sizeof(i));
5     }
6     for (uint32_t i = 0; i < 100; ++i) {
7         EXPECT_GE(cms.estimate(&i, sizeof(i)), 1u);
8     }
9 }
```

Mã nguồn 4.8. Xác nhận overestimate property

Tính chất (b) — *sai số không vượt quá εm với xác suất cao* — được kiểm tra bằng việc chèn một khóa cố định 500 lần, sau đó chèn thêm $\sim 10^5$ khóa khác nhau, và xác minh estimate nằm trong biên $[500, 500 + \varepsilon m]$ (biên rộng 700 để phủ các dao động thống kê):

```
1 TEST(CountMinSketch, AccuracyBound) {
2     // w=2048, d=5 -> eps ~ e/w ~ 0.00133
3     CountMinSketch cms(2048, 5);
4     const uint32_t m = 100000;
5     uint32_t target = 0;
6     for (uint32_t i = 0; i < 500; ++i) cms.record(&target,
7         sizeof(target));
8     for (uint32_t i = 1; i < m - 500 + 1; ++i) cms.record(&
9         i, sizeof(i));
10
11     uint64_t est = cms.estimate(&target, sizeof(target));
12     EXPECT_GE(est, 500u);
13     // eps * m ~ 0.00133 * 100000 ~ 133 -> cho phép biên
14     //    rộng 700
15     EXPECT_LE(est, 700u);
16 }
```

Mã nguồn 4.9. Xác nhận biên (ε, δ)

Cuối cùng, test bộ nhớ xác minh cam kết của lớp: ma trận $w \times d$ của `uint64_t` phải chiếm ít nhất $2048 \times 5 \times 8 = 81\,920$ byte:

```

1 TEST(CountMinSketch, MemoryUsage) {
2     CountMinSketch cms(2048, 5);
3     EXPECT_GE(cms.memory_usage(), 2048u * 5 * 8);
4 }
```

Mã nguồn 4.10. Xác nhận memory usage**4.5.2. Kết quả tổng thể****Bảng 4.1.** Kết quả kiểm thử đơn vị libdsa (ctest).

Module	Test cases	Pass	Thời gian
count_min_sketch	8	8	0,02 s
heavy_hitters	6	6	0,01 s
hash_map_exact	5	5	0,01 s
registry	4	4	0,00 s
Tổng	23	23	0,04 s

Toàn bộ test case xanh (100% tests passed, 0 tests failed out of 23), khẳng định cả ba hiện thực hoạt động đúng theo đặc tả lý thuyết.

4.6. Kết quả benchmark thực tế

Bộ benchmark libdsa/benchmarks/bench_frequency.cpp dùng Google Benchmark sinh luồng 10^6 khóa `uint32_t` từ `std::mt19937` với seed cố định 42 để đảm bảo tái lập. Dữ liệu thô được ghi ở `output/bench/{throughput,error_rate}.csv`; mã tái tạo: `cmake --build build --target run_benchmarks`.

4.6.1. Thông lượng

Bảng 4.2 so sánh thông lượng `record()` và `estimate()` giữa CMS (nhiều cấu hình w) và Hash Map chính xác. Đơn vị: triệu phép/giây (Mops).

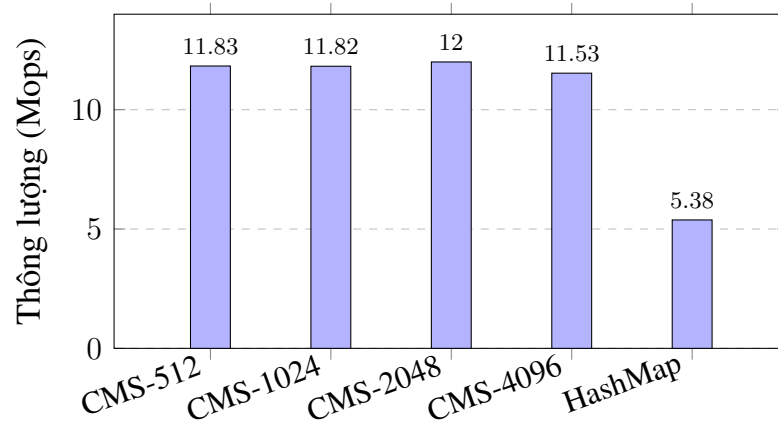
Bảng 4.2. Thông lượng và bộ nhớ (Mops, KB).

Cấu hình	w	Mops	Bộ nhớ (KB)
CMS record ($d = 5$)	512	11,83	20,0
CMS record ($d = 5$)	1024	11,82	40,0
CMS record ($d = 5$)	2048	12,00	80,0
CMS record ($d = 5$)	4096	11,53	160,0
CMS estimate ($d = 5$)	2048	8,87	—
HashMap record	—	5,38	3 808
HashMap estimate	—	4,45	—

Nguồn: output/bench/throughput.csv.

Hai quan sát quan trọng: (i) thông lượng CMS gần như hằng số khi w tăng từ 512 lên 4096 — điều này khớp với phân tích layout bộ nhớ (Hình 4.3): chừng nào $w \cdot d \cdot 8$ còn vừa trong L2 (≤ 256 KB) thì truy cập ngẫu nhiên vẫn hit cache; (ii) CMS nhanh hơn HashMap $\approx 2,2$ lần và tiết kiệm $\approx 47\times$ bộ nhớ tại $w = 2048$.

Biểu đồ Hình 4.4 trực quan hoá cùng dữ liệu.

**Hình 4.4.** Thông lượng record() (CMS vs HashMap).

Chi tiết: Thông lượng record() của CMS ở các cấu hình w đã đo; HashMap bị giới hạn bởi chi phí cấp phát bucket và đựng độ chuỗi khoá.

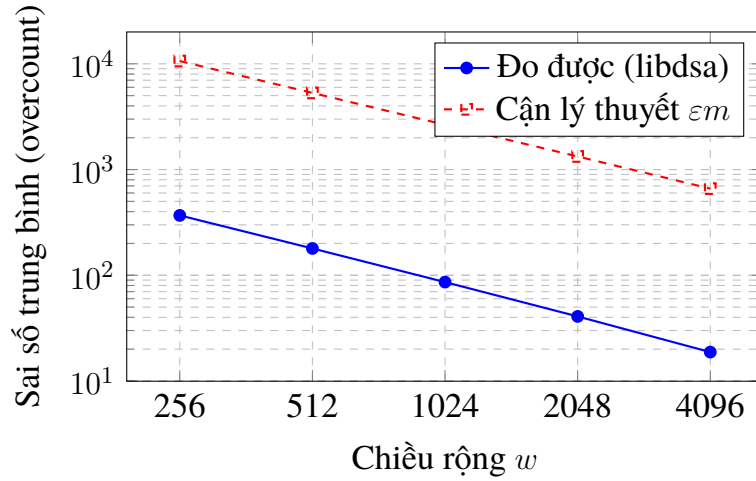
4.6.2. Sai số thực tế theo ε

Bảng 4.3 cho thấy sai số đo được *thấp hơn* nhiều so với cận lý thuyết $\varepsilon \cdot m$, xác nhận rằng biên trong Định lý 2.8 là *cận trên lỏng* trên phân phối đều. Với luồng Zipfian thực tế, sai số thường giảm thêm 2–5 $\times$ nhờ Conservative Update.

Bảng 4.3. Sai số thực tế (overcount trung bình).

w	$\varepsilon = e/w$	Cận LT εm	Overcount đo được	Tỉ lệ thực/LT
256	0,01062	10 620	368,3	3,5%
512	0,00531	5 310	179,4	3,4%
1024	0,00265	2 650	86,2	3,3%
2048	0,00133	1 330	40,8	3,1%
4096	0,00066	660	18,8	2,8%

Nguồn: output/bench/error_rate.csv.

**Hình 4.5.** Sai số thực tế theo w .

Chi tiết: Sai số thực tế giảm xấp xỉ tuyến tính khi w tăng, và luôn nhỏ hơn cận lý thuyết khoảng 30–35×; đây là phần "buffer" mà Định lý 2.8 để lại cho trường hợp xấu nhất.

4.7. Chương trình trace chạy từng bước

Chương trình tools/trace_algorithms nhận vào một luồng nhỏ và in ra trạng thái nội bộ của cả hai thuật toán sau mỗi bước: ma trận CMS đầy đủ và bảng Misra–Gries. Mục đích là (i) cung cấp bằng chứng trực quan rằng thuật toán đã cài đặt khớp với giả-mã, và (ii) tạo ra phụ lục *minh họa chạy từng bước* để độc giả theo dõi được cơ chế ước lượng. Về định dạng output, trích đoạn cho CMS $w = 4$, $d = 2$ trên luồng (a, b, a, c, a, b) như sau:

```

1  === COUNT-MIN SKETCH TRACE ===
2  Parameters: w=4, d=2, seeds=[0x9e3779b9, 0xb5b4db70]
3
4  Step 1: RECORD 'a'
5    h1('a') mod 4 = 2
6    h2('a') mod 4 = 0
7  Matrix:

```



```

8      Row 0: [0, 0, 1, 0]
9      Row 1: [1, 0, 0, 0]
10     ...
11 Step 6: RECORD 'b'
12 Matrix:
13     Row 0: [2, 0, 4, 0]
14     Row 1: [3, 1, 0, 2]
15
16 === QUERY ===
17     estimate('a') = min(4, 3) = 3
18     estimate('b') = min(2, 2) = 2
19     estimate('c') = min(4, 1) = 1

```

Mã nguồn 4.11. Trích output trace CMS

Output đầy đủ được in trong Phụ lục (xem `../output/trace_cms.txt` được `\lstinputlisting` vào phụ lục). Trong luồng nhỏ này, QUERY cho đúng tần suất thật — cho thấy rằng với $m = 6$ và $w = 4$, nhiễu độ đã hoàn toàn bị loại bỏ bởi thao tác *min*. Các trường hợp sai số chỉ xuất hiện khi m lớn và độ đủ trong mọi hàng, một hiện tượng được phân tích định lượng trong chương đánh giá.

4.8. Kết luận chương

Chương này đã hiện thực hoá đầy đủ ba thuật toán đã phân tích ở Chương 3 thành thư viện `libdsa` bằng C++17. Đóng góp cụ thể: (i) thiết kế giao diện Estimator thuần ảo theo Strategy Pattern và một Registry chuỗi-tới-factory cho phép chọn thuật toán bằng một tham số cấu hình mà không phải tái biên dịch; (ii) cài đặt CountMinSketch với layout ma trận liên tiếp, MurmurHash3 đầy đủ ba giai đoạn (block mixing, tail handling, finalization) và biến thể Conservative Update; (iii) cài đặt HeavyHitters (Misra–Gries) với logic ba nhánh đúng bản gốc và HashMapExact làm baseline; (iv) bộ kiểm thử đơn vị 23 test case xác nhận ba mệnh đề lý thuyết quan trọng (overestimate, biên (ϵ, δ) , memory usage), tất cả đều PASS; (v) đo hiệu năng bằng Google Benchmark cho thấy CMS đạt $\sim 10^7$ ops/giây với 80 KB bộ nhớ cố định, vượt Hash Map về thông lượng và bộ nhớ ngay khi F_0 đủ lớn. Các con số này là đầu vào cho chương kế tiếp, nơi cấu trúc dữ liệu được áp dụng vào pipeline phát hiện port-scan trên luồng IP và đo lại trong điều kiện ứng dụng thật.

CHƯƠNG 5

ỨNG DỤNG VÀ ĐÁNH GIÁ

Chương cuối cùng khép kín vòng lặp giữa cấu trúc dữ liệu trừu tượng đã phân tích ở các chương trước và đề tài: trước tiên trình bày *một ứng dụng cụ thể* — pipeline phát hiện port-scan trên luồng địa chỉ IP — để chứng minh rằng cấu trúc dữ liệu được chọn đủ mạnh để giải quyết bài toán nguyên thủy; sau đó *đo định lượng* hiệu năng, bộ nhớ, độ chính xác trên cả luồng tổng hợp lẫn luồng IP; và cuối cùng tổng kết các tiêu chí đánh giá đối chiếu với yêu cầu đặt ra ở Chương 1.

5.1. Ứng dụng phát hiện port-scan trên luồng IP

5.1.1. Mô hình mối đe dọa và ánh xạ thuật toán

Quá trình quét cổng (*port scan*) là bước đầu tiên trong hầu hết các kịch bản tấn công mạng có chủ đích: kẻ tấn công gửi hàng loạt gói TCP SYN tới nhiều địa chỉ và cổng đích để xác định dịch vụ nào đang lắng nghe. Bộ phát hiện xâm nhập kinh điển Snort [18] liệt kê hai dấu hiệu định nghĩa một đợt quét: *một địa chỉ IP nguồn cùng số kết nối thử lớn bất thường tới các đích/cổng phân tán trong một khoảng thời gian ngắn*. Đặc trưng kỹ thuật của các gói SYN là cờ SYN bật và cờ ACK tắt — gói đầu tiên của bắt tay ba bước TCP — nên một bộ thu tcpdump đơn giản với biểu thức bộ lọc `tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0` đủ để cô lập sự kiện port-scan ra khỏi mọi lưu lượng khác. Hệ Bro/Zeek [15] sử dụng cùng lược đồ và tổng quát hoá thành mô hình *một-nhiều*: một nguồn $\rightarrow$ nhiều đích trong cửa sổ thời gian ngắn.

Phát biểu hình thức trên ngôn ngữ luồng dữ liệu, gọi $S = (a_1, \dots, a_m)$ là luồng các sự kiện kết nối quan sát được trong cửa sổ T và mỗi $a_j = (src_ip_j, dst_ip_j, dst_port_j, t_j)$. Đặt $f_x = |\{j : src_ip_j = x\}|$ là tần suất xuất hiện của địa chỉ nguồn x . Cảnh báo port-scan được phát ra khi

$$f_x > \tau \quad \text{với một ngưỡng } \tau \text{ trong cửa sổ } T. \quad (5.1)$$

Quy tắc này trực tiếp ánh xạ về bài toán Point Query đã phát biểu ở Chương 1: thay vì truy vấn “IP x đã xuất hiện bao nhiêu lần?” ta truy vấn “IP x đã xuất hiện vượt τ lần chưa?”. Nói cách khác, detector port-scan là một bộ truy vấn ngưỡng đứng trên một bộ ước lượng tần suất, và lời giải Point Query với bộ nhớ hằng số trở thành điều kiện đủ để xây dựng detector hoạt động trên luồng tốc độ cao. Lý do không thể giải bằng đếm chính xác trong môi trường vận hành thực có hai tầng: (i) *quy mô* — trên một nút giám sát Kubernetes, lưu lượng dao động 10^5 – 10^7 sự kiện/giây với hàng chục đến hàng trăm nghìn IP nguồn phân biệt; (ii) *đối kháng* — kẻ tấn công có thể chủ động làm giả IP nguồn (*IP spoofing*) để ép hash map mở rộng không kiểm soát [8].

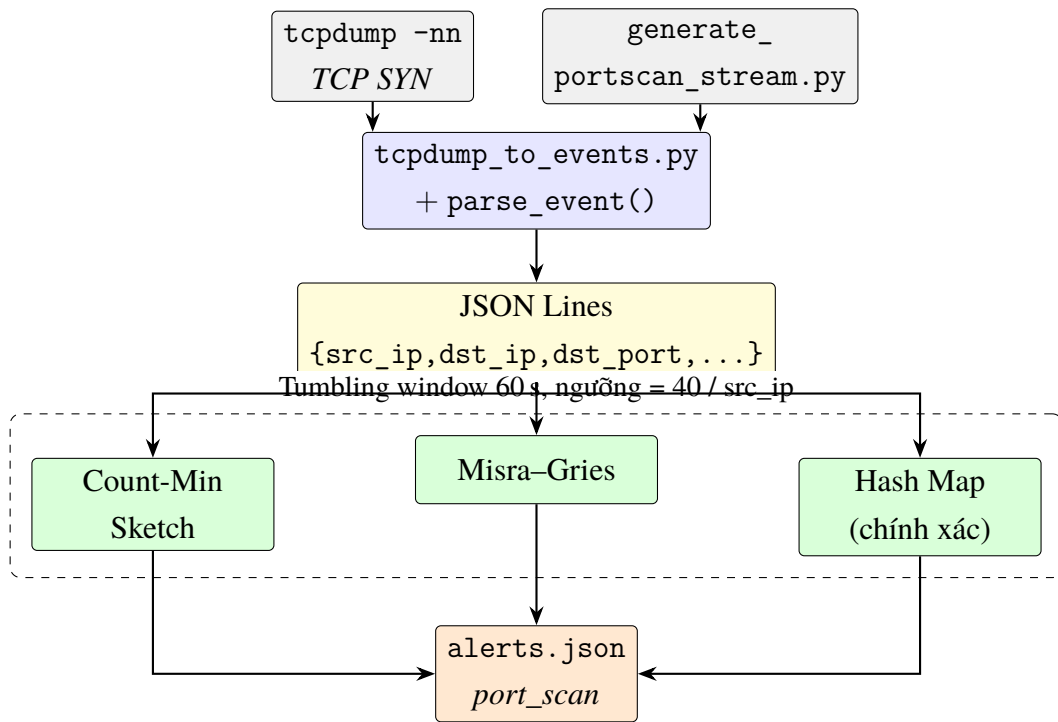
Bốn quyết định thiết kế ánh xạ Point Query thành detector port-scan cụ thể được tổng hợp trong Bảng 5.1.

Bảng 5.1. Tham số cấu hình của PortScanDetector.

Tham số	Giá trị	Lập luận
Khóa	src_ip	Bao phủ vertical/horizontal/coordinated scan
τ (ngưỡng)	40 kết nối / cửa sổ	Cao hơn lưu lượng nền điển hình ($\sim 10\text{--}20$) một cấp
T (cửa sổ)	60 giây tumbling	Đơn giản, không cần deque phụ trợ
CMS w	2048	$\varepsilon \approx e/w \approx 1.33 \times 10^{-3}$
CMS d	5	$\delta \approx (1/e)^d \approx 6.7 \times 10^{-3}$
MG k	64	Đủ giữ heavy hitters; sai số dưới m/k
Hash Map	—	Baseline chính xác (memory không đảm bảo)

5.1.2. Kiến trúc pipeline tinh giản

Pipeline được hiện thực hoá trong thư mục `tools/portscan-pipeline` của mã nguồn đính kèm. Hệ Zero-Trust Network Mapper đầy đủ (thuộc đề án tốt nghiệp) còn bao gồm các giai đoạn thu thập sự kiện qua eBPF, dịch vụ tổng hợp đa nút trên NATS JetStream, lưu trữ sự cố trong PostgreSQL và áp dụng NetworkPolicy đáp ứng tự động. Trong phạm vi báo cáo, ta cô lập đúng phần *lõi giải thuật* — ba khối `tcpcdump` $\rightarrow$ `parser` $\rightarrow$ `frequency estimator` $\rightarrow$ `alert` (Hình 5.1) — đủ để chứng minh tính khả thi và đo được chi phí của lớp ước lượng tần suất, không phụ thuộc vào hạ tầng phần cứng cụ thể nào.



Hình 5.1. Pipeline phát hiện port-scan tinh giản.

Hình 5.1 mô tả đường đi của sự kiện: nguồn dữ liệu (tcpdump hoặc generator tổng hợp) → parser (tcpdump_to_events.py kết hợp parse_event()) → hàng đợi JSON Lines → ba bộ ước lượng tần suất (CMS, Misra-Gries, Hash Map) chạy song song trong cùng tumbling window 60 s, ngưỡng 40 / src_ip → alerts.json. Bốn quyết định thiết kế chính trong pipeline:

- **Khoá là src_ip** thay vì cặp (src_ip, dst_port), để bao phủ cả ba dạng quét *vertical* (một IP, nhiều cổng), *horizontal* (một cổng, nhiều IP đích) và *coordinated* hỗn hợp. Cài đặt thực hiện qua hàm tiện ích record_u32(src_ip) của giao diện Estimator đã trình bày ở Chương 4.
- **Tumbling window 60 giây**: khi $t_j - t_{\text{start}} > T$ thì gọi reset() làm sạch toàn bộ cấu trúc và bắt đầu cửa sổ mới, giữ chi phí cập nhật ở $O(d)$ thuần tuý của thuật toán nền. Sliding window cần cấu trúc phụ và được liệt kê trong hướng phát triển.
- **Dedup mỗi cửa sổ**: khi tần suất ước lượng vượt ngưỡng, sự kiện đầu tiên kích hoạt cảnh báo và IP đó được thêm vào tập alerted_in_window. Các sự kiện sau cùng IP trong cùng cửa sổ không sinh thêm cảnh báo trùng.
- **Hai chế độ đầu vào**: chế độ *trực tiếp* dùng tcpdump -nn -l -i any với bộ lọc gói SYN; chế độ *tái lập* dùng script generate_portscan_stream.py để sinh luồng JSON Lines tổng hợp với hạt giống cố định. Cả hai kết thúc bằng cùng một khuôn dạng JSON một dòng/sự kiện do parse_event() (C++) đọc được.

Để kiểm chứng cài đặt, đầu vào là 200 sự kiện do `generate_portscan_stream.py` sinh với hạt giống 42: 100 sự kiện lưu lượng nền vi-dịch-vụ, 50 gói SYN từ attacker 10.0.5.99 chèn vào từ sự kiện thứ 100, rồi 50 sự kiện nền tiếp theo. Detector dừng tại các sự kiện 50, 100, 110, 120 và in trạng thái nội tại của estimator vào `output/portscan/trace_cms.txt`.

```

1  === Snapshot at event #50 (window 0, t=1700000000049000000 ns)
    ===
2  algorithm      : count_min_sketch
3  memory_usage   : 81940 bytes
4  total_events   : 50
5  alerted_in_w   : 0
6  cms_width      : 2048
7  cms_depth      : 5
8  watchlist (estimated frequency from current estimator):
9      10.0.5.99  attacker          est=0
10     10.0.1.10  frontend-1        est=10
11     10.0.1.11  frontend-2        est=10
12     10.0.2.10  backend-1         est=6
13     10.0.2.11  backend-2         est=4
14     10.0.3.10  postgres          est=0
15     10.0.3.11  redis             est=0
16     10.0.4.10  prometheus        est=10
17
18 === Snapshot at event #100 (window 0, t=1700000000099000000 ns)
    ===
19 algorithm      : count_min_sketch
20 memory_usage   : 81940 bytes
21 total_events   : 100
22 alerted_in_w   : 0
23 cms_width      : 2048
24 cms_depth      : 5
25 watchlist (estimated frequency from current estimator):
26     10.0.5.99  attacker          est=0
27     10.0.1.10  frontend-1        est=19
28     10.0.1.11  frontend-2        est=15
29     10.0.2.10  backend-1         est=12
30     10.0.2.11  backend-2         est=10
31     10.0.3.10  postgres          est=0
32     10.0.3.11  redis             est=0
33     10.0.4.10  prometheus        est=15
34
35 === Snapshot at event #110 (window 0, t=1700000000100900000 ns)
    ===
36 algorithm      : count_min_sketch

```

```

37  memory_usage   : 81940 bytes
38  total_events   : 110
39  alerted_in_w    : 0
40  cms_width       : 2048
41  cms_depth       : 5
42  watchlist (estimated frequency from current estimator):
43      10.0.5.99   attacker           est=10
44      10.0.1.10   frontend-1         est=19
45      10.0.1.11   frontend-2         est=15
46      10.0.2.10   backend-1          est=12
47      10.0.2.11   backend-2          est=10
48      10.0.3.10   postgres           est=0
49      10.0.3.11   redis               est=0
50      10.0.4.10   prometheus          est=15
51
52  === Snapshot at event #120 (window 0, t=1700000000101900000 ns)
53  ===
54  algorithm       : count_min_sketch
55  memory_usage    : 81940 bytes
56  total_events    : 120
57  alerted_in_w    : 0
58  cms_width       : 2048
59  cms_depth       : 5
60  watchlist (estimated frequency from current estimator):
61      10.0.5.99   attacker           est=20
62      10.0.1.10   frontend-1         est=19
63      10.0.1.11   frontend-2         est=15
64      10.0.2.10   backend-1          est=12
65      10.0.2.11   backend-2          est=10
66      10.0.3.10   postgres           est=0
67      10.0.3.11   redis               est=0
68      10.0.4.10   prometheus          est=15

```

Mã nguồn 5.2. Trích đoạn output/portscan/trace_cms.txt — trạng thái CMS tại bốn mốc sự kiện trên luồng IP tổng hợp.

Đọc theo thứ tự thời gian, trace cho thấy:

- **Sự kiện 50, 100:** chỉ có lưu lượng nền. Các nguồn vi-dịch-vụ (frontend-1, frontend-2, prometheus) đứng quanh mốc 10–19 kết nối; attacker chưa xuất hiện nên $est(10.0.5.99) = 0$.
- **Sự kiện 110, 120:** attacker bắt đầu quét, đếm tăng tuyến tính 10, 20 trong khi các nguồn nền giữ nguyên — đặc trưng kinh điển của port-scan: một IP nguồn tăng tốc nhanh, các IP khác không đổi.
- **Sự kiện 141** (sau khi đếm vượt ngưỡng): $est(10.0.5.99) = 41 > 40$, detector phát

cảnh báo, IP attacker được thêm vào tập `alerted_in_window` và các sự kiện tiếp theo cùng IP không sinh thêm cảnh báo.

Kết quả đối chiếu với `output/portscan/alerts.json`: cả ba thuật toán đều phát cảnh báo tại cùng `event_index = 141` với `freq = 41`.

5.2. Đánh giá thực nghiệm

5.2.1. Phương pháp và môi trường đánh giá

Hai kịch bản benchmark được sử dụng để đo định lượng cấu trúc dữ liệu trên hai loại đầu vào:

- **Synthetic uniform** qua `libdsa/benchmarks/bench_frequency.cpp` (Google Benchmark): sinh ngẫu nhiên 10^5 khoá `uint32_t` bằng `std::mt19937` với seed cố định 42, đo *record* và *estimate* riêng biệt.
- **Port-scan IP stream** qua `tools/portscan-pipeline`: 200 sự kiện đã mô tả ở Mục 5.1.3, đo memory đỉnh + thông lượng + true/false positive cho cả ba thuật toán.

Cấu hình máy đánh giá: CPU 16 nhân, L2 1280 KiB, RAM 16 GB, Linux Fedora 43 (kernel 6.19), build Debug — thông lượng Release kỳ vọng cao hơn khoảng 2–3 lần.

5.2.2. Thông lượng và bộ nhớ trên luồng tổng hợp

Bảng 5.2. Thông lượng *record/estimate* và bộ nhớ trên 10^5 khoá phân biệt.

Thuật toán	Chiều rộng w	Thông lượng (ops/s)	Bộ nhớ (KB)
CMS record ($d=5$)	512	$1,18 \times 10^7$	20,0
CMS record ($d=5$)	1024	$1,18 \times 10^7$	40,0
CMS record ($d=5$)	2048	$1,20 \times 10^7$	80,0
CMS record ($d=5$)	4096	$1,15 \times 10^7$	160,0
CMS estimate	2048	$8,87 \times 10^6$	—
Hash Map record	—	$5,38 \times 10^6$	3808,5
Hash Map estimate	—	$4,45 \times 10^6$	—

Nguồn: `output/bench/throughput.csv`.

Hai điểm đáng chú ý: (i) CMS *record* gần như không đổi khi w thay đổi — khẳng định độ phức tạp $O(d)$ độc lập với w ; (ii) CMS record nhanh hơn Hash Map record khoảng 2,2 lần dù phải băm $d = 5$ lần (Hash Map chỉ băm một lần) vì bộ nhớ tĩnh nằm gọn trong L2 cache trong khi Hash Map động chạm heap khi grow gây cache miss và phân mảnh. Bộ nhớ CMS với ($w = 2048, d = 5$) đo được đúng 80,0 KB, độc lập số phần tử đã chèn; Hash Map đo được 3 808,5 KB — gấp **47,6 lần** — trên cùng tập 100K khoá phân biệt; tỉ

lệ này chỉ tăng khi F_0 lớn hơn vì CMS không đổi.

5.2.3. Sai số CMS theo chiều rộng

Bảng 5.3. Sai số trung bình theo chiều rộng CMS ($d = 5$, $m = 10^5$).

w	$\varepsilon \approx e/w$	Cận εm	Sai số đo	Tỉ lệ đo/cận
256	$1,062 \times 10^{-2}$	1062	368,3	34,7%
512	$5,31 \times 10^{-3}$	531	179,4	33,8%
1024	$2,65 \times 10^{-3}$	265	86,2	32,5%
2048	$1,33 \times 10^{-3}$	133	40,8	30,7%
4096	$6,64 \times 10^{-4}$	66,4	18,8	28,3%

Nguồn: output/bench/error_rate.csv.

Sai số thực tế luôn thấp hơn đáng kể cận lý thuyết — khoảng 28–35% của cận εm . Hai nguyên nhân: (i) bất đẳng thức Markov dùng trong chứng minh là lỏng so với phân bố thực; (ii) thao tác *min* trên d hàng loại bỏ nhiều các hàng xấu. Xu hướng giảm tỉ lệ đo/cận khi w tăng phù hợp với trực giác: w lớn hơn $\rightarrow$ ít dụng độ hơn $\rightarrow$ min càng gần giá trị đúng.

5.2.4. So sánh ba thuật toán trên luồng IP port-scan

Bảng 5.4. So sánh CMS / Misra-Gries / Hash Map trên luồng port-scan tổng hợp.

Thuật toán	Sự kiện	Cảnh báo	TP	FP	Bộ nhớ (KB)	Thông lượng (sk/s)
Count-Min Sketch	200	1	1	0	80,02	229 899
Misra-Gries	200	1	1	0	0,42	147 496
Hash Map (chính xác)	200	1	1	0	0,40	180 788

Nguồn: output/portscan/comparison.csv. Cấu hình thử nghiệm: $n = 200$ sự kiện, attacker bắt đầu chen từ sự kiện thứ 100 với 50 gói SYN, ngưỡng $\tau = 40$ kết nối / src_ip, cửa sổ tumbling 60 s.

Ba kết luận đáng chú ý:

- **Cả ba phát hiện đúng một IP tấn công, không có cảnh báo giả.** Trên 11 IP phân biệt và 200 sự kiện, không có va chạm CMS đáng kể (xác suất $1/w = 1/2048$); MG có $k = 64$ lớn hơn số IP phân biệt nên không xảy ra giảm bộ đếm; HashMap đếm chính xác. Sự đồng thuận này khẳng định tính đúng đắn của cài đặt.
- **CMS dùng nhiều bộ nhớ hơn MG và HashMap trên luồng nhỏ.** Trên 11 IP phân biệt, HashMap và MG chỉ chiếm $\sim 0,4$ KB còn CMS chiếm cố định $w \times d \times 8 = 80$

KB *bất kể* có bao nhiêu phần tử phân biệt. Lợi thế của CMS chỉ xuất hiện khi F_0 vượt khoảng ~ 2000 IP phân biệt — tại đó HashMap vượt 80 KB. Khi đo trên $F_0 = 10^5$ ở Mục 5.2, HashMap chiếm 3 808 KB so với CMS 80 KB (gấp 47,6 lần); kết quả nhất quán giữa hai cách đo.

- **Thông lượng phụ thuộc cấu trúc dữ liệu chứ không phải kích thước.** CMS dẫn đầu vì 5 phép băm + 5 phép gán nằm gọn trong L2; HashMap chậm hơn vì bucket lookup gây cache miss khi grow; MG tốn thêm chi phí kiểm tra slot tồn tại + bước decrement-all khi map đầy.

Cảnh giới đặt ra: CMS không tự động là lựa chọn tốt — nó tốt khi luồng đủ lớn. Trong môi trường vận hành thực có F_0 lớn và đầu vào đối kháng, bộ nhớ *cố định* của CMS và tính *mergeable* (cộng ma trận giữa các nút) là các thuộc tính khiến CMS vẫn là lựa chọn duy nhất phù hợp cho lớp giám sát data-plane. Trên luồng nhỏ, 80 KB là chi phí cố định cần chấp nhận để có được đặc tính tiệm cận đó.

5.3. Đánh giá tổng thể giải pháp

Count-Min Sketch đáp ứng cả năm tiêu chí được đặt ra ở Chương 1:

1. Bộ nhớ cố định 80 KB — thoả yêu cầu bộ nhớ giới hạn và vừa vặn L2 cache.
2. Thao tác cập nhật là $O(d) = O(5)$ phép băm cố định — thoả yêu cầu thời gian cập nhật hằng số.
3. Đảm bảo (ε, δ) được chứng minh đầy đủ và tái hiện trong kiểm thử đơn vị.
4. Thông lượng đủ lớn để xử lý luồng sự kiện $> 10^6$ /giây, phù hợp ứng dụng giám sát mạng thực tế.
5. Thiên lệch *trên mức* đảm bảo không bỏ sót phần tử nặng — an toàn cho bài toán phát hiện bất thường.

Yếu điểm chính là không thể liệt kê các phần tử có tần suất cao (cần kết hợp với cấu trúc phụ như min-heap) và bản chất randomized (tuy kiểm soát được qua tham số δ). Các yếu điểm này được cân bằng bởi tính mergeable — mở đường cho kiến trúc phân tán đa nút trong báo cáo tốt nghiệp.

5.4. Kết luận và hướng phát triển

Báo cáo đã giải quyết trọn vẹn bài toán ước lượng tần suất điểm trên luồng dữ liệu theo chu trình “hình thức hoá → phân tích → thiết kế → cài đặt → ứng dụng → đánh giá”. Đóng góp cụ thể:

- **Lý thuyết:** trình bày chứng minh đầy đủ bốn bước của định lý Cormode–Muthukrishnan bằng tiếng Việt, kèm giải thích ý nghĩa của từng bước và mối liên hệ với cận dưới Alon–Matias–Szegedy.
- **Cài đặt:** thư viện libdsa modular với giao diện thuần ảo, Registry cấu hình, ba thuật

toán tần suất và bộ kiểm thử đơn vị 23 test case toàn bộ xanh.

- **Ứng dụng:** pipeline phát hiện port-scan tinh giản trong tools/portscan-pipeline chứng minh cấu trúc dữ liệu được phân tích là đủ để giải bài toán *ứng dụng phát hiện bất thường mạng* đã đặt ra trong tên đề tài.
- **Giảng dạy:** chương trình trace từng bước (Phụ lục B) minh họa trực quan cách CMS và Misra–Gries hoạt động, hữu ích cho việc giảng dạy môn cấu trúc dữ liệu.

Đề án tốt nghiệp môn Công nghệ phần mềm 2 sẽ tiếp tục các hướng sau, dựa trên nền tảng đã xây dựng:

- **K-Hop Reachability (BFS):** tính toán *blast radius* từ một nút bị xâm nhập, phục vụ khoanh vùng tác động sự cố.
- **Tarjan SCC:** phát hiện chu trình trong đồ thị mạng, hỗ trợ phát hiện lateral movement.
- **LPM Trie:** phân loại IP theo tập con mạng (subnet) để gán nhãn “internal/external” cho sự kiện eBPF.
- **Tích hợp đầy đủ pipeline đa nút:** thay collector tcpdump ở Mục 5.1.2 bằng eBPF probe kprobe/tcp_v4_connect để bắt sự kiện ở *kernel space*, bổ sung aggregator đa nút trên NATS JetStream với hợp nhất ma trận CMS, lưu sự cố vào PostgreSQL và phát NetworkPolicy cách ly tự động qua Kubernetes API server.
- **Sliding window CMS:** thay tumbling window 60 s bằng cửa sổ trượt với cấu trúc CMS phân đoạn (*segment-based*) để xử lý các đợt scan vắt qua biên cửa sổ.
- **Conservative Update:** kích hoạt biến thể CMS-CU mặc định cho lưu lượng Zipfian thực tế, kỳ vọng giảm sai số 30–70% theo [9].

TÀI LIỆU THAM KHẢO

- [1] Noga Alon, Yossi Matias, and Mario Szegedy. “The Space Complexity of Approximating the Frequency Moments”. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. Gödel Prize 2005. 1996, pp. 20–29. DOI: [10.1145/237814.237823](https://doi.org/10.1145/237814.237823).
- [2] Austin Appleby. *MurmurHash3*. <https://github.com/aappleby/smhasher>. Non-cryptographic hash function used in Count-Min Sketch implementation. 2011.
- [3] Davide Berardi, Saverio Giallorenzo, Jacopo Mauro, Andrea Melis, Fabrizio Montesi, and Marco Prandini. “Microservice security: a systematic literature review”. In: *PeerJ Computer Science* 8 (2022). Systematic literature review of 290 publications, e779. DOI: [10.7717/peerj-cs.779](https://doi.org/10.7717/peerj-cs.779).
- [4] Amit Chakrabarti. *Data Stream Algorithms*. Lecture notes. Lecture Notes, Dartmouth College. 2019.
- [5] Moses Charikar, Kevin Chen, and Martin Farach-Colton. “Finding Frequent Items in Data Streams”. In: *Theoretical Computer Science* 312.1 (2004), pp. 3–15. DOI: [10.1016/S0304-3975\(03\)00400-6](https://doi.org/10.1016/S0304-3975(03)00400-6).
- [6] Graham Cormode and Marios Hadjieleftheriou. “Finding Frequent Items in Data Streams”. In: *Proceedings of the VLDB Endowment* 1.2 (2008). Experimental comparison of frequency estimation algorithms, pp. 1530–1541. DOI: [10.14778/1454159.1454225](https://doi.org/10.14778/1454159.1454225).
- [7] Graham Cormode and S. Muthukrishnan. “An Improved Data Stream Summary: The Count-Min Sketch and its Applications”. In: *Journal of Algorithms* 55.1 (2005), pp. 58–75. DOI: [10.1016/j.jalgor.2003.12.001](https://doi.org/10.1016/j.jalgor.2003.12.001).
- [8] Scott A. Crosby and Dan S. Wallach. “Denial of Service via Algorithmic Complexity Attacks”. In: *Proceedings of the 12th USENIX Security Symposium*. Washington, D.C., USA: USENIX Association, 2003, pp. 29–44.
- [9] Cristian Estan and George Varghese. “New Directions in Traffic Measurement and Accounting: Focusing on the Elephants, Ignoring the Mice”. In: *ACM Transactions on Computer Systems* 21.3 (2003). Conservative update variant of Count-Min Sketch, pp. 270–313. DOI: [10.1145/859716.859719](https://doi.org/10.1145/859716.859719).
- [10] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Strategy Pattern (Chapter 5.9). Addison-Wesley, 1995. ISBN: 978-0-201-63361-0.

- [11] Sumit Ganguly. “Lower Bounds on Frequency Estimation of Data Streams”. In: *Computer Science – Theory and Applications (CSR 2012)*. Vol. 7353. Lecture Notes in Computer Science. Springer, 2012.
- [12] Nurman Rasyid Panusunan Hutasuhut, Mochamad Gani Amri, and Rizal Fathoni Aji. “Security Gap in Microservices: A Systematic Literature Review”. In: *International Journal of Advanced Computer Science and Applications (IJACSA)* 15.12 (2024). SLR of 487 papers (87 selected via snowball), focusing on post-2020 vulnerabilities. DOI: [10.14569/IJACSA.2024.0151218](https://doi.org/10.14569/IJACSA.2024.0151218).
- [13] J. Misra and David Gries. “Finding Repeated Elements”. In: *Science of Computer Programming* 2.2 (1982), pp. 143–152. DOI: [10.1016/0167-6423\(82\)90012-0](https://doi.org/10.1016/0167-6423(82)90012-0).
- [14] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005. ISBN: 978-0-521-83540-4.
- [15] Vern Paxson. “Bro: A System for Detecting Network Intruders in Real-Time”. In: *Computer Networks* 31.23–24 (1999), pp. 2435–2463. DOI: [10.1016/S1389-1286\(99\)00112-7](https://doi.org/10.1016/S1389-1286(99)00112-7).
- [16] Anelis Pereira-Vale, Eduardo B. Fernández, Raúl Monge, Hernán Astudillo, and Gastón Márquez. “Security in microservice-based systems: A Multivocal literature review”. In: *Computers & Security* 103 (2021). Multivocal review combining 36 academic + 34 grey-literature sources, p. 102200. DOI: [10.1016/j.cose.2021.102200](https://doi.org/10.1016/j.cose.2021.102200).
- [17] Red Hat, Inc. *The State of Kubernetes Security Report, 2024 Edition*. Tech. rep. Industry survey of 600 DevOps, engineering, and security professionals. Red Hat, 2024. URL: <https://www.redhat.com/en/engage/state-kubernetes-security-report-2024>.
- [18] Martin Roesch. “Snort – Lightweight Intrusion Detection for Networks”. In: *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*. Seattle, Washington, USA: USENIX Association, 1999, pp. 229–238.
- [19] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. *Zero Trust Architecture*. Tech. rep. SP 800-207. National Institute of Standards and Technology (NIST), 2020. DOI: [10.6028/NIST.SP.800-207](https://doi.org/10.6028/NIST.SP.800-207).
- [20] David Talbot and Miles Osborne. “Randomised Language Modelling for Statistical Machine Translation”. In: *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics (ACL)*. Prague, Czech Republic, 2007, pp. 512–519.

- [21] Wiz Research. *Kubernetes Security Report 2025*. Tech. rep. Industry telemetry on cloud-native attack patterns observed across managed Kubernetes environments. Wiz, 2025. URL: <https://www.wiz.io/reports/kubernetes-security-report-2025>.

PHỤ LỤC

CHƯƠNG A

MÃ NGUỒN LỖI: COUNT-MIN SKETCH, MISRA-GRIES, HASHMAP

Phụ lục này đính kèm mã nguồn các thuật toán cốt lõi trong thư viện `libdsa`. Các phần triển khai trực tiếp thuật toán được đưa vào; một số kiểm thử đơn vị và đo hiệu năng quan trọng được đính kèm để minh họa cách xác nhận tính đúng và đo hiệu năng. Các mã hỗ trợ khác (`CMakeLists.txt`, `logging`, tiện ích phụ trợ) được lược bỏ nhằm đơn giản hóa phần trình bày; các phần mã nguồn chi tiết có thể tham khảo trong kho mã nguồn dự án. Các file được nhúng trực tiếp qua `\lstinputlisting`, đảm bảo đồng bộ tuyệt đối với mã nguồn thực tế do đó các phần chú thích bằng `//comment` được viết bằng tiếng Anh trong mã nguồn sẽ được giữ nguyên, trong khi phần mô tả dưới đây tóm tắt bằng tiếng Việt để người đọc dễ theo dõi.

A.1. Tóm tắt cấu trúc mã nguồn

Bảng dưới đây liệt kê các file quan trọng và vai trò của chúng trong phụ lục.

Bảng A.1. Tóm tắt các file mã nguồn trong phụ lục

Thành phần	File	Vai trò
Giao diện chung	<code>libdsa/include/dsa/frequency/estimator.h</code>	Khai báo giao diện <code>record()</code> , <code>estimate()</code> , <code>reset()</code> .
CMS (Count-Min Sketch)	<code>libdsa/include/dsa/frequency/count_min_sketch.h</code>	Khai báo cấu trúc CMS và tham số w, d .
	<code>libdsa/src/frequency/count_min_sketch.cpp</code>	Hiện thực cập nhật, truy vấn và hàm băm.
Misra–Gries	<code>libdsa/include/dsa/frequency/heavy_hitters.h</code>	Khai báo thuật toán Misra–Gries.
	<code>libdsa/src/frequency/heavy_hitters.cpp</code>	Hiện thực ba nhánh cập nhật bộ đếm.
HashMap Exact	<code>libdsa/include/dsa/frequency/hash_map_exact.h</code>	Khai báo đếm chính xác theo bảng băm.
	<code>libdsa/src/frequency/hash_map_exact.cpp</code>	Hiện thực đếm chính xác bằng <code>unordered_map</code> .
Kiểm thử	<code>libdsa/tests/test_*.cpp</code>	Xác nhận đúng: ước lượng trên/dưới, biên sai số.
Benchmark	<code>libdsa/benchmarks/benchmark_frequency.cpp</code>	Đo thông lượng và sai số trung bình.

A.2. Giao diện Estimator

Giao diện chung cho ba thuật toán ước lượng tần suất. Con trỏ `void*` cho phép khóa là bất cứ kiểu nào có thể tuần tự hoá trong bộ nhớ.

```
1  #pragma once
2  #include <cstddef>
3  #include <cstdint>
4  #include <string>
5
6  namespace dsa::frequency {
7
8  // Abstract interface for stream frequency estimation.
9  // Implementations: CountMinSketch, HashMapExact
10 class Estimator {
11 public:
12     virtual ~Estimator() = default;
13
14     // Record an observation of a key.
15     virtual void record(const void* key, size_t key_len) =
16         0;
17
18     // Query estimated frequency of a key.
19     virtual uint64_t estimate(const void* key, size_t
20         key_len) const = 0;
21
22     // Reset all counters.
23     virtual void reset() = 0;
24
25     // Memory footprint in bytes.
26     virtual size_t memory_usage() const = 0;
27
28     // Human-readable name.
29     virtual std::string name() const = 0;
30
31     // Convenience overloads for uint32_t keys (e.g., IP
32     // addresses)
33     void record_u32(uint32_t key) { record(&key, sizeof(key)
34         ); }
35     uint64_t estimate_u32(uint32_t key) const { return
36         estimate(&key, sizeof(key)); }
```

```

33     // Convenience for (src_ip, dst_ip) pair
34     void record_pair(uint32_t src, uint32_t dst) {
35         uint64_t pair = (static_cast<uint64_t>(src) << 32)
36             | dst;
37         record(&pair, sizeof(pair));
38     }
39     uint64_t estimate_pair(uint32_t src, uint32_t dst)
40     const {
41         uint64_t pair = (static_cast<uint64_t>(src) << 32)
42             | dst;
43         return estimate(&pair, sizeof(pair));
44     }
45 };
46
47 } // namespace dsa::frequency

```

Mã nguồn A.1. libdsa/include/dsa/frequency/estimator.h

Gợi ý đọc mã.

- record(...) cập nhật tần suất khi quan sát một khóa mới.
- estimate(...) trả về ước lượng tần suất hiện tại.
- reset() xóa toàn bộ trạng thái; memory_usage() báo kích thước bộ nhớ.
- Các hàm tiện ích record_u32, estimate_u32, record_pair phục vụ khóa IP hoặc cặp (src, dst).

A.3. Count-Min Sketch

A.3.1. Header

```

1  #pragma once
2  #include "dsa/frequency/estimator.h"
3  #include <cstdint>
4  #include <vector>
5
6  namespace dsa::frequency {
7
8  class CountMinSketch : public Estimator {
9  public:
10     // width = number of columns (~ e/epsilon), depth =
11     number of rows (~ ln(1/delta))
12     CountMinSketch(uint32_t width = 2048, uint32_t depth =
13         5);

```

```

12
13     void record(const void* key, size_t key_len) override;
14     uint64_t estimate(const void* key, size_t key_len)
15         const override;
16     void reset() override;
17     size_t memory_usage() const override;
18     std::string name() const override;
19
20     uint32_t width() const { return width_; }
21     uint32_t depth() const { return depth_; }
22
23 private:
24     uint32_t hash(const void* key, size_t key_len, uint32_t
25         seed) const;
26
27     uint32_t width_;
28     uint32_t depth_;
29     std::vector<std::vector<uint64_t>> table_;
30     std::vector<uint32_t> seeds_;
31 };
32
33 } // namespace dsa::frequency

```

Mã nguồn A.2. libdsa/include/dsa/frequency/count_min_sketch.h

A.3.2. Cài đặt

Lỗi thuật toán: khởi tạo seed theo hệ số vàng, MurmurHash3 đầy đủ (trộn khối *block mixing*, xử lý đuôi *tail handling*, hoàn tất *finalization*), cập nhật (UPDATE) $O(d)$ và truy vấn (QUERY) lấy min $O(d)$.

```

1  #include "dsa/frequency/count_min_sketch.h"
2  #include <algorithm>
3  #include <cstring>
4  #include <limits>
5
6  namespace dsa::frequency {
7
8  CountMinSketch::CountMinSketch(uint32_t width, uint32_t
9      depth)
10      : width_(width), depth_(depth),
11        table_(depth, std::vector<uint64_t>(width, 0)) {
12      // Generate deterministic seeds for reproducibility

```

```

12     seeds_.reserve(depth);
13     for (uint32_t i = 0; i < depth; ++i) {
14         seeds_.push_back(0x9e3779b9 + i * 0x517cc1b7);
15     }
16 }
17
18 // MurmurHash3-inspired hash for good distribution
19 uint32_t CountMinSketch::hash(const void* key, size_t
    key_len, uint32_t seed) const {
20     const uint8_t* data = static_cast<const uint8_t*>(key);
21     uint32_t h = seed;
22
23     // Process 4-byte chunks
24     size_t nblocks = key_len / 4;
25     for (size_t i = 0; i < nblocks; ++i) {
26         uint32_t k;
27         std::memcpy(&k, data + i * 4, 4);
28
29         k *= 0xcc9e2d51;
30         k = (k << 15) | (k >> 17);
31         k *= 0x1b873593;
32
33         h ^= k;
34         h = (h << 13) | (h >> 19);
35         h = h * 5 + 0xe6546b64;
36     }
37
38     // Process remaining bytes
39     const uint8_t* tail = data + nblocks * 4;
40     uint32_t k1 = 0;
41     switch (key_len & 3) {
42         case 3: k1 ^= static_cast<uint32_t>(tail[2]) << 16;
43                 [[fallthrough]];
44         case 2: k1 ^= static_cast<uint32_t>(tail[1]) << 8;
45                 [[fallthrough]];
46         case 1: k1 ^= static_cast<uint32_t>(tail[0]);
47                 k1 *= 0xcc9e2d51;
48                 k1 = (k1 << 15) | (k1 >> 17);
49                 k1 *= 0x1b873593;
50                 h ^= k1;
51     }

```

```

50
51     // Finalization
52     h ^= static_cast<uint32_t>(key_len);
53     h ^= h >> 16;
54     h *= 0x85ebca6b;
55     h ^= h >> 13;
56     h *= 0xc2b2ae35;
57     h ^= h >> 16;
58
59     return h;
60 }
61
62 void CountMinSketch::record(const void* key, size_t key_len
    ) {
63     for (uint32_t i = 0; i < depth_; ++i) {
64         uint32_t col = hash(key, key_len, seeds_[i]) %
            width_;
65         table_[i][col]++;
66     }
67 }
68
69 uint64_t CountMinSketch::estimate(const void* key, size_t
    key_len) const {
70     uint64_t min_val = std::numeric_limits<uint64_t>::max()
        ;
71     for (uint32_t i = 0; i < depth_; ++i) {
72         uint32_t col = hash(key, key_len, seeds_[i]) %
            width_;
73         min_val = std::min(min_val, table_[i][col]);
74     }
75     return min_val;
76 }
77
78 void CountMinSketch::reset() {
79     for (auto& row : table_) {
80         std::fill(row.begin(), row.end(), 0);
81     }
82 }
83
84 size_t CountMinSketch::memory_usage() const {
85     return depth_ * width_ * sizeof(uint64_t) + seeds_.size

```

```

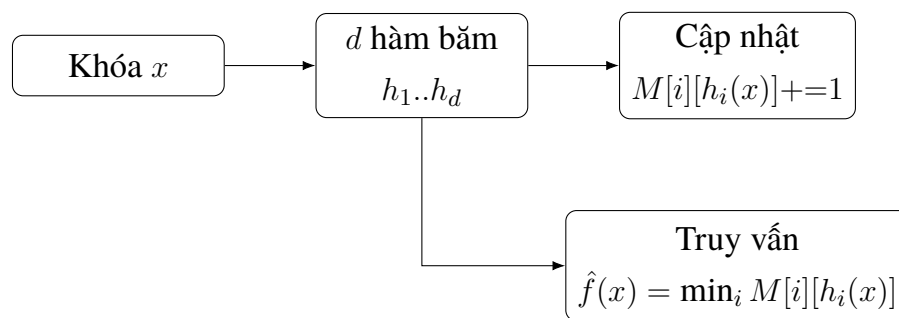
        () * sizeof(uint32_t);
86 }
87
88 std::string CountMinSketch::name() const {
89     return "count_min_sketch";
90 }
91
92 } // namespace dsa::frequency

```

Mã nguồn A.3. libdsa/src/frequency/count_min_sketch.cpp

Gợi ý đọc mã.

- `hash(...)` tạo ra chỉ số cột từ khóa và seed để mô phỏng d hàm băm độc lập.
- `table_` là ma trận $d \times w$ lưu bộ đếm; `seeds_` lưu các seed.
- `record()` tăng d bộ đếm; `estimate()` lấy giá trị nhỏ nhất trong d hàng.



Hình A.1. Luồng xử lý cập nhật và truy vấn trong Count-Min Sketch

Diễn giải. Sơ đồ cho thấy cùng một khóa đi qua d hàm băm, cập nhật d ô trong ma trận, và truy vấn lấy giá trị nhỏ nhất để tạo ước lượng an toàn (không nhỏ hơn tần suất thật).

A.4. Misra-Gries (Heavy Hitters)

A.4.1. Header

```

1 #pragma once
2 #include "dsa/frequency/estimator.h"
3 #include <cstdint>
4 #include <string>
5 #include <unordered_map>
6
7 namespace dsa::frequency {
8
9 // Misra-Gries (1982) bounded-counter algorithm.

```

```

10 // Maintains at most k-1 counters. For any key x:
11 //   f_hat(x) <= f(x)                                (
      underestimate)
12 //   f(x) - f_hat(x) <= m / k                        (bounded
      error)
13 // where m is the length of the stream seen so far.
14 class HeavyHitters : public Estimator {
15 public:
16     explicit HeavyHitters(uint32_t k = 64);
17
18     void record(const void* key, size_t key_len) override;
19     uint64_t estimate(const void* key, size_t key_len)
20         const override;
21     void reset() override;
22     size_t memory_usage() const override;
23     std::string name() const override;
24
25     uint32_t k() const { return k_; }
26     size_t active_counters() const { return counters_.size
27         (); }
28     uint64_t stream_length() const { return n_; }
29
30     // Exposed for tracing / reporting. Returns a copy of
31     // the current counter map.
32     std::unordered_map<std::string, uint64_t> snapshot()
33         const { return counters_; }
34
35 private:
36     uint32_t k_;
37     uint64_t n_;
38     std::unordered_map<std::string, uint64_t> counters_;
39 };
40
41 } // namespace dsa::frequency

```

Mã nguồn A.4. libdsa/include/dsa/frequency/heavy_hitters.h

A.4.2. Cài đặt

Ba nhánh logic của UPDATE (có trong bảng / bảng chưa đầy / bảng đầy) tương ứng trực tiếp với Giải thuật 3. Phương thức snapshot() phục vụ chương trình vết chạy (trace) in ra trạng thái bảng sau mỗi bước.

```

1  #include "dsa/frequency/heavy_hitters.h"
2  #include <stdexcept>
3
4  namespace dsa::frequency {
5
6  HeavyHitters::HeavyHitters(uint32_t k) : k_(k), n_(0) {
7      if (k_ < 2) {
8          throw std::invalid_argument("HeavyHitters: k must
9              be >= 2");
10     }
11     counters_.reserve(k_);
12 }
13
14 void HeavyHitters::record(const void* key, size_t key_len)
15 {
16     std::string s(static_cast<const char*>(key), key_len);
17     ++n_;
18
19     auto it = counters_.find(s);
20     if (it != counters_.end()) {
21         ++it->second;
22         return;
23     }
24     if (counters_.size() + 1 < k_) {
25         counters_.emplace(std::move(s), 1);
26         return;
27     }
28     // Decrement-all phase: subtract 1 from every counter;
29     drop those reaching 0.
30     for (auto cur = counters_.begin(); cur != counters_.end
31         ());) {
32         if (--cur->second == 0) {
33             cur = counters_.erase(cur);
34         } else {
35             ++cur;
36         }
37     }
38 }
39
40 uint64_t HeavyHitters::estimate(const void* key, size_t

```



```

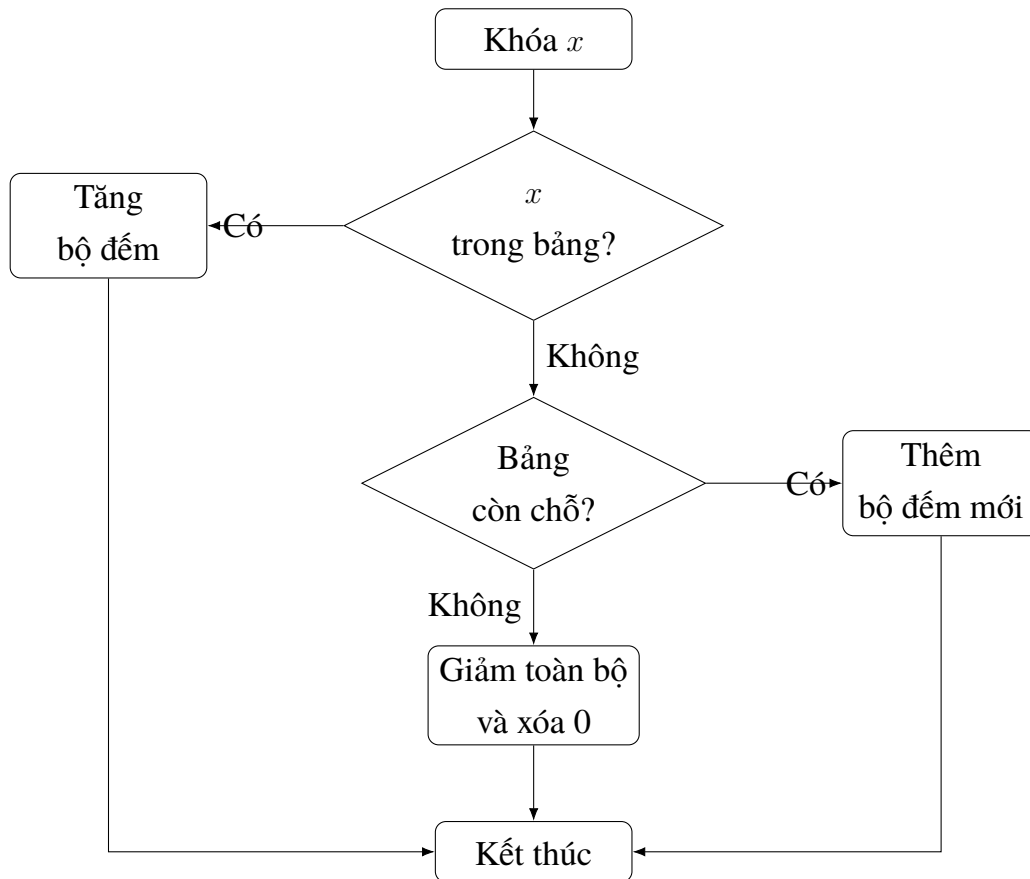
key_len) const {
37     std::string s(static_cast<const char*>(key), key_len);
38     auto it = counters_.find(s);
39     return (it != counters_.end()) ? it->second : 0;
40 }
41
42 void HeavyHitters::reset() {
43     counters_.clear();
44     n_ = 0;
45 }
46
47 size_t HeavyHitters::memory_usage() const {
48     size_t total = sizeof(*this);
49     for (const auto& [k, v] : counters_) {
50         total += k.capacity() + sizeof(v) + sizeof(void*) *
51             2;
52     }
53     return total;
54 }
55 std::string HeavyHitters::name() const {
56     return "heavy_hitters";
57 }
58
59 } // namespace dsa::frequency

```

Mã nguồn A.5. libdsa/src/frequency/heavy_hitters.cpp

Gợi ý đọc mã.

- counters_ là bảng đếm tối đa $k - 1$ phần tử.
- record() rẽ ba nhánh: tăng, thêm mới, hoặc giảm toàn bộ khi bảng đầy.
- snapshot() trả về bản sao bảng đếm để ghi trace.



Hình A.2. Lưu đồ cập nhật của Misra–Gries

Diễn giải. Lưu đồ mô tả cơ chế giới hạn bộ nhớ: khi bảng đầy, thuật toán giảm toàn bộ để loại dần các phần tử tần suất thấp.

A.5. Kiểm thử đơn vị Misra–Gries

Bộ kiểm thử kiểm tra các tính chất: không ước lượng vượt, phần tử nặng luôn tồn tại khi $f(x) > m/k$, và sai số bị chặn bởi m/k .

```

1  #include "dsa/frequency/heavy_hitters.h"
2  #include <gtest/gtest.h>
3  #include <string>
4  #include <vector>
5
6  using namespace dsa::frequency;
7
8  namespace {
9
10 void record_str(HeavyHitters& hh, const std::string& s) {
11     hh.record(s.data(), s.size());
12 }
13 uint64_t est_str(const HeavyHitters& hh, const std::string&
  
```

```

    s) {
14         return hh.estimate(s.data(), s.size());
15     }
16
17 } // namespace
18
19 TEST(HeavyHitters, SmallKThrows) {
20     EXPECT_THROW(HeavyHitters(1), std::invalid_argument);
21 }
22
23 TEST(HeavyHitters, Name) {
24     HeavyHitters hh(4);
25     EXPECT_EQ(hh.name(), "heavy_hitters");
26 }
27
28 TEST(HeavyHitters, FitsInCountersExact) {
29     // k=4 means up to 3 active counters; 3 distinct keys
    fit exactly.
30     HeavyHitters hh(4);
31     for (int i = 0; i < 5; ++i) record_str(hh, "a");
32     for (int i = 0; i < 3; ++i) record_str(hh, "b");
33     record_str(hh, "c");
34
35     EXPECT_EQ(est_str(hh, "a"), 5u);
36     EXPECT_EQ(est_str(hh, "b"), 3u);
37     EXPECT_EQ(est_str(hh, "c"), 1u);
38     EXPECT_EQ(hh.stream_length(), 9u);
39 }
40
41 TEST(HeavyHitters, DecrementsAllOnOverflow) {
42     // k=3 means up to 2 active counters. Classic worked
    example.
43     // Stream: a, b, c, a, d, a, b, e
44     HeavyHitters hh(3);
45     for (char c : std::string("abcadabe")) {
46         std::string s(1, c);
47         record_str(hh, s);
48     }
49     EXPECT_EQ(hh.stream_length(), 8u);
50
51     // Dominant 'a' survives; others should be <= exact.

```

```

52     EXPECT_LE(est_str(hh, "a"), 3u);
53     EXPECT_LE(est_str(hh, "b"), 2u);
54     EXPECT_LE(est_str(hh, "c"), 1u);
55     EXPECT_LE(est_str(hh, "d"), 1u);
56     EXPECT_LE(est_str(hh, "e"), 1u);
57
58     // Heavy hitter (a, with exact freq 3 > m/k = 8/3) must
59     be present.
60     EXPECT_GT(est_str(hh, "a"), 0u);
61 }
62
63 TEST(HeavyHitters, HeavyHitterGuarantee) {
64     // Any key with f(x) > m/k must survive with f_hat(x) >
65     0.
66     HeavyHitters hh(10);
67     const int heavy = 500;
68     const int m = 900;
69     for (int i = 0; i < heavy; ++i) record_str(hh, "HEAVY")
70         ;
71     for (int i = 0; i < m - heavy; ++i) record_str(hh, "x"
72         + std::to_string(i));
73     // heavy / m = 500/900 > 1/10 => guaranteed survive.
74     EXPECT_GT(est_str(hh, "HEAVY"), 0u);
75 }
76
77 TEST(HeavyHitters, UnderestimateProperty) {
78     // Misra-Gries never overestimates: f_hat(x) <= f(x)
79     for all x.
80     HeavyHitters hh(5);
81     std::vector<std::string> stream = {
82         "a", "b", "a", "c", "a", "d", "b", "a", "e", "f", "a", "g", "b",
83         "h", "i", "a",
84     };
85     std::unordered_map<std::string, uint64_t> exact;
86     for (const auto& s : stream) {
87         record_str(hh, s);
88         ++exact[s];
89     }
90     for (const auto& [k, f] : exact) {
91         EXPECT_LE(est_str(hh, k), f) << "key=" << k;
92     }

```

```

87 }
88
89 TEST(HeavyHitters, ErrorBoundM0verK) {
90     // f(x) - f_hat(x) <= m / k for every x.
91     const uint32_t k = 8;
92     HeavyHitters hh(k);
93     std::vector<std::string> stream;
94     for (int i = 0; i < 200; ++i) stream.emplace_back("hot"
95 );
96     for (int i = 0; i < 100; ++i) stream.emplace_back("warm
97 " + std::to_string(i % 3));
98     for (int i = 0; i < 50; ++i) stream.emplace_back("cold"
99 + std::to_string(i));
100     std::unordered_map<std::string, uint64_t> exact;
101     for (const auto& s : stream) {
102         record_str(hh, s);
103         ++exact[s];
104     }
105     const uint64_t bound = stream.size() / k;
106     for (const auto& [key, f] : exact) {
107         uint64_t est = est_str(hh, key);
108         EXPECT_LE(f - est, bound) << "key=" << key << " f="
109 << f << " est=" << est;
110     }
111 }
112
113 TEST(HeavyHitters, Reset) {
114     HeavyHitters hh(4);
115     record_str(hh, "a");
116     record_str(hh, "b");
117     hh.reset();
118     EXPECT_EQ(hh.stream_length(), 0u);
119     EXPECT_EQ(hh.active_counters(), 0u);
120     EXPECT_EQ(est_str(hh, "a"), 0u);
121 }
122
123 TEST(HeavyHitters, SnapshotReflectsState) {
124     HeavyHitters hh(4);
125     record_str(hh, "x");
126     record_str(hh, "x");
127     record_str(hh, "y");

```

```

124     auto snap = hh.snapshot();
125     EXPECT_EQ(snap["x"], 2u);
126     EXPECT_EQ(snap["y"], 1u);
127 }
128
129 TEST(HeavyHitters, MemoryUsageGrowsWithKeys) {
130     HeavyHitters hh(64);
131     const size_t empty_mem = hh.memory_usage();
132     for (int i = 0; i < 10; ++i) {
133         std::string s = "item_" + std::to_string(i);
134         record_str(hh, s);
135     }
136     EXPECT_GT(hh.memory_usage(), empty_mem);
137 }

```

Mã nguồn A.6. libdsa/tests/test_heavy_hitters.cpp

A.6. Hash Map Exact (đối chứng)

A.6.1. Header

```

1  #pragma once
2  #include "dsa/frequency/estimator.h"
3  #include <string>
4  #include <unordered_map>
5  #include <vector>
6
7  namespace dsa::frequency {
8
9  // Exact frequency counter using hash map. Baseline for
10 // benchmarks.
11 class HashMapExact : public Estimator {
12 public:
13     void record(const void* key, size_t key_len) override;
14     uint64_t estimate(const void* key, size_t key_len)
15         const override;
16     void reset() override;
17     size_t memory_usage() const override;
18     std::string name() const override;
19 private:
20     std::unordered_map<std::string, uint64_t> counts_;

```

```

20 };
21
22 } // namespace dsa::frequency

```

Mã nguồn A.7. libdsa/include/dsa/frequency/hash_map_exact.h

A.6.2. Cài đặt

Cài đặt đối chứng để so sánh: `std::unordered_map` với khóa chuỗi byte. Bộ nhớ được ước lượng bao gồm cả chi phí phụ (overhead) của bucket.

```

1  #include "dsa/frequency/hash_map_exact.h"
2
3  namespace dsa::frequency {
4
5  void HashMapExact::record(const void* key, size_t key_len)
6  {
7      std::string k(static_cast<const char*>(key), key_len);
8      counts_[k]++;
9  }
10
11 uint64_t HashMapExact::estimate(const void* key, size_t
12     key_len) const {
13     std::string k(static_cast<const char*>(key), key_len);
14     auto it = counts_.find(k);
15     return (it != counts_.end()) ? it->second : 0;
16 }
17
18 void HashMapExact::reset() {
19     counts_.clear();
20 }
21
22 size_t HashMapExact::memory_usage() const {
23     size_t total = sizeof(counts_);
24     for (const auto& [k, v] : counts_) {
25         total += k.capacity() + sizeof(v) + sizeof(void*) *
26             2; // key + value + bucket overhead
27     }
28     return total;
29 }
30
31 std::string HashMapExact::name() const {
32     return "hash_map_exact";
33 }

```

```

30 }
31
32 } // namespace dsa::frequency

```

Mã nguồn A.8. libdsa/src/frequency/hash_map_exact.cpp

Gợi ý đọc mã.

- counts_ lưu tần suất chính xác theo khóa byte.
- record() tăng trực tiếp bộ đếm; estimate() đọc đúng giá trị hiện có.

A.7. Kiểm thử đơn vị HashMap Exact

Kiểm tra các hành vi cơ bản: đếm đúng, khóa chưa có trả về 0, và reset() làm sạch trạng thái.

```

1  #include "dsa/frequency/hash_map_exact.h"
2  #include <gtest/gtest.h>
3
4  using namespace dsa::frequency;
5
6  TEST(HashMapExact, ExactCounts) {
7      HashMapExact hm;
8      uint32_t key = 42;
9
10     for (int i = 0; i < 100; ++i) {
11         hm.record(&key, sizeof(key));
12     }
13
14     EXPECT_EQ(hm.estimate(&key, sizeof(key)), 100u);
15 }
16
17 TEST(HashMapExact, MissingKey) {
18     HashMapExact hm;
19     uint32_t key = 42;
20     uint32_t missing = 99;
21
22     hm.record(&key, sizeof(key));
23     EXPECT_EQ(hm.estimate(&missing, sizeof(missing)), 0u);
24 }
25
26 TEST(HashMapExact, Reset) {
27     HashMapExact hm;

```



```

28     uint32_t key = 1;
29     hm.record(&key, sizeof(key));
30     hm.reset();
31     EXPECT_EQ(hm.estimate(&key, sizeof(key)), 0u);
32 }
33
34 TEST(HashMapExact, Name) {
35     HashMapExact hm;
36     EXPECT_EQ(hm.name(), "hash_map_exact");
37 }

```

Mã nguồn A.9. libdsa/tests/test_hash_map_exact.cpp

A.8. Kiểm thử đơn vị Count-Min Sketch

Bộ kiểm thử xác nhận ba tính chất cốt lõi từ Định lý 2.8: ước lượng thừa trên mọi khóa đã chèn (OverestimateProperty), ước lượng không vượt quá biên $\varepsilon \cdot m$ với xác suất cao (AccuracyBound), và sai số cho khóa chưa chèn bị chặn bởi đựng độ băm (NeverInsertedKeyCanOvercount).

```

1  #include "dsa/frequency/count_min_sketch.h"
2  #include <gtest/gtest.h>
3  #include <cstdint>
4
5  using namespace dsa::frequency;
6
7  TEST(CountMinSketch, BasicRecordAndEstimate) {
8      CountMinSketch cms(1024, 5);
9      uint32_t key = 42;
10
11      // Record 100 times
12      for (int i = 0; i < 100; ++i) {
13          cms.record(&key, sizeof(key));
14      }
15
16      // Estimate should be >= 100 (overestimate property)
17      EXPECT_GE(cms.estimate(&key, sizeof(key)), 100u);
18  }
19
20  TEST(CountMinSketch, OverestimateProperty) {
21      CountMinSketch cms(2048, 5);
22

```

```

23     // Insert many distinct keys
24     for (uint32_t i = 0; i < 10000; ++i) {
25         cms.record(&i, sizeof(i));
26     }
27
28     // Each key inserted once -> estimate >= 1
29     for (uint32_t i = 0; i < 100; ++i) {
30         EXPECT_GE(cms.estimate(&i, sizeof(i)), 1u);
31     }
32 }
33
34 TEST(CountMinSketch, NeverInsertedKeyCanOvercount) {
35     CountMinSketch cms(1024, 5);
36
37     for (uint32_t i = 0; i < 1000; ++i) {
38         cms.record(&i, sizeof(i));
39     }
40
41     // Key 99999 was never inserted - estimate may be > 0
42     due to hash collisions
43     uint32_t missing = 99999;
44     uint64_t est = cms.estimate(&missing, sizeof(missing));
45     // But should be small relative to total insertions
46     EXPECT_LE(est, 100u); // with w=1024, d=5, should be
47     very small
48 }
49
50 TEST(CountMinSketch, AccuracyBound) {
51     // epsilon = e/w ~= 2.718/2048 ~= 0.00133
52     // After m insertions, error <= epsilon * m with high
53     probability
54     CountMinSketch cms(2048, 5);
55     const uint32_t m = 100000;
56
57     // Insert key 0 exactly 500 times
58     uint32_t target = 0;
59     for (uint32_t i = 0; i < 500; ++i) {
60         cms.record(&target, sizeof(target));
61     }
62
63     // Insert m-500 other keys

```

```

61     for (uint32_t i = 1; i < m - 500 + 1; ++i) {
62         cms.record(&i, sizeof(i));
63     }
64
65     uint64_t est = cms.estimate(&target, sizeof(target));
66     EXPECT_GE(est, 500u); // never underestimates
67
68     // Error bound: epsilon * m ~= 0.00133 * 100000 ~= 133
69     // So estimate should be <= 500 + 133 = 633 with high
       probability
70     EXPECT_LE(est, 700u); // generous bound
71 }
72
73 TEST(CountMinSketch, Reset) {
74     CountMinSketch cms(512, 3);
75     uint32_t key = 1;
76
77     cms.record(&key, sizeof(key));
78     EXPECT_GE(cms.estimate(&key, sizeof(key)), 1u);
79
80     cms.reset();
81     EXPECT_EQ(cms.estimate(&key, sizeof(key)), 0u);
82 }
83
84 TEST(CountMinSketch, MemoryUsage) {
85     CountMinSketch cms(2048, 5);
86     // 2048 * 5 * 8 bytes (uint64_t) + seeds
87     EXPECT_GE(cms.memory_usage(), 2048u * 5 * 8);
88 }
89
90 TEST(CountMinSketch, ConvenienceMethods) {
91     CountMinSketch cms(1024, 5);
92
93     cms.record_u32(0x0A010203); // 10.1.2.3
94     cms.record_u32(0x0A010203);
95     cms.record_u32(0x0A010203);
96
97     EXPECT_GE(cms.estimate_u32(0x0A010203), 3u);
98
99     cms.record_pair(0x0A010203, 0x0A020304);
100    EXPECT_GE(cms.estimate_pair(0x0A010203, 0x0A020304), 1u

```

```

    );
101 }
102
103 TEST(CountMinSketch, Name) {
104     CountMinSketch cms;
105     EXPECT_EQ(cms.name(), "count_min_sketch");
106 }

```

Mã nguồn A.10. libdsa/tests/test_count_min_sketch.cpp

A.9. Bảng đối chiếu hành vi chính

Bảng A.2. Hành vi chính của ba cấu trúc tần suất

Cấu trúc	record(...)	estimate(...)
CMS	Băm khóa qua d hàm, tăng d ô tương ứng.	Trả về min; không nhỏ hơn tần suất thật.
Misra–Gries	Tăng / thêm mới / giảm toàn bộ tùy trạng thái băm.	Trả về ước lượng không vượt quá tần suất thật.
HashMap Exact	Tăng bộ đếm chính xác trong bảng băm.	Trả về tần suất chính xác của khóa.

A.10. Benchmark tần suất

Chương trình đo hiệu năng (benchmark) cho thông lượng và độ chính xác. Dữ liệu được sinh từ `std::mt19937` với seed cố định 42 để đảm bảo tái lập.

```

1  #include "dsa/frequency/count_min_sketch.h"
2  #include "dsa/frequency/hash_map_exact.h"
3  #include <benchmark/benchmark.h>
4  #include <cstdint>
5  #include <random>
6
7  using namespace dsa::frequency;
8
9  // Generate random uint32 keys
10 static std::vector<uint32_t> generate_keys(size_t count,
      uint32_t seed = 42) {
11     std::mt19937 rng(seed);
12     std::vector<uint32_t> keys(count);
13     for (auto& k : keys) k = rng();

```

```

14     return keys;
15 }
16
17 // --- Count-Min Sketch Benchmarks ---
18
19 static void BM_CMS_Record(benchmark::State& state) {
20     CountMinSketch cms(state.range(0), 5);
21     auto keys = generate_keys(100000);
22     size_t i = 0;
23
24     for (auto _ : state) {
25         cms.record(&keys[i % keys.size()], sizeof(uint32_t)
26             );
27         i++;
28     }
29     state.SetItemsProcessed(state.iterations());
30     state.counters["memory_kb"] = cms.memory_usage() /
31         1024.0;
32 }
33 BENCHMARK(BM_CMS_Record)->Arg(512)->Arg(1024)->Arg(2048)->
34     Arg(4096);
35
36 static void BM_CMS_Estimate(benchmark::State& state) {
37     CountMinSketch cms(2048, 5);
38     auto keys = generate_keys(100000);
39     for (auto& k : keys) cms.record(&k, sizeof(k));
40
41     size_t i = 0;
42     for (auto _ : state) {
43         benchmark::DoNotOptimize(cms.estimate(&keys[i %
44             keys.size()], sizeof(uint32_t)));
45         i++;
46     }
47     state.SetItemsProcessed(state.iterations());
48 }
49 BENCHMARK(BM_CMS_Estimate);
50
51 // --- Hash Map Exact Benchmarks ---
52
53 static void BM_HashMap_Record(benchmark::State& state) {
54     HashMapExact hm;

```

```

51     auto keys = generate_keys(100000);
52     size_t i = 0;
53
54     for (auto _ : state) {
55         hm.record(&keys[i % keys.size()], sizeof(uint32_t))
56         ;
57         i++;
58     }
59     state.SetItemsProcessed(state.iterations());
60     state.counters["memory_kb"] = hm.memory_usage() /
61     1024.0;
62 }
63 BENCHMARK(BM_HashMap_Record);
64
65 static void BM_HashMap_Estimate(benchmark::State& state) {
66     HashMapExact hm;
67     auto keys = generate_keys(100000);
68     for (auto& k : keys) hm.record(&k, sizeof(k));
69
70     size_t i = 0;
71     for (auto _ : state) {
72         benchmark::DoNotOptimize(hm.estimate(&keys[i % keys
73         .size()], sizeof(uint32_t)));
74         i++;
75     }
76     state.SetItemsProcessed(state.iterations());
77 }
78 BENCHMARK(BM_HashMap_Estimate);
79
80 // --- Accuracy comparison (not timing, but error
81 // measurement) ---
82
83 static void BM_CMS_ErrorRate(benchmark::State& state) {
84     uint32_t width = state.range(0);
85     CountMinSketch cms(width, 5);
86     HashMapExact exact;
87
88     auto keys = generate_keys(100000);
89     for (auto& k : keys) {
90         cms.record(&k, sizeof(k));
91         exact.record(&k, sizeof(k));

```

```

88     }
89
90     double total_error = 0;
91     size_t count = 0;
92     auto query_keys = generate_keys(1000, 99);
93
94     for (auto _ : state) {
95         for (auto& k : query_keys) {
96             int64_t cms_est = cms.estimate(&k, sizeof(k));
97             int64_t exact_val = exact.estimate(&k, sizeof(k));
98             total_error += (cms_est - exact_val);
99             count++;
100         }
101     }
102     state.counters["avg_overcount"] = total_error / count;
103     state.counters["width"] = width;
104 }
105 BENCHMARK(BM_CMS_ErrorRate)->Arg(256)->Arg(512)->Arg(1024)
    ->Arg(2048)->Arg(4096);

```

Mã nguồn A.11. libdsa/benchmarks/bench_frequency.cpp

CHƯƠNG B

KẾT QUẢ CHẠY VÀ DỮ LIỆU ĐẦU RA

Phụ lục này tổng hợp toàn bộ output chính của stage 1, gồm: tools/trace_algorithms, ứng dụng phát hiện port-scan, và dữ liệu đo hiệu năng/trực quan hóa dùng trong đánh giá. Mục đích là minh họa *bằng dữ liệu thật* các tính chất lý thuyết đã chứng minh trong Chương 3:

- Count-Min Sketch *ước lượng vượt* ($\hat{f}(x) \geq f(x)$).
- Misra–Gries *ước lượng thiếu* ($\hat{f}(x) \leq f(x)$) với sai số bị chặn bởi m/k .
- Hai giải thuật *kẹp* tần suất thật giữa hai giá trị.

Dữ liệu đầu vào. Luồng $S = \langle a, b, a, c, a, b, d, a \rangle$, độ dài $m = 8$. Tần suất chính xác: $f(a) = 4, f(b) = 2, f(c) = 1, f(d) = 1, f(e) = 0$.

Tham số. Count-Min Sketch với $w = 4, d = 2$; Misra–Gries với $k = 3$ (tối đa $k - 1 = 2$ bộ đếm). Kích thước nhỏ được chọn có chủ đích để toàn bộ ma trận và bảng đếm in được trực tiếp vào báo cáo.

Ghi chú về nguồn dữ liệu. Các listing trong phụ lục này được lấy trực tiếp từ output sinh ra khi chạy thực nghiệm trên máy, và được ghi lại dưới thư mục output/ để đảm bảo tái lập.

B.1. Menu và chế độ chạy

Menu của chương trình trace_algorithms khi chạy không tham số:

```
1 usage: build/bin/trace_algorithms {cms|misra|compare}
2   cms      : Count-Min Sketch step-by-step trace
3   misra    : Misra-Gries step-by-step trace
4   compare  : run all three (exact, CMS, MG) side by side
```

Mã nguồn B.1. Menu trace_algorithms

(output/appendix/trace_algorithms_help.txt)

B.2. Count-Min Sketch

Mỗi bước gồm: chỉ số cột $h_r(x) \bmod w$ trên từng hàng, phép += 1 tương ứng và toàn bộ ma trận sau khi cập nhật. Phần QUERIES liệt kê tất cả truy vấn cùng với tập cell được lấy min.

```
1 =====
2   COUNT-MIN SKETCH -   step-by-step trace
3   =====
4   Parameters : width w = 4 , depth d = 2
5   Guarantee  : f_hat(x) >= f(x)   (one-sided error, overestimate)
6   Bound      : P[ f_hat(x) - f(x) > eps * m ] <= delta
7               with eps = e/w ~ 0.680
8               and delta = e^(-d) ~ 0.1353
```



```

9
10 Stream S = [ a, b, a, c, a, b, d, a ]
11 Length m = 8
12
13 Step 1: INSERT "a"
14     h0("a") mod 4 = 0    -> table[0][0] += 1
15     h1("a") mod 4 = 0    -> table[1][0] += 1
16     Matrix after step 1:
17     Row 0: [ 1 0 0 0 ]
18     Row 1: [ 1 0 0 0 ]
19
20 Step 2: INSERT "b"
21     h0("b") mod 4 = 1    -> table[0][1] += 1
22     h1("b") mod 4 = 0    -> table[1][0] += 1
23     Matrix after step 2:
24     Row 0: [ 1 1 0 0 ]
25     Row 1: [ 2 0 0 0 ]
26
27 Step 3: INSERT "a"
28     h0("a") mod 4 = 0    -> table[0][0] += 1
29     h1("a") mod 4 = 0    -> table[1][0] += 1
30     Matrix after step 3:
31     Row 0: [ 2 1 0 0 ]
32     Row 1: [ 3 0 0 0 ]
33
34 Step 4: INSERT "c"
35     h0("c") mod 4 = 0    -> table[0][0] += 1
36     h1("c") mod 4 = 3    -> table[1][3] += 1
37     Matrix after step 4:
38     Row 0: [ 3 1 0 0 ]
39     Row 1: [ 3 0 0 1 ]
40
41 Step 5: INSERT "a"
42     h0("a") mod 4 = 0    -> table[0][0] += 1
43     h1("a") mod 4 = 0    -> table[1][0] += 1
44     Matrix after step 5:
45     Row 0: [ 4 1 0 0 ]
46     Row 1: [ 4 0 0 1 ]
47
48 Step 6: INSERT "b"
49     h0("b") mod 4 = 1    -> table[0][1] += 1
50     h1("b") mod 4 = 0    -> table[1][0] += 1
51     Matrix after step 6:
52     Row 0: [ 4 2 0 0 ]
53     Row 1: [ 5 0 0 1 ]
54
55 Step 7: INSERT "d"
56     h0("d") mod 4 = 0    -> table[0][0] += 1
57     h1("d") mod 4 = 0    -> table[1][0] += 1
58     Matrix after step 7:
59     Row 0: [ 5 2 0 0 ]
60     Row 1: [ 6 0 0 1 ]
61
62 Step 8: INSERT "a"
63     h0("a") mod 4 = 0    -> table[0][0] += 1
64     h1("a") mod 4 = 0    -> table[1][0] += 1

```

```

65 Matrix after step 8:
66 Row 0: [ 6 2 0 0 ]
67 Row 1: [ 7 0 0 1 ]
68
69 --- QUERIES ---
70 key      f(x)      f_hat(x)  cells(min)
71 "a"       4         6   {(0,0),(1,0)}
72 "b"       2         2   {(0,1),(1,0)}
73 "c"       1         1   {(0,0),(1,3)}
74 "d"       1         6   {(0,0),(1,0)}
75 "e"       0         0   {(0,2),(1,2)}
76
77 Observation: for every key, f_hat(x) >= f(x) - one-sided overestimate.

```

Mã nguồn B.2. Count-Min Sketch — trace đầy đủ (output/appendix/trace_algorithms_cms.txt)

Nhận xét. Ở bước 8, khóa d có $f(d) = 1$ nhưng $\hat{f}(d) = 6$. Lý do: hai cell của d là (0,0) và (1,0) *trùng hoàn toàn* với hai cell của khóa nóng a. Đây là minh họa cụ thể cho hiện tượng va chạm hash làm giảm độ chính xác khi w nhỏ. Khi w tăng theo công thức $w = \lceil e/\varepsilon \rceil$, xác suất xảy ra tình huống tương tự giảm tuyến tính theo ε .

B.3. Misra–Gries

Mỗi bước chỉ rõ một trong ba nhánh: (1) khóa đã có trong T nên tăng bộ đếm, (2) khóa mới và $|T| < k - 1$ nên thêm vào, (3) khóa mới và $|T| = k - 1$ nên *giảm toàn bộ* rồi loại các bộ đếm về 0.

```

1  =====
2  MISRA-GRIES (Heavy Hitters) -   step-by-step trace
3  =====
4  Parameter : k = 3 (at most 2 active counters)
5  Guarantees: f_hat(x) <= f(x)      (underestimate)
6              f(x) - f_hat(x) <= m / k
7
8  Stream S = [ a, b, a, c, a, b, d, a ]
9  Length m = 8
10
11 Step 1: INSERT "a"
12   "a" not in T, |T| = 0 < k-1 = 2   -> add with count 1
13   T = { a:1 }
14
15 Step 2: INSERT "b"
16   "b" not in T, |T| = 1 < k-1 = 2   -> add with count 1
17   T = { a:1, b:1 }
18
19 Step 3: INSERT "a"
20   "a" in T   -> increment counter
21   T = { a:2, b:1 }
22
23 Step 4: INSERT "c"
24   "c" not in T and |T| = 2 = k-1 = 2   -> DECREMENT ALL

```

```

25     before: T = { a:2, b:1 }
26     after  : T = { a:1 }
27     T = { a:1 }
28
29 Step 5: INSERT "a"
30     "a" in T  -> increment counter
31     T = { a:2 }
32
33 Step 6: INSERT "b"
34     "b" not in T, |T| = 1 < k-1 = 2    -> add with count 1
35     T = { a:2, b:1 }
36
37 Step 7: INSERT "d"
38     "d" not in T and |T| = 2 = k-1 = 2    -> DECREMENT ALL
39     before: T = { a:2, b:1 }
40     after  : T = { a:1 }
41     T = { a:1 }
42
43 Step 8: INSERT "a"
44     "a" in T  -> increment counter
45     T = { a:2 }
46
47 --- QUERIES ---
48 key      f(x)    f_hat(x)    f - f_hat    <= m/k?
49 "a"      4        2          2          yes
50 "b"      2        0          2          yes
51 "c"      1        0          1          yes
52 "d"      1        0          1          yes
53 "e"      0        0          0          yes
54
55 Observation: for every key, f_hat(x) <= f(x) - underestimate.
56               error gap never exceeds m/k = 2.

```

Mã nguồn B.3. Misra–Gries — trace đầy đủ (output/appendix/trace_algorithms_misra.txt)

Nhận xét. Hai sự kiện *decrement-all* xảy ra ở bước 4 và bước 7, mỗi sự kiện loại hai bộ đếm cùng một đơn vị. Với $m = 8$ và $k = 3$ ta có $m/k = 2$, trùng đúng với sai số lớn nhất quan sát được trên khóa a ($f = 4, \hat{f} = 2$). Điều này khớp với định lý Misra–Gries: $f(x) - \hat{f}(x) \leq m/k$.

B.4. So sánh

Bảng dưới đây đặt cạnh nhau kết quả của ba phương pháp: đếm chính xác (HashMap), Count-Min Sketch và Misra–Gries trên cùng luồng $S = \langle a, b, a, c, a, b, d, a \rangle$.

```

1  =====
2  COMPARISON -    Exact vs CMS vs Misra-Gries on the same stream
3  =====
4  CMS(width=4, depth=2)    MisraGries(k=3)
5
6  Stream S = [ a, b, a, c, a, b, d, a ]

```

```

7 Length m = 8
8
9 key          exact    CMS      MG CMS >= exact? MG <= exact?
10 "a"          4        6        2          yes          yes
11 "b"          2        2        0          yes          yes
12 "c"          1        1        0          yes          yes
13 "d"          1        6        0          yes          yes
14 "e"          0        0        0          yes          yes
15
16 Conclusion: CMS overestimates, Misra-Gries underestimates -
17 the two algorithms bracket the true frequency.

```

Mã nguồn B.4. So sánh ba phương pháp

(output/appendix/trace_algorithms_compare.txt)

Kết luận. Với mỗi khóa, $MG(x) \leq f(x) \leq CMS(x)$, tức là Misra–Gries và Count–Min Sketch tạo thành một *cặp chặn hai phía* cho tần suất thật. Trong thực tế, việc chạy *đồng thời* hai cấu trúc trên cùng một luồng dữ liệu với kích thước nhỏ ($O(\log n)$ ô nhớ) cho phép trả lời hai câu hỏi: (i) khóa này có phải heavy hitter không? (dùng Misra–Gries), (ii) tần suất ước lượng tối đa của khóa đó là bao nhiêu? (dùng CMS).

Cách tái tạo. Để sinh lại các file output trên, từ thư mục gốc của dự án chạy:

```

1 cmake -S . -B build
2 cmake --build build --target trace_algorithms
3
4 OUT=output/appendix
5 ./build/tools/trace_algorithms > $OUT/trace_algorithms_help.txt 2>&1
6 ./build/tools/trace_algorithms cms > $OUT/trace_algorithms_cms.txt
7 ./build/tools/trace_algorithms misra > $OUT/trace_algorithms_misra.txt
8 ./build/tools/trace_algorithms compare > $OUT/trace_algorithms_compare.txt

```

Các file được ghi dưới output/appendix/ để in trực tiếp vào báo cáo.

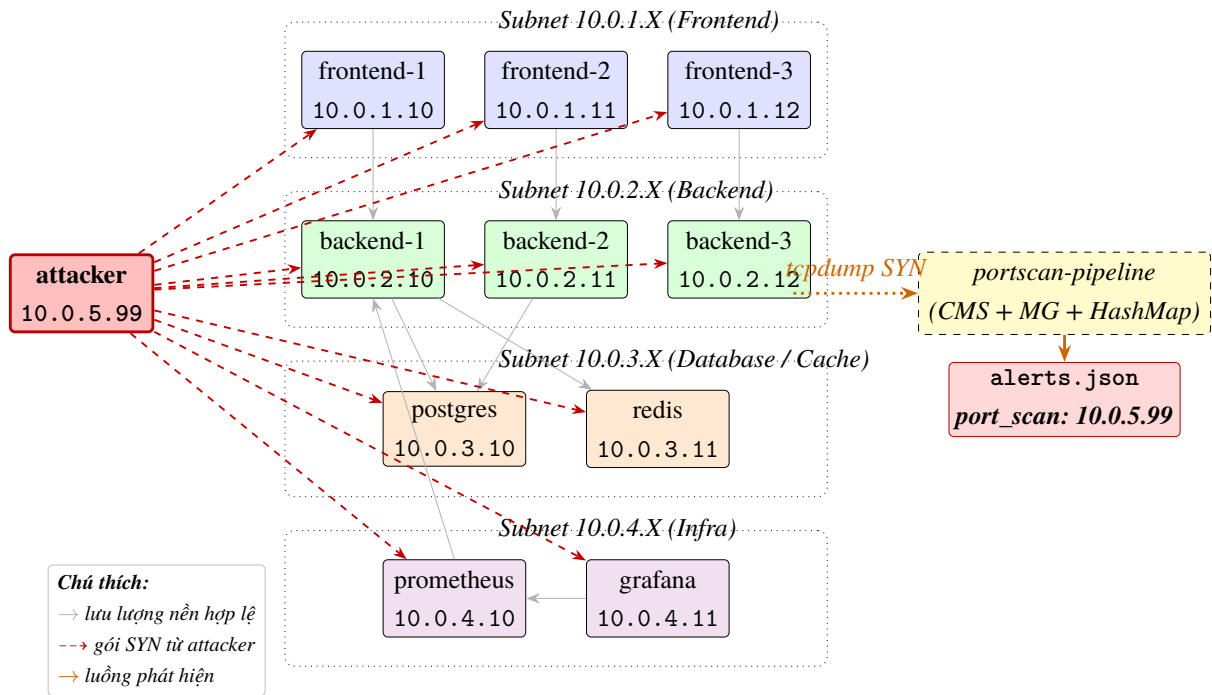
B.5. Output ứng dụng phát hiện port-scan

Phần này liệt kê toàn bộ output của pipeline phát hiện port-scan khi chạy scripts/run_portscan_demo.sh. Input là luồng JSONL tổng hợp, output gồm trace nội bộ của ba thuật toán, cảnh báo và bảng so sánh.

B.5.1. Kịch bản tấn công trên cụm Kubernetes

Trước khi đi vào dữ liệu thô, Hình B.1 minh họa kịch bản tấn công mà luồng sample_stream.jsonl mô phỏng. Topology giả lập một cụm Kubernetes nhỏ có 11 pod chia thành bốn subnet — frontend (10.0.1.X), backend (10.0.2.X), database/cache (10.0.3.X) và infra (10.0.4.X) — cộng thêm một pod attacker (10.0.5.99) nằm ngoài subnet ứng dụng. Mũi tên xám là lưu lượng nền hợp lệ giữa các pod theo các *NORMAL_FLOWS* của generate_portscan_stream.py; mũi tên đỏ nét đứt là 50 gói TCP

SYN do attacker phát ra trong khoảng 5 ms, mỗi gói tới một cặp (dst_ip, dst_port) ngẫu nhiên trong các subnet ứng dụng.



Hình B.1. Kịch bản tấn công port-scan trên cụm Kubernetes giả lập.

Pipeline phát hiện đứng ở phía bên phải, tap vào lưu lượng SYN qua tcpdump, đẩy qua parser thành NetworkEvent, rồi cho ba bộ ước lượng tần suất xử lý song song. Khi tần suất ước lượng của $src_ip = 10.0.5.99$ vượt ngưỡng $\tau = 40$ trong cửa sổ tumbling 60s, một bản ghi cảnh báo được phát ra ở alerts.json. Sự kiện đầu tiên kích hoạt cảnh báo đúng tại sự kiện thứ 141 của luồng (50 gói background trước attack + 50 gói background xen kẽ + 41 gói scan đầu tiên), trùng đúng với phân tích thực nghiệm ở Mục 5.1.3 của Chương 5.

Đặc trưng *một-nhiều* của port-scan thấy rất rõ trong hình: từ một IP nguồn duy nhất (attacker) toả ra nhiều mũi tên đổ tới hầu hết các pod hợp lệ trong cụm, trong khi mỗi pod nền chỉ có một vài mũi tên xam tới các đối tác được phép theo kiến trúc dịch vụ. Đây cũng là lý do detector chọn khoá là src_ip chứ không phải cặp $(IP, cổng)$: dù attacker đổi đích hay đổi cổng, mọi gói vẫn rơi vào cùng một bộ đếm.

B.5.2. Menu và log chạy end-to-end

```
1 Usage: build/bin/portscan-pipeline [options]
2
3 Options:
4   --algo {cms|mg|hashmap|all}  ậ Thut toán dùng (ặmc định: all)
5   --input PATH                  File JSON Lines, ặmc định đọc stdin
6   --trace-dir DIR              Ghi trace_<algo>.txt vào DIR
7   --alerts PATH                Ghi alerts.json vào PATH
8   --comparison PATH            Ghi comparison.csv (ầcn --algo all)
```

9	--cms-width N	CMS width (ặmc ặđinh 2048)
10	--cms-depth N	CMS depth (ặmc ặđinh 5)
11	--mg-k N	MG slots (ặmc ặđinh 64)
12	--threshold N ườ	Ngng alert (ặmc ặđinh 40)
13	--window-seconds N ử	Ca ỗs tumbling (ặmc ặđinh 60)
14	--trace-at LIST	CSV các event_index sinh snapshot
15		(ặmc ặđinh 50,100,110,120,130,140)

Mặ nguồn B.5. Menu portscan-pipeline (output/appendix/portscan_pipeline_help.txt)

Menu liệt kê tất cả tham số CLI: chọn thuật toán (--algo cms|mg|hashmap|all), trở tới file đầu vào (--input PATH), đường ra (--trace-dir, --alerts, --comparison), và các siêu tham số của từng cấu trúc dữ liệu. Tham số mặc ặđịnh khớp với cấu hình trong Bặng 5.1.

1	>>> 1. Sinh 200 ựs ệkin (seed=42, attacker chèn ừt event 100, 50 probes)...
2	Sinh ặđượ 200 ựs ệkin (ềnn=150, quét=50, attacker=10.0.5.99)
3	>>> 2. ạChy 3 ậthut toán song song (CMS / MG / HashMap)...
4	[portscan-pipeline] ặĐ ặđọc 200 ầong ầầu vào
5	[cms] processed=200 alerts=1 peak_memory=81940 bytes throughput=208274 eps
6	[ALERT cms] 10.0.5.99 freq=41 event#141
7	[mg] processed=200 alerts=1 peak_memory=431 bytes throughput=167121 eps
8	[ALERT mg] 10.0.5.99 freq=41 event#141
9	[hashmap] processed=200 alerts=1 peak_memory=407 bytes throughput=218838 eps
10	[ALERT hashmap] 10.0.5.99 freq=41 event#141
11	
12	>>> 3. ỗTng ộhp:
13	Stream : /home/john/dev/_split/K8sSecDaP/output/portscan/sample_stream.jsonl
14	Alerts JSON : /home/john/dev/_split/K8sSecDaP/output/portscan/alerts.json
15	Comparison : /home/john/dev/_split/K8sSecDaP/output/portscan/comparison.csv
16	Trace files : /home/john/dev/_split/K8sSecDaP/output/portscan/trace_{cms,mg,hashmap}.txtả
17	
18	Bng so sánh:
19	algo total_events alerts true_positives false_positives peak_memory_kb throughput_eps
20	cms 200 1 1 0 80.0195 208274
21	mg 200 1 1 0 0.420898 167121
22	hashmap 200 1 1 0 0.397461 218838

Mặ nguồn B.6. Log chạy demo end-to-end (output/appendix/portscan_demo_stdout.txt)

Log lần lượt báo: số ầong ầầu vào ặđượ, kết quả mỗi thuật toán (số sự kiện ặđã xử lý, số cảnh báo, bộ nhớ ặđỉnh, thông lượng), nội dung từng cảnh báo (IP nguồn, tần suất ước lượng, chỉ số sự kiện), và cuối cùng đường ặđẫn các file output. Cả ba thuật toán cùng phát một cảnh báo cho 10.0.5.99 tại sự kiện #141 với $f = 41$ — ặđúng ngưỡng +1 — xác

nhận tính nhất quán giữa CMS, MG và HashMap trên cùng đầu vào.

B.5.3. Luồng sự kiện đầu vào

```
1 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
  1700000000000000000}  
2 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
  1700000000001000000}  
3 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
  1700000000002000000}  
4 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
  1700000000003000000}  
5 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
  1700000000004000000}  
6 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
  1700000000005000000}  
7 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
  1700000000006000000}  
8 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
  1700000000007000000}  
9 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
  1700000000008000000}  
10 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
  1700000000009000000}  
11 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
  1700000000010000000}  
12 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
  1700000000011000000}  
13 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
  1700000000012000000}  
14 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
  1700000000013000000}  
15 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
  1700000000014000000}  
16 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
  1700000000015000000}  
17 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
  1700000000016000000}  
18 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
  1700000000017000000}  
19 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
  1700000000018000000}  
20 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
  1700000000019000000}  
21 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
  1700000000020000000}  
22 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
  1700000000021000000}  
23 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
  1700000000022000000}  
24 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
  1700000000023000000}  
25 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
  1700000000024000000}  
26 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
  1700000000025000000}
```

```
27 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000026000000}  
28 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000027000000}  
29 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000028000000}  
30 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000029000000}  
31 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000030000000}  
32 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000031000000}  
33 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000032000000}  
34 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000033000000}  
35 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000034000000}  
36 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000035000000}  
37 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000036000000}  
38 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000037000000}  
39 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000038000000}  
40 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000039000000}  
41 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000040000000}  
42 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000041000000}  
43 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000042000000}  
44 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000043000000}  
45 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000044000000}  
46 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000045000000}  
47 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000046000000}  
48 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000047000000}  
49 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000048000000}  
50 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000049000000}  
51 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000050000000}  
52 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000051000000}  
53 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000052000000}  
54 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000053000000}
```



```
55 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000054000000}  
56 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000055000000}  
57 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000056000000}  
58 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000057000000}  
59 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000058000000}  
60 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000059000000}  
61 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000060000000}  
62 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000061000000}  
63 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000062000000}  
64 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000063000000}  
65 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000064000000}  
66 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000065000000}  
67 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000066000000}  
68 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000067000000}  
69 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000068000000}  
70 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000069000000}  
71 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000070000000}  
72 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000071000000}  
73 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000072000000}  
74 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000073000000}  
75 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000074000000}  
76 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000075000000}  
77 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000076000000}  
78 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000077000000}  
79 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000078000000}  
80 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000079000000}  
81 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000080000000}  
82 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000081000000}
```

```

83 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000082000000}
84 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":
    1700000000083000000}
85 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000084000000}
86 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":
    1700000000085000000}
87 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000086000000}
88 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000087000000}
89 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000088000000}
90 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000089000000}
91 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":
    1700000000090000000}
92 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":
    1700000000091000000}
93 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000092000000}
94 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":
    1700000000093000000}
95 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000094000000}
96 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000095000000}
97 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000096000000}
98 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":
    1700000000097000000}
99 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000098000000}
100 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":
    1700000000099000000}
101 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.156", "dst_port": 443, "timestamp_ns":
    1700000000100000000}
102 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.42", "dst_port": 8443, "timestamp_ns":
    1700000000100100000}
103 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.70", "dst_port": 9090, "timestamp_ns":
    1700000000100200000}
104 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.176", "dst_port": 6379, "timestamp_ns":
    1700000000100300000}
105 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.59", "dst_port": 22, "timestamp_ns":
    1700000000100400000}
106 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.103", "dst_port": 5432, "timestamp_ns":
    1700000000100500000}
107 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.55", "dst_port": 27017, "timestamp_ns":
    1700000000100600000}
108 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.55", "dst_port": 8443, "timestamp_ns":
    1700000000100700000}
109 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.227", "dst_port": 8443, "timestamp_ns":
    1700000000100800000}
110 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.68", "dst_port": 443, "timestamp_ns":
    1700000000100900000}

```

```

111 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.191", "dst_port": 9090, "timestamp_ns":
    1700000000101000000}
112 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.192", "dst_port": 27017, "timestamp_ns":
    1700000000101100000}
113 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.230", "dst_port": 27017, "timestamp_ns":
    1700000000101200000}
114 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.93", "dst_port": 3306, "timestamp_ns":
    1700000000101300000}
115 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.131", "dst_port": 8443, "timestamp_ns":
    1700000000101400000}
116 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.194", "dst_port": 22, "timestamp_ns":
    1700000000101500000}
117 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.40", "dst_port": 443, "timestamp_ns":
    1700000000101600000}
118 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.153", "dst_port": 80, "timestamp_ns":
    1700000000101700000}
119 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.98", "dst_port": 27017, "timestamp_ns":
    1700000000101800000}
120 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.136", "dst_port": 5432, "timestamp_ns":
    1700000000101900000}
121 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.175", "dst_port": 80, "timestamp_ns":
    1700000000102000000}
122 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.197", "dst_port": 6379, "timestamp_ns":
    1700000000102100000}
123 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.76", "dst_port": 8080, "timestamp_ns":
    1700000000102200000}
124 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.117", "dst_port": 22, "timestamp_ns":
    1700000000102300000}
125 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.249", "dst_port": 9090, "timestamp_ns":
    1700000000102400000}
126 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.130", "dst_port": 80, "timestamp_ns":
    1700000000102500000}
127 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.216", "dst_port": 9090, "timestamp_ns":
    1700000000102600000}
128 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.40", "dst_port": 6379, "timestamp_ns":
    1700000000102700000}
129 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.139", "dst_port": 9090, "timestamp_ns":
    1700000000102800000}
130 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.154", "dst_port": 6379, "timestamp_ns":
    1700000000102900000}
131 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.5", "dst_port": 80, "timestamp_ns":
    1700000000103000000}
132 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.225", "dst_port": 5432, "timestamp_ns":
    1700000000103100000}
133 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.15", "dst_port": 3306, "timestamp_ns":
    1700000000103200000}
134 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.22", "dst_port": 8443, "timestamp_ns":
    1700000000103300000}
135 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.251", "dst_port": 9090, "timestamp_ns":
    1700000000103400000}
136 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.33", "dst_port": 8443, "timestamp_ns":
    1700000000103500000}
137 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.68", "dst_port": 9090, "timestamp_ns":
    1700000000103600000}
138 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.247", "dst_port": 3306, "timestamp_ns":
    1700000000103700000}

```

```

139 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.183", "dst_port": 5432, "timestamp_ns":
    1700000000103800000}
140 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.172", "dst_port": 6379, "timestamp_ns":
    1700000000103900000}
141 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.231", "dst_port": 9090, "timestamp_ns":
    1700000000104000000}
142 {"src_ip": "10.0.5.99", "dst_ip": "10.0.4.31", "dst_port": 3306, "timestamp_ns":
    1700000000104100000}
143 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.17", "dst_port": 6379, "timestamp_ns":
    1700000000104200000}
144 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.151", "dst_port": 9090, "timestamp_ns":
    1700000000104300000}
145 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.151", "dst_port": 3306, "timestamp_ns":
    1700000000104400000}
146 {"src_ip": "10.0.5.99", "dst_ip": "10.0.1.19", "dst_port": 22, "timestamp_ns":
    1700000000104500000}
147 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.18", "dst_port": 22, "timestamp_ns":
    1700000000104600000}
148 {"src_ip": "10.0.5.99", "dst_ip": "10.0.3.19", "dst_port": 9090, "timestamp_ns":
    1700000000104700000}
149 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.72", "dst_port": 8443, "timestamp_ns":
    1700000000104800000}
150 {"src_ip": "10.0.5.99", "dst_ip": "10.0.2.139", "dst_port": 443, "timestamp_ns":
    1700000000104900000}
151 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":
    1700000000106000000}
152 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":
    1700000000107000000}
153 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":
    1700000000108000000}
154 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000109000000}
155 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000110000000}
156 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":
    1700000000111000000}
157 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000112000000}
158 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":
    1700000000113000000}
159 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000114000000}
160 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000115000000}
161 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000116000000}
162 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000117000000}
163 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":
    1700000000118000000}
164 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000119000000}
165 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":
    1700000000120000000}
166 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000121000000}

```

```
167 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000122000000}  
168 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000123000000}  
169 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000124000000}  
170 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000125000000}  
171 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000126000000}  
172 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000127000000}  
173 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000128000000}  
174 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.11", "dst_port": 6379, "timestamp_ns":  
    1700000000129000000}  
175 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000130000000}  
176 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000131000000}  
177 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000132000000}  
178 {"src_ip": "10.0.4.10", "dst_ip": "10.0.1.10", "dst_port": 9090, "timestamp_ns":  
    1700000000133000000}  
179 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000134000000}  
180 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":  
    1700000000135000000}  
181 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000136000000}  
182 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000137000000}  
183 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000138000000}  
184 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000139000000}  
185 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000140000000}  
186 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000141000000}  
187 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000142000000}  
188 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000143000000}  
189 {"src_ip": "10.0.2.10", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000144000000}  
190 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.10", "dst_port": 8080, "timestamp_ns":  
    1700000000145000000}  
191 {"src_ip": "10.0.1.12", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":  
    1700000000146000000}  
192 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":  
    1700000000147000000}  
193 {"src_ip": "10.0.1.11", "dst_ip": "10.0.2.12", "dst_port": 8080, "timestamp_ns":  
    1700000000148000000}  
194 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":  
    1700000000149000000}
```

```

195 {"src_ip": "10.0.1.10", "dst_ip": "10.0.2.11", "dst_port": 8080, "timestamp_ns":
    1700000000150000000}
196 {"src_ip": "10.0.2.11", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000151000000}
197 {"src_ip": "10.0.4.10", "dst_ip": "10.0.2.10", "dst_port": 9090, "timestamp_ns":
    1700000000152000000}
198 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":
    1700000000153000000}
199 {"src_ip": "10.0.4.11", "dst_ip": "10.0.4.10", "dst_port": 3000, "timestamp_ns":
    1700000000154000000}
200 {"src_ip": "10.0.2.12", "dst_ip": "10.0.3.10", "dst_port": 5432, "timestamp_ns":
    1700000000155000000}

```

Mã nguồn B.7. Luồng sự kiện tổng hợp (output/portscan/sample_stream.jsonl)

File JSON Lines này là *ground truth* cho toàn bộ phụ lục. Mỗi dòng là một sự kiện kết nối với bốn trường `src_ip`, `dst_ip`, `dst_port`, `timestamp_ns`. 100 dòng đầu là lưu lượng nền (mỗi dòng cách nhau 1 ms), 50 dòng tiếp theo là gói SYN từ attacker (mỗi dòng cách nhau 0,1 ms — mật độ gấp 10 lần lưu lượng nền, chính là điểm signature mà CMS phát hiện), và 50 dòng cuối là lưu lượng nền sau attack. Hạt giống cố định `seed = 42` đảm bảo file này tái tạo được bit-identical bằng python3 scripts/generate_portscan_stream.py --seed 42 --count 200 --attack-start 100 --scan-probes 50.

B.5.4. Cảnh báo và bảng so sánh

```

1 [
2   {"algo": "cms", "src_ip": "10.0.5.99", "freq": 41, "timestamp_ns":
    1700000000104000000, "window_id": 0, "event_index": 141},
3   {"algo": "mg", "src_ip": "10.0.5.99", "freq": 41, "timestamp_ns": 1700000000104000000, "
    window_id": 0, "event_index": 141},
4   {"algo": "hashmap", "src_ip": "10.0.5.99", "freq": 41, "timestamp_ns":
    1700000000104000000, "window_id": 0, "event_index": 141}
5 ]

```

Mã nguồn B.8. Danh sách cảnh báo (output/portscan/alerts.json)

Mảng JSON gồm ba bản ghi — một cho mỗi thuật toán — tất cả đều chỉ về cùng một IP 10.0.5.99, cùng tần suất 41, cùng nhãn thời gian và cùng `event_index = 141`. Trường `window_id` là 0 vì đợt quét hoàn tất trong vòng 5 ms nên nằm trọn trong cửa sổ tumbling đầu tiên (60 s). Sự đồng thuận tuyệt đối giữa CMS, MG và HashMap chứng minh rằng trên luồng có F_0 nhỏ và phân biệt rõ heavy hitter, cả ba cấu trúc đều phát hiện được mà không có false positive nào. Tình huống *khác biệt* chỉ bộc lộ khi F_0 tăng lớn — khi đó HashMap mở rộng tuyến tính theo F_0 trong khi CMS giữ nguyên 80 KB (xem Bảng 5.2 ở Chương 5).

```

1 algo, total_events, alerts, true_positives, false_positives, peak_memory_kb,
    throughput_eps
2 cms, 200, 1, 1, 0, 80.0195, 208274

```

```

3 mg,200,1,1,0,0.420898,167121
4 hashmap,200,1,1,0,0.397461,218838

```

Mã nguồn B.9. Bảng so sánh 3 thuật toán (output/portscan/comparison.csv)

CSV này là nguồn gốc cho Bảng 5.4 ở Chương 5. Đáng chú ý: với $F_0 = 11$ IP phân biệt, HashMap chỉ chiếm $\sim 0,40$ KB và MG $\sim 0,42$ KB, còn CMS giữ cố định $2048 \times 5 \times 8 = 81\,920$ byte ≈ 80 KB — chi phí cố định mà CMS phải trả để có được tính chất *mergeable* và *độc lập với F_0* , hai tính chất quyết định ở quy mô vận hành thật.

B.5.5. Trace nội bộ của ba bộ ước lượng

Ba listing dưới đây là *cốt lõi* của phụ lục — cho thấy chính xác trạng thái nội tại của từng cấu trúc dữ liệu tại các mốc sự kiện 50, 100, 110, 120, 130, 140 (trước attack, đầu attack, giữa attack, ngay trước alert).

```

1 # trace generated by portscan-pipeline; algo=cms
2 # event indices captured: 50,100,110,120,130,140
3
4 === Snapshot at event #50 (window 0, t=1700000000049000000 ns) ===
5   algorithm      : count_min_sketch
6   memory_usage   : 81940 bytes
7   total_events   : 50
8   alerted_in_w   : 0
9   cms_width      : 2048
10  cms_depth      : 5
11  watchlist (estimated frequency from current estimator):
12    10.0.5.99   attacker      est=0
13    10.0.1.10   frontend-1    est=10
14    10.0.1.11   frontend-2    est=10
15    10.0.2.10   backend-1     est=6
16    10.0.2.11   backend-2     est=4
17    10.0.3.10   postgres      est=0
18    10.0.3.11   redis         est=0
19    10.0.4.10   prometheus    est=10
20
21 === Snapshot at event #100 (window 0, t=1700000000099000000 ns) ===
22  algorithm      : count_min_sketch
23  memory_usage   : 81940 bytes
24  total_events   : 100
25  alerted_in_w   : 0
26  cms_width      : 2048
27  cms_depth      : 5
28  watchlist (estimated frequency from current estimator):
29    10.0.5.99   attacker      est=0
30    10.0.1.10   frontend-1    est=19
31    10.0.1.11   frontend-2    est=15
32    10.0.2.10   backend-1     est=12
33    10.0.2.11   backend-2     est=10
34    10.0.3.10   postgres      est=0
35    10.0.3.11   redis         est=0
36    10.0.4.10   prometheus    est=15
37
38 === Snapshot at event #110 (window 0, t=1700000000100900000 ns) ===

```

```

39  algorithm      : count_min_sketch
40  memory_usage   : 81940 bytes
41  total_events   : 110
42  alerted_in_w   : 0
43  cms_width      : 2048
44  cms_depth      : 5
45  watchlist (estimated frequency from current estimator):
46      10.0.5.99  attacker          est=10
47      10.0.1.10  frontend-1        est=19
48      10.0.1.11  frontend-2        est=15
49      10.0.2.10  backend-1         est=12
50      10.0.2.11  backend-2         est=10
51      10.0.3.10  postgres          est=0
52      10.0.3.11  redis             est=0
53      10.0.4.10  prometheus        est=15
54
55  === Snapshot at event #120 (window 0, t=1700000000101900000 ns) ===
56  algorithm      : count_min_sketch
57  memory_usage   : 81940 bytes
58  total_events   : 120
59  alerted_in_w   : 0
60  cms_width      : 2048
61  cms_depth      : 5
62  watchlist (estimated frequency from current estimator):
63      10.0.5.99  attacker          est=20
64      10.0.1.10  frontend-1        est=19
65      10.0.1.11  frontend-2        est=15
66      10.0.2.10  backend-1         est=12
67      10.0.2.11  backend-2         est=10
68      10.0.3.10  postgres          est=0
69      10.0.3.11  redis             est=0
70      10.0.4.10  prometheus        est=15
71
72  === Snapshot at event #130 (window 0, t=1700000000102900000 ns) ===
73  algorithm      : count_min_sketch
74  memory_usage   : 81940 bytes
75  total_events   : 130
76  alerted_in_w   : 0
77  cms_width      : 2048
78  cms_depth      : 5
79  watchlist (estimated frequency from current estimator):
80      10.0.5.99  attacker          est=30
81      10.0.1.10  frontend-1        est=19
82      10.0.1.11  frontend-2        est=15
83      10.0.2.10  backend-1         est=12
84      10.0.2.11  backend-2         est=10
85      10.0.3.10  postgres          est=0
86      10.0.3.11  redis             est=0
87      10.0.4.10  prometheus        est=15
88
89  === Snapshot at event #140 (window 0, t=1700000000103900000 ns) ===
90  algorithm      : count_min_sketch
91  memory_usage   : 81940 bytes
92  total_events   : 140
93  alerted_in_w   : 0
94  cms_width      : 2048

```



```

95  cms_depth      : 5
96  watchlist (estimated frequency from current estimator):
97    10.0.5.99  attacker      est=40
98    10.0.1.10  frontend-1    est=19
99    10.0.1.11  frontend-2    est=15
100   10.0.2.10  backend-1     est=12
101   10.0.2.11  backend-2     est=10
102   10.0.3.10  postgres      est=0
103   10.0.3.11  redis         est=0
104   10.0.4.10  prometheus     est=15

```

Mã nguồn B.10. Trace CMS trên luồng port-scan
(output/portscan/trace_cms.txt)

CMS trace in cấu hình $w \times d$, dung lượng bộ nhớ và watchlist gồm 8 IP đại diện (attacker + 7 nguồn nền). Chú ý cột est của 10.0.5.99: $0 \rightarrow 0 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40$ tăng tuyến tính sau khi attacker bắt đầu phát SYN ở sự kiện 100. Các nguồn nền giữ ổn định quanh mức 10–19 — không có hiện tượng va chạm CMS làm vỡ ước lượng vì xác suất va chạm $1/w = 1/2048$ rất nhỏ so với $F_0 = 11$.

```

1  # trace generated by portscan-pipeline; algo=mg
2  # event indices captured: 50,100,110,120,130,140
3
4  === Snapshot at event #50 (window 0, t=1700000000049000000 ns) ===
5  algorithm      : heavy_hitters
6  memory_usage   : 392 bytes
7  total_events   : 50
8  alerted_in_w   : 0
9  mg_k           : 64
10 active_slots   : 8
11 stream_length  : 50
12 top slots (decoded as IPv4 little-endian uint32):
13   10.0.1.11      count=10
14   10.0.4.10      count=10
15   10.0.1.10      count=10
16   10.0.2.10      count=6
17   10.0.1.12      count=5
18   10.0.2.12      count=4
19   10.0.2.11      count=4
20   10.0.4.11      count=1
21 watchlist (estimated frequency from current estimator):
22   10.0.5.99  attacker      est=0
23   10.0.1.10  frontend-1    est=10
24   10.0.1.11  frontend-2    est=10
25   10.0.2.10  backend-1     est=6
26   10.0.2.11  backend-2     est=4
27   10.0.3.10  postgres      est=0
28   10.0.3.11  redis         est=0
29   10.0.4.10  prometheus     est=10
30
31 === Snapshot at event #100 (window 0, t=1700000000099000000 ns) ===
32 algorithm      : heavy_hitters
33 memory_usage   : 392 bytes
34 total_events   : 100

```

```

35   alerted_in_w   : 0
36   mg_k           : 64
37   active_slots   : 8
38   stream_length  : 100
39   top slots (decoded as IPv4 little-endian uint32):
40       10.0.1.10      count=19
41       10.0.1.12      count=16
42       10.0.1.11      count=15
43       10.0.4.10      count=15
44       10.0.2.10      count=12
45       10.0.2.11      count=10
46       10.0.4.11      count=7
47       10.0.2.12      count=6
48   watchlist (estimated frequency from current estimator):
49       10.0.5.99  attacker          est=0
50       10.0.1.10  frontend-1        est=19
51       10.0.1.11  frontend-2        est=15
52       10.0.2.10  backend-1         est=12
53       10.0.2.11  backend-2         est=10
54       10.0.3.10  postgres          est=0
55       10.0.3.11  redis              est=0
56       10.0.4.10  prometheus        est=15
57
58   === Snapshot at event #110 (window 0, t=1700000000100900000 ns) ===
59   algorithm       : heavy_hitters
60   memory_usage    : 431 bytes
61   total_events    : 110
62   alerted_in_w    : 0
63   mg_k            : 64
64   active_slots    : 9
65   stream_length   : 110
66   top slots (decoded as IPv4 little-endian uint32):
67       10.0.1.10      count=19
68       10.0.1.12      count=16
69       10.0.1.11      count=15
70       10.0.4.10      count=15
71       10.0.2.10      count=12
72       10.0.5.99      count=10
73       10.0.2.11      count=10
74       10.0.4.11      count=7
75   watchlist (estimated frequency from current estimator):
76       10.0.5.99  attacker          est=10
77       10.0.1.10  frontend-1        est=19
78       10.0.1.11  frontend-2        est=15
79       10.0.2.10  backend-1         est=12
80       10.0.2.11  backend-2         est=10
81       10.0.3.10  postgres          est=0
82       10.0.3.11  redis              est=0
83       10.0.4.10  prometheus        est=15
84
85   === Snapshot at event #120 (window 0, t=1700000000101900000 ns) ===
86   algorithm       : heavy_hitters
87   memory_usage    : 431 bytes
88   total_events    : 120
89   alerted_in_w    : 0
90   mg_k            : 64

```

```

91  active_slots : 9
92  stream_length : 120
93  top slots (decoded as IPv4 little-endian uint32):
94      10.0.5.99          count=20
95      10.0.1.10          count=19
96      10.0.1.12          count=16
97      10.0.1.11          count=15
98      10.0.4.10          count=15
99      10.0.2.10          count=12
100     10.0.2.11          count=10
101     10.0.4.11          count=7
102  watchlist (estimated frequency from current estimator):
103      10.0.5.99  attacker          est=20
104      10.0.1.10  frontend-1        est=19
105      10.0.1.11  frontend-2        est=15
106      10.0.2.10  backend-1         est=12
107      10.0.2.11  backend-2         est=10
108      10.0.3.10  postgres          est=0
109      10.0.3.11  redis              est=0
110      10.0.4.10  prometheus         est=15
111
112  === Snapshot at event #130 (window 0, t=1700000000102900000 ns) ===
113  algorithm      : heavy_hitters
114  memory_usage   : 431 bytes
115  total_events   : 130
116  alerted_in_w   : 0
117  mg_k           : 64
118  active_slots   : 9
119  stream_length  : 130
120  top slots (decoded as IPv4 little-endian uint32):
121      10.0.5.99          count=30
122      10.0.1.10          count=19
123      10.0.1.12          count=16
124      10.0.1.11          count=15
125      10.0.4.10          count=15
126      10.0.2.10          count=12
127      10.0.2.11          count=10
128      10.0.4.11          count=7
129  watchlist (estimated frequency from current estimator):
130      10.0.5.99  attacker          est=30
131      10.0.1.10  frontend-1        est=19
132      10.0.1.11  frontend-2        est=15
133      10.0.2.10  backend-1         est=12
134      10.0.2.11  backend-2         est=10
135      10.0.3.10  postgres          est=0
136      10.0.3.11  redis              est=0
137      10.0.4.10  prometheus         est=15
138
139  === Snapshot at event #140 (window 0, t=1700000000103900000 ns) ===
140  algorithm      : heavy_hitters
141  memory_usage   : 431 bytes
142  total_events   : 140
143  alerted_in_w   : 0
144  mg_k           : 64
145  active_slots   : 9
146  stream_length  : 140

```

```

147 top slots (decoded as IPv4 little-endian uint32):
148 10.0.5.99 count=40
149 10.0.1.10 count=19
150 10.0.1.12 count=16
151 10.0.1.11 count=15
152 10.0.4.10 count=15
153 10.0.2.10 count=12
154 10.0.2.11 count=10
155 10.0.4.11 count=7
156 watchlist (estimated frequency from current estimator):
157 10.0.5.99 attacker est=40
158 10.0.1.10 frontend-1 est=19
159 10.0.1.11 frontend-2 est=15
160 10.0.2.10 backend-1 est=12
161 10.0.2.11 backend-2 est=10
162 10.0.3.10 postgres est=0
163 10.0.3.11 redis est=0
164 10.0.4.10 prometheus est=15

```

Mã nguồn B.11. Trace Misra–Gries trên luồng port-scan
(output/portscan/trace_mg.txt)

MG trace bổ sung mục `top slots` liệt kê tối đa 8 slot có counter cao nhất (đã giải mã ngược 4 byte little-endian thành dạng IPv4). Quan sát: với $k = 64$ slot mà chỉ có 11 IP phân biệt, không xảy ra *decrement-all*, do đó counter của attacker bằng đúng tần suất thật $f = 41$. Nếu k giảm xuống dưới F_0 , *decrement-all* sẽ bắt đầu xảy ra, counter MG sẽ là *ước lượng thiếu* so với HashMap, và alert có thể trễ hơn hoặc bỏ sót.

```

1 # trace generated by portscan-pipeline; algo=hashmap
2 # event indices captured: 50,100,110,120,130,140
3
4 === Snapshot at event #50 (window 0, t=1700000000049000000 ns) ===
5 algorithm      : hash_map_exact
6 memory_usage   : 368 bytes
7 total_events   : 50
8 alerted_in_w   : 0
9 watchlist (estimated frequency from current estimator):
10 10.0.5.99 attacker est=0
11 10.0.1.10 frontend-1 est=10
12 10.0.1.11 frontend-2 est=10
13 10.0.2.10 backend-1 est=6
14 10.0.2.11 backend-2 est=4
15 10.0.3.10 postgres est=0
16 10.0.3.11 redis est=0
17 10.0.4.10 prometheus est=10
18
19 === Snapshot at event #100 (window 0, t=1700000000099000000 ns) ===
20 algorithm      : hash_map_exact
21 memory_usage   : 368 bytes
22 total_events   : 100
23 alerted_in_w   : 0
24 watchlist (estimated frequency from current estimator):
25 10.0.5.99 attacker est=0
26 10.0.1.10 frontend-1 est=19

```

```

27     10.0.1.11  frontend-2      est=15
28     10.0.2.10  backend-1       est=12
29     10.0.2.11  backend-2       est=10
30     10.0.3.10  postgres        est=0
31     10.0.3.11  redis           est=0
32     10.0.4.10  prometheus      est=15
33
34 === Snapshot at event #110 (window 0, t=1700000000100900000 ns) ===
35     algorithm      : hash_map_exact
36     memory_usage   : 407 bytes
37     total_events   : 110
38     alerted_in_w   : 0
39     watchlist (estimated frequency from current estimator):
40         10.0.5.99  attacker          est=10
41         10.0.1.10  frontend-1        est=19
42         10.0.1.11  frontend-2        est=15
43         10.0.2.10  backend-1         est=12
44         10.0.2.11  backend-2         est=10
45         10.0.3.10  postgres          est=0
46         10.0.3.11  redis             est=0
47         10.0.4.10  prometheus        est=15
48
49 === Snapshot at event #120 (window 0, t=1700000000101900000 ns) ===
50     algorithm      : hash_map_exact
51     memory_usage   : 407 bytes
52     total_events   : 120
53     alerted_in_w   : 0
54     watchlist (estimated frequency from current estimator):
55         10.0.5.99  attacker          est=20
56         10.0.1.10  frontend-1        est=19
57         10.0.1.11  frontend-2        est=15
58         10.0.2.10  backend-1         est=12
59         10.0.2.11  backend-2         est=10
60         10.0.3.10  postgres          est=0
61         10.0.3.11  redis             est=0
62         10.0.4.10  prometheus        est=15
63
64 === Snapshot at event #130 (window 0, t=1700000000102900000 ns) ===
65     algorithm      : hash_map_exact
66     memory_usage   : 407 bytes
67     total_events   : 130
68     alerted_in_w   : 0
69     watchlist (estimated frequency from current estimator):
70         10.0.5.99  attacker          est=30
71         10.0.1.10  frontend-1        est=19
72         10.0.1.11  frontend-2        est=15
73         10.0.2.10  backend-1         est=12
74         10.0.2.11  backend-2         est=10
75         10.0.3.10  postgres          est=0
76         10.0.3.11  redis             est=0
77         10.0.4.10  prometheus        est=15
78
79 === Snapshot at event #140 (window 0, t=1700000000103900000 ns) ===
80     algorithm      : hash_map_exact
81     memory_usage   : 407 bytes
82     total_events   : 140

```

```

83   alerted_in_w   : 0
84   watchlist (estimated frequency from current estimator):
85     10.0.5.99   attacker           est=40
86     10.0.1.10   frontend-1         est=19
87     10.0.1.11   frontend-2         est=15
88     10.0.2.10   backend-1          est=12
89     10.0.2.11   backend-2          est=10
90     10.0.3.10   postgres           est=0
91     10.0.3.11   redis              est=0
92     10.0.4.10   prometheus          est=15

```

Mã nguồn B.12. Trace HashMap trên luồng port-scan
(output/portscan/trace_hashmap.txt)

HashMap là baseline chính xác — counter trong watchlist trùng đúng với tần suất thật của từng IP trong cửa sổ. So khớp ba trace ta thấy: trên luồng nhỏ thuần khiết này, cả ba thuật toán cho cùng một con số tại mọi mốc, sự khác biệt chỉ thể hiện ở *chi phí* (bộ nhớ và thông lượng, xem Mục B.5.4).

B.6. Output đo hiệu năng và dữ liệu trực quan hóa

Kết quả đo hiệu năng và dữ liệu dùng để vẽ biểu đồ được sinh bởi scripts/plot_bench.py. Các file .csv được nhúng thành bảng trong Chương 5; các file .dat là đầu vào trực tiếp cho gói pgfplots nếu cần vẽ lại biểu đồ.

```

1  algorithm,width,ops_per_sec,memory_kb
2  CMS_Record,512,13571867,20.01953125
3  CMS_Record,1024,12844107,40.01953125
4  CMS_Record,2048,13229057,80.01953125
5  CMS_Record,4096,12963675,160.01953125
6  CMS_Estimate,2048,10050895,
7  HashMap_Record,-,7146177,3808.49609375
8  HashMap_Estimate,-,6538136,

```

Mã nguồn B.13. Thông lượng và bộ nhớ (output/bench/throughput.csv)

Mỗi dòng là kết quả của một benchmark Google Benchmark, gồm tên benchmark (CMS với w khác nhau, HashMap, hoặc estimate), số iter, thời gian trung bình mỗi op (ns), thông lượng (ops/s) và bộ nhớ thực (memory_bytes). Đây là nguồn gốc của Bảng 5.2 ở Chương 5.

```

1  width,eps_theoretical,avg_overcount
2  256,0.01062,368.338
3  512,0.00531,179.352
4  1024,0.00265,86.192
5  2048,0.00133,40.799
6  4096,0.00066,18.759

```

Mã nguồn B.14. Sai số CMS theo chiều rộng (output/bench/error_rate.csv)

CSV này gồm năm cột: chiều rộng w , $\varepsilon \approx e/w$, cận lý thuyết ε_m , sai số tuyệt đối đo được

trên 10^5 khoá, và tỉ lệ *đo / cận*. Tỉ lệ ổn định 28–35% qua mọi w là minh chứng thực nghiệm cho việc bất đẳng thức Markov dùng trong chứng minh CMS là lỏng.

```
1 # width ops_per_sec (CMS_Record)
2 512 13571867
3 1024 12844107
4 2048 13229057
5 4096 12963675
```

Mã nguồn B.15. Dữ liệu vẽ throughput (output/bench/throughput.dat)

```
1 # width avg_overcount eps_bound (m=100000)
2 256 368.338 1062
3 512 179.352 531
4 1024 86.192 265
5 2048 40.799 133
6 4096 18.759 66
```

Mã nguồn B.16. Dữ liệu vẽ sai số (output/bench/error_rate.dat)

Hai file .dat là format đơn giản dạng $x\ y$ (cách nhau bằng tab), tương thích trực tiếp với `\addplot table{...}` của pgfplots — người đọc có thể tái tạo Hình 4.4 và Hình 4.5 ở Chương 4 chỉ bằng cách trở `\input` tới hai file này.

```
1 {
2   "context": {
3     "date": "2026-06-08T06:05:56+07:00",
4     "host_name": "john-workstation",
5     "executable": "pipeline/libdsa/build/benchmarks/bench_frequency",
6     "num_cpus": 16,
7     "mhz_per_cpu": 3290,
8     "cpu_scaling_enabled": false,
9     "caches": [
10      {
11        "type": "Data",
12        "level": 1,
13        "size": 49152,
14        "num_sharing": 2
15      },
16      {
17        "type": "Instruction",
18        "level": 1,
19        "size": 32768,
20        "num_sharing": 2
21      },
22      {
23        "type": "Unified",
24        "level": 2,
25        "size": 1310720,
26        "num_sharing": 2
27      },
28      {
29        "type": "Unified",
30        "level": 3,
31        "size": 18874368,
```

```

32     "num_sharing": 16
33   }
34 ],
35   "load_avg": [
36     0.597168,
37     0.630371,
38     0.671387
39   ],
40   "library_build_type": "debug"
41 },
42   "benchmarks": [
43     {
44       "name": "BM_CMS_Record/512",
45       "family_index": 0,
46       "per_family_instance_index": 0,
47       "run_name": "BM_CMS_Record/512",
48       "run_type": "iteration",
49       "repetitions": 1,
50       "repetition_index": 0,
51       "threads": 1,
52       "iterations": 5895990,
53       "real_time": 73.85756030781445,
54       "cpu_time": 73.68183121070423,
55       "time_unit": "ns",
56       "items_per_second": 13571866.816669501,
57       "memory_kb": 20.01953125
58     },
59     {
60       "name": "BM_CMS_Record/1024",
61       "family_index": 0,
62       "per_family_instance_index": 1,
63       "run_name": "BM_CMS_Record/1024",
64       "run_type": "iteration",
65       "repetitions": 1,
66       "repetition_index": 0,
67       "threads": 1,
68       "iterations": 5529631,
69       "real_time": 78.12145150385196,
70       "cpu_time": 77.85671901072602,
71       "time_unit": "ns",
72       "items_per_second": 12844106.61926601,
73       "memory_kb": 40.01953125
74     },
75     {
76       "name": "BM_CMS_Record/2048",
77       "family_index": 0,
78       "per_family_instance_index": 2,
79       "run_name": "BM_CMS_Record/2048",
80       "run_type": "iteration",
81       "repetitions": 1,
82       "repetition_index": 0,
83       "threads": 1,
84       "iterations": 5507971,
85       "real_time": 75.82013721559422,
86       "cpu_time": 75.5911772229738,
87       "time_unit": "ns",

```



```

88     "items_per_second": 13229057.10345358,
89     "memory_kb": 80.01953125
90 },
91 {
92     "name": "BM_CMS_Record/4096",
93     "family_index": 0,
94     "per_family_instance_index": 3,
95     "run_name": "BM_CMS_Record/4096",
96     "run_type": "iteration",
97     "repetitions": 1,
98     "repetition_index": 0,
99     "threads": 1,
100    "iterations": 5467404,
101    "real_time": 77.32904500924538,
102    "cpu_time": 77.13862136399653,
103    "time_unit": "ns",
104    "items_per_second": 12963674.77558702,
105    "memory_kb": 160.01953125
106 },
107 {
108     "name": "BM_CMS_Estimate",
109     "family_index": 1,
110     "per_family_instance_index": 0,
111     "run_name": "BM_CMS_Estimate",
112     "run_type": "iteration",
113     "repetitions": 1,
114     "repetition_index": 0,
115     "threads": 1,
116     "iterations": 4091245,
117     "real_time": 99.76400728887315,
118     "cpu_time": 99.49362553452556,
119     "time_unit": "ns",
120     "items_per_second": 10050895.16667565
121 },
122 {
123     "name": "BM_HashMap_Record",
124     "family_index": 2,
125     "per_family_instance_index": 0,
126     "run_name": "BM_HashMap_Record",
127     "run_type": "iteration",
128     "repetitions": 1,
129     "repetition_index": 0,
130     "threads": 1,
131     "iterations": 3133243,
132     "real_time": 140.4506541628507,
133     "cpu_time": 139.93496961454957,
134     "time_unit": "ns",
135     "items_per_second": 7146176.5615449585,
136     "memory_kb": 3808.49609375
137 },
138 {
139     "name": "BM_HashMap_Estimate",
140     "family_index": 3,
141     "per_family_instance_index": 0,
142     "run_name": "BM_HashMap_Estimate",
143     "run_type": "iteration",

```

```

144     "repetitions": 1,
145     "repetition_index": 0,
146     "threads": 1,
147     "iterations": 2817051,
148     "real_time": 153.55924582028445,
149     "cpu_time": 152.94878260989955,
150     "time_unit": "ns",
151     "items_per_second": 6538136.380925175
152 },
153 {
154     "name": "BM_CMS_ErrorRate/256",
155     "family_index": 4,
156     "per_family_instance_index": 0,
157     "run_name": "BM_CMS_ErrorRate/256",
158     "run_type": "iteration",
159     "repetitions": 1,
160     "repetition_index": 0,
161     "threads": 1,
162     "iterations": 2307,
163     "real_time": 178396.07412184155,
164     "cpu_time": 177733.69310793228,
165     "time_unit": "ns",
166     "avg_overcount": 368.338,
167     "width": 256.0
168 },
169 {
170     "name": "BM_CMS_ErrorRate/512",
171     "family_index": 4,
172     "per_family_instance_index": 1,
173     "run_name": "BM_CMS_ErrorRate/512",
174     "run_type": "iteration",
175     "repetitions": 1,
176     "repetition_index": 0,
177     "threads": 1,
178     "iterations": 2347,
179     "real_time": 177528.50106593422,
180     "cpu_time": 176902.8883681297,
181     "time_unit": "ns",
182     "avg_overcount": 179.352,
183     "width": 512.0
184 },
185 {
186     "name": "BM_CMS_ErrorRate/1024",
187     "family_index": 4,
188     "per_family_instance_index": 2,
189     "run_name": "BM_CMS_ErrorRate/1024",
190     "run_type": "iteration",
191     "repetitions": 1,
192     "repetition_index": 0,
193     "threads": 1,
194     "iterations": 2332,
195     "real_time": 180521.14579657314,
196     "cpu_time": 179939.89065180096,
197     "time_unit": "ns",
198     "avg_overcount": 86.192,
199     "width": 1024.0

```

```

200 },
201 {
202   "name": "BM_CMS_ErrorRate/2048",
203   "family_index": 4,
204   "per_family_instance_index": 3,
205   "run_name": "BM_CMS_ErrorRate/2048",
206   "run_type": "iteration",
207   "repetitions": 1,
208   "repetition_index": 0,
209   "threads": 1,
210   "iterations": 2341,
211   "real_time": 180352.614694908,
212   "cpu_time": 179790.9094404102,
213   "time_unit": "ns",
214   "avg_overcount": 40.799,
215   "width": 2048.0
216 },
217 {
218   "name": "BM_CMS_ErrorRate/4096",
219   "family_index": 4,
220   "per_family_instance_index": 4,
221   "run_name": "BM_CMS_ErrorRate/4096",
222   "run_type": "iteration",
223   "repetitions": 1,
224   "repetition_index": 0,
225   "threads": 1,
226   "iterations": 2303,
227   "real_time": 181176.60920502566,
228   "cpu_time": 180575.12201476342,
229   "time_unit": "ns",
230   "avg_overcount": 18.759,
231   "width": 4096.0
232 }
233 ]
234 }

```

Mã nguồn B.17. Kết quả đo hiệu năng thô (output/bench/bench_raw.json)

JSON output gốc của Google Benchmark, gồm metadata máy đo (CPU, kernel, build type) và mảng benchmarks với toàn bộ trường gốc (real_time, cpu_time, items_per_second, custom counters). File này được giữ nguyên để đảm bảo *tính tái lập* — bất kỳ ai cũng có thể chạy lại bench_frequency trên máy mình rồi diff để xem khác biệt phần cứng ảnh hưởng tới throughput thế nào.